

TUTORIAL FRAMEWORK CODEIGNITER 4

UNTUK PEMULA DAN IMPLEMENTASI

PADA APLIKASI UJIAN TOEFL

DISUSUN OLEH

Ach. Faisol S. Arifin

PEMBIMBING

Drs. Budi Soesilo, M.T.

2021

Jurusan Teknik Informatika
Fakultas Teknik
Universitas Trunojoyo Madura

Tahun 2021

LEMBAR KESEPAKATAN

LEMBAR KESEPAKATAN PEMBUATAN BUKU TUTORIAL FRAMEWORK CODEIGNITER 4

1. Nama Mahasiswa : Ach. Faisol S. Arifin (180411100073)
2. Program Studi : Teknik Informatika
3. Jenis : Buku Tutorial
4. Topik/Judul Buku : Tutorial Framework Codeigniter 4 untuk Pemula
5. Outline Buku :
 - Pengenalan Framework Codeigniter 4 dan Konsep MVC
 - Instalasi dan Konfigurasi Codeigniter
 - Struktur Folder dan Environment
 - Mengenal Routing, Controller, Views, dan Sistem Templeting
 - Koneksi Database dan Model
 - CRUD dengan Codeigniter
 - Upload File, Penanganan dan Validasi Form
 - Membuat Pagination dan Pencarian
 - Implementasi Codeigniter 4 (Membuat Aplikasi Ujian Toefl)
 - Upload Project Codeigniter ke Web Hosting

Bangkalan, 10 Maret 2021

Pengusul,
Mahasiswa,



Ach. Faisol S. Arifin
180411100073

Menyetujui,

Dosen Penguji



Imamah, S.Kom., M.Kom
NIP 198507212014042001

Dosen Pembimbing



Drs. Budi Soesilo, M.T.
NIP 19670531 200312 1 001

LEMBAR PENGESAHAN

Telah diperiksa dan diuji oleh :

Pada Tanggal :

Dengan Nilai :

Menyetujui,

Dosen Pembimbing

Drs. Budi Soesilo, M.T.

NIP 19670531 200312 1 001

Mengetahui,

Koordinator Kerja Praktek

Yoga Ddwitya Pramudita,S.Kom,M.Cs

NIP 19840413 200812 1 002

KATA PENGANTAR

Alhamdulillah, puji syukur kepada Allah SWT, atas segala limpahan rahmat dan karunianya yang diberikan kepada penulis, sehingga buku tutorial ***“Tutorial Framework CodeIgniter 4 untuk Pemula dan Implementasi pada Aplikasi Ujian Toefl”*** dapat penulis selesaikan sebagai syarat untuk menyelesaikan Mata Kuliah Kerja Praktek jurusan Teknik Informatika Universitas Trunojoyo Madura.

Setelah melalui perjuangan yang sangat melelahkan, dan telah menguras banyak waktu, pikiran, tenaga bahkan materi, akhirnya buku tutorial ini dapat penulis selesaikan berkat dukungan dari berbagai pihak yang telah memberikan bantuan moril maupun materiil. Untuk itu pada kesempatan ini penulis mengucapkan banyak terima kasih kepada:

1. Drs. Budi Soesilo, M.T. selaku dosen pembimbing yang telah bersedia meluangkan waktu untuk membimbing dan memberi saran dan masukan selama penyusunan buku tutorial ini.
2. Kedua orang tua dan keluarga yang telah memberikan do'a dan dukungan selama penyusunan buku tutorial ini
3. Semua teman-teman dan rekan seperjuangan, serta semua pihak yang mendukung yang tidak bisa penulis sebutkan satu persatu.

Atas keterbatasan pengetahuan dan pengalaman, penulis menyadari bahwa apa yang telah dihasilkan penulis dalam buku tutorial ini masih jauh dari kata sempurna. Oleh karena itu, penulis mengharapkan segala bentuk kritik dan saran dari pembaca demi kesempurnaan buku tutorial ini.

Akhir kata, semoga buku tutorial ini dapat bermanfaat semua pihak dibidang Teknik Informatika pada khususnya dan semua para pembaca pada umumnya.

Sumenep, 20 April 2021

Penulis

DAFTAR ISI

LEMBAR KESEPAKATAN	i
LEMBAR PENGESAHAN	iii
KATA PENGANTAR	iv
DAFTAR ISI	v
BAB 1	1
PENDAHULUAN.....	1
1.1. LATAR BELAKANG	1
1.2. TUJUAN	7
1.3. MANFAAT	7
BAB 2	9
STUDI PUSTAKA	9
2.1. INSTALASI CODEIGNITER SECARA MANUAL	9
2.2. INSTALASI CODEIGNITER MENGGUNAKAN COMPOSER ...	11
2.3. MENJALANKAN CODEIGNITER	15
BAB 3	18
STRUKTUR FOLDER DAN ENVIRONMENT	18
3.1. STRUKTUR FOLDER CODEIGNITER 4	18
3.2. MEMAHAMI ENVIRONMENT	21
BAB 4	26
MENGENAL CONTROLLER, ROUTING, VIEWS, DAN SISTEM TEMPLATING.....	26
4.1. MENGENAL DAN MEMBUAT CONTROLLER.....	26
4.2. MENGENAL DAN MELAKUKAN ROUTING URL	28
4.3. MENGENAL DAN MEMBUAT VIEWS.....	32

4.4.	MEMBUAT SISTEM TEMPLETING/LAYOUTS	35
BAB 5		37
	KONEKSI DATABASE MYSQL DAN MODEL	37
5.1.	APA ITU MYSQL ?	37
5.2.	MEMBUAT DATABASE MYSQL.....	38
5.3.	KONFIGURASI KONEKSI CODEIGNITER DENGAN MYSQL...	41
5.4.	MENGENAL DAN MEMBUAT MODEL SEDERHANA	42
BAB 6		48
	CRUD DENGAN CODEIGNITER 4	48
6.1.	MENAMPILKAN DATA DARI DATABASE.....	48
6.2.	MEMBUAT INSERT DATA	52
6.3.	MEMBUAT DELETE DATA.....	56
6.4.	MEMBUAT UPDATE DATA	58
BAB 7		64
	PAGINATION DAN PENCARIAN DATA	64
7.1.	MEMBUAT PAGINATION.....	64
7.2.	MEMBUAT PENCARIAN DATA	68
BAB 8		71
	UPLOAD FILE, PENANGANAN DAN VALIDASI FORM.....	71
8.1.	MENGENAL VALIDASI FORM	71
8.2.	MEMBUAT VALIDASI FORM.....	74
8.3.	MENGENAL VALIDASI UNTUK UPLOAD FILE.....	81
8.4.	MEMBUAT DAN MEMVALIDASI FORM UPLOAD FILE.....	81
BAB 9		85

IMPLEMENTASI CODEIGNITER 4 (MEMBUAT APLIKASI UJIAN TOEFL SEDERHANA).....	85
9.1. RANCANGAN APLIKASI	85
9.2. MEMBUAT DATABASE APLIKASI	86
9.3. MEMBUAT HALAMAN UJIAN.....	88
9.3.1. Sistem Layouting.....	88
9.3.2. Halaman Login	90
9.3.3. Halaman Home	95
9.3.4. Halaman Section.....	98
9.3.5. Halaman Direction.....	103
9.3.6. Halaman Exam	105
9.3.7. Halaman Finish	120
9.3.8. Halaman Score.....	123
9.4. MEMBUAT HALAMAN ADMIN.....	126
9.4.1. Data Peserta	126
9.4.2. Hapus Peserta.....	130
9.4.3. Tambah Peserta.....	130
9.4.4. Edit Peserta.....	133
9.4.5. Data Section.....	136
9.4.6. Edit Section.....	138
9.4.7. Data Artikel Reading	141
9.4.8. Hapus Artikel Reading	144
9.4.9. Tambah Artikel Reading	145
9.4.10. Edit Artikel Reading	147
9.4.11. Data Audio Listening.....	150

9.4.12.	Hapus Audio Listening	153
9.4.13.	Tambah Sound Listening	154
9.4.14.	Edit Audio Listening	156
9.4.15.	Data Soal	159
9.4.16.	Hapus Soal	162
9.4.17.	Tambah Soal	162
9.4.18.	Edit Soal	167
9.4.19.	Hasil Toefl	172
BAB 10		175
	UPLOAD PROJECT CODEIGNITER KE WEB HOSTING	175
10.1.	MENDAFTAR LAYANAN WEB HOSTING	175
10.2.	IMPORT DATABASE KE WEB HOSTING	177
10.3.	INSTALASI DAN KONFIGURASI PROJECT	179
DAFTAR PUSTAKA		183

BAB 1

PENDAHULUAN

1.1. LATAR BELAKANG

Aplikasi web menjadi sistem yang populer digunakan saat ini, seperti untuk kebutuhan e-commerce, e-government, e-learning, maupun sebagai program kompleks lainnya untuk kebutuhan tertentu. Dalam pengembangan aplikasi web besar, pastinya kita akan membutuhkan sarana yang dapat mempermudah dalam pengembangan aplikasi, mulai dari pengelolaan database, hingga keamanan aplikasi. Salah satu solusi yang dapat kita gunakan adalah menggunakan framework dari bahasa pemrograman tertentu. Framework adalah sebuah kerangka kerja yang dapat digunakan seorang developer untuk mempermudah dalam mengembangkan sebuah aplikasi. Didalam sebuah framework telah disediakan fungsi-fungsi yang dapat kita gunakan sesuai kebutuhan.

Dalam mengembangkan aplikasi web, kita dapat menggunakan framework-framework populer dan terkenal. Seperti halnya CodeIgniter, Laravel, Yii, CakePHP, dll. Dalam pembahasan ini, kita akan belajar tentang framework CodeIgniter 4 dan mengimplementasikannya pada aplikasi ujian toefl.

CodeIgniter adalah sebuah framework opensource berbasis PHP untuk mengembangkan aplikasi web yang dinamis[1]. Pada tanggal 28 Februari 2006 framework ini pertama kali mulai diperkenalkan. Sejak framework ini rilis terus mengalami perkembangan mulai dari versi 2, ke versi 3, dan versi saat ini versi 4 dengan fitur dan keandalannya semakin meningkat. Dengan adanya framework ini, seorang developer dimungkinkan akan mengembangkan project jauh lebih cepat, dari pada harus

menuliskan code dari awal. CodeIgniter telah menyediakan kumpulan library untuk tugas tertentu yang umumnya sering digunakan. CodeIgniter juga *multi platform* yang mendukung berbagai sistem operasi untuk diinstall dan dijalankan, seperti Windows, Linux, dan Mac Os. Namun sebelum melangkah untuk menggunakan Framework CodeIgniter 4, kita disarankan untuk memiliki pengetahuan bahasa pemrograman PHP terutama konsep *object oriented programming* dalam PHP.

Dalam CodeIgniter menganut konsep Model-View-Controller development pattern yang memisahkan antara logika aplikasi dan tampilannya. CodeIgniter paling dikenal karena kecepatannya jika dibandingkan dengan framework PHP lainnya. Seorang pembuat PHP Rasmus Lerdorf dalam acara frOSCon pada pada Agustus 2008, pernah mengatakan bahwa dia menyukai CodeIgniter karena kecepatannya, *"because it is faster, lighter and the least like a framework."*[2].

CodeIgniter merupakan kerangka kerja yang menyediakan toolkit sederhana untuk membangun sebuah aplikasi web yang lengkap. Dari sekian banyak framework PHP yang ada, CodeIgniter memiliki keunggulan tersendiri dari segi fitur dan keandalannya. Berikut ini merupakan keunggulan yang dimiliki oleh CodeIgniter sehingga menarik perhatian para developer web.

- Menjadi framework yang memiliki ukuran lebih kecil dari framework lain.
- Framework yang hampir zero configuration.
- Tidak mengharuskan kita menggunakan command line.
- Tidak mengharuskan untuk mematuhi aturan pengkodean yang ditetapkan.

- Tidak ada ketentuan untuk mempelajari bahasa template tertentu.
- Menghindari kerumitan, dan memberikan solusi yang sederhana.
- Memiliki dokumentasi yang detail dan jelas.

Namun, ada beberapa sumber mengatakan bahwa CodeIgniter tidak ditunjuk untuk pengembangan web dengan skala besar karena keterbatasan library yang disediakan dan sulitnya mencari library tambahan yang terverifikasi resmi, karena CodeIgniter sendiri tidak menyediakan library tambahan untuk mendukung dalam pengembangan aplikasi web dengan skala besar.

Sekitar tahun 2015 merupakan saat diluncurkannya versi stabil dari CodeIgniter versi 3, saat ini versi ini sudah tergolong tua dan sudah lama jika dibandingkan dengan framework lainnya. Dan pada tanggal 24 Februari 2020, CodeIgniter mulai memperkenalkan versi terbarunya, yaitu CodeIgniter versi 4 dengan lebih memperbaharui *resources* dan versi PHP yang digunakan, sehingga pada versi 4 ini terkesan mirip dengan gaya yang dimiliki Laravel [3]. Berikut merupakan perbedaan yang dimiliki oleh CodeIgniter versi 3 dan versi 4.

- **Dukungan versi PHP**

CodeIgniter versi 3 mendukung dan menggunakan PHP versi minimum 5.6, sedangkan dalam versi 4 telah menggunakan PHP versi minimum 7.2

- **Struktur Directory**

Struktur folder yang dimiliki sangatlah berbeda, pada versi 3 memiliki susunan 2 directory saja, yaitu `/application`, dan `/system`. Sedangkan versi 4 memiliki lebih banyak susunan directory, yaitu `/app`, `/vendor`, `/public`, `/writables`, `/test`.

- **Penggunaan Namespace**

Pada versi 3 CodeIgniter tidak menggunakan namespace dalam proses autoloading, dan pada versi 4 sudah mulai menerapkan namespace seperti yang digunakan oleh Laravel.

- **Penggunaan Entitas**

CodeIgniter versi 4 mulai menyediakan dukungan penggunaan fitur Entitas yang mewakili kolom yang terdapat dalam tabel, namun pada versi 3 tidak memiliki fitur ini, untuk menggunakan perlu membuat *third library*.

- **Performance**

Penggunaan versi PHP 7 pada CodeIgniter versi 4 membuatnya memiliki tingkat kinerja yang lebih baik dari pada versi 3.

- **Pengaturan**

CodeIgniter versi 4 meminimalkan konfigurasi awal pada saat akan menjalankannya, sedangkan pada versi 3 memerlukan sejumlah konfigurasi awal yang benar.

- **Dukungan terhadap file .env**

Pada versi 4, CodeIgniter mulai mendukung file environment yang berekstensi .env yang dapat mempermudah dalam mengkonfigurasi, namun pada versi 3 tidak ada dukungan terhadap file environment.

Setelah CodeIgniter menghadirkan versi 4, framework ini mulai melakukan perubahan yang signifikan mulai dari fitur dan gaya yang dimilikinya. Maka dari itu kita akan mulai mempelajari penggunaan CodeIgniter 4, yang saat ini menjadi versi terakhir dalam perkembangan CodeIgniter.

CodeIgniter didasarkan pada pola pengembangan Model-View-Controller. MVC sendiri merupakan kependekan dari Model View Controller. MVC adalah sebuah pendekatan pemrograman

perangkat lunak yang memisahkan antara application logic (logika aplikasi) dari presentation logic (antarmuka aplikasi)[4]. Secara sederhana dengan konsep mvc akan memisahkan antara desain, data, dan proses. Berikut merupakan peran yang dilakukan masing-masing komponen dari MVC.

1. Model

Model merupakan representasi dari struktur data. Model menjadi komponen tempat menuliskan perintah yang berhubungan dengan query SQL. Biasanya dalam model terdapat class yang nantinya berfungsi untuk membantu mengambil, menambahkan ataupun memperbaharui data dalam database.

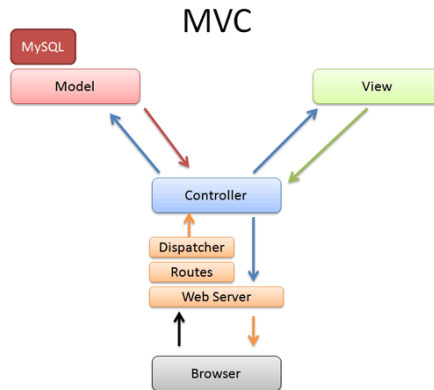
2. View

View merupakan komponen yang berhubungan dengan sesuatu yang akan ditampilkan pada pengguna (*user interface*). View menjadi tempat untuk menuliskan baris perintah, seperti HTML, CSS, maupun Java Script, RSS, dll. Didalam View hanya akan menampilkan data sebuah variabel yang telah dipassing dari controller, dan tidak disarankan untuk memanggil data secara langsung dari view.

3. Controller

Controller menjadi komponen yang bertindak sebagai perantara antara Model, View, maupun resource lain yang diperlukan untuk memproses request HTTP dan menampilkan hasil request tersebut. Didalam Controller akan terdapat class dan fungsi-fungsi untuk memproses request dari View untuk meminta data dari Model. Controller biasanya bertugas untuk menyediakan variabel untuk dipassing ke dalam View, memanggil data dari suatu Model, menyediakan pesan kesalahan/*error*, mengerjakan proses

logika tertentu, dan melakukan penanganan validasi sebuah form.

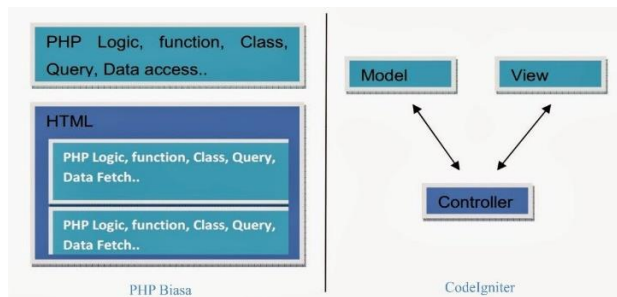


Gambar 1.2 Arsitektur MVC

Dari gambar 1.2 diatas, merupakan representasi dari pemetaan yang terjadi dalam aplikasi MVC . Browser akan berinteraksi dengan web server untuk meminta halaman tertentu, kemudian router akan mencari controller mana yang harus menangani permintaan tersebut. Controller akan menerima, memproses, dan merespon permintaan tersebut. Jika permintaan hanya sebuah tampilan *user interface* saja, controller hanya akan memanggil view saja. Namun jika request berhubungan dengan data di database, controller akan menghubungi Model, dan Model akan menjalankan perintah query untuk aksi tertentu ke database. Selanjutnya permintaan tersebut diteruskan pada web server untuk ditampilkan pada web browser.

Dalam hal tersebut, dapat dikatakan bahwa controller menjadi jantung dari aplikasi untuk mengarahkan apa yang harus dilakukan aplikasi dari setiap kali mendapat request, dan model menjadi representasi data, dan view untuk menyajikan tampilannya. Jika dibandingkan dengan PHP biasa, dengan model MVC program akan

lebih terstruktur dan rapi, sehingga akan mempermudah dalam pengerjaan project dalam sebuah tim.



Gambar 1.3 Perbedaan PHP Biasa dengan MVC

Secara umum dengan menggunakan PHP biasa, program akan meload semua library dan fungsi yang digabungkan dengan HTML untuk dieksekusi oleh PHP. Hal tersebut tidak menjadi masalah jika diterapkan dalam kasus kecil, namun jika skala aplikasi sudah besar kita biasanya lebih sulit untuk memaintenance jika tidak didukung dengan arsitektur yang baik.

1.2. TUJUAN

Dengan adanya buku tutorial ini, diharapkan dapat mempermudah seseorang yang ingin terjun menjadi web developer, memiliki pengetahuan dasar tentang framework CodeIgniter 4, dan dapat menggunakan framework tersebut untuk mengembangkan aplikasi web.

1.3. MANFAAT

Setelah menyelesaikan semua bab yang ada dalam buku ini, pembaca akan mendapatkan manfaat :

1. Mendapatkan pengetahuan dasar mengenai CodeIgniter 4.

2. Mengetahui penggunaan MVC yang juga diterapkan pada framework lain.
3. Merasakan perbedaan yang lebih lebih dimudahkan dari pada menggunakan PHP native.
4. Membuat pengelolaan data dengan mudah dengan bantuan *query builder*.
5. Membuat aplikasi toefl sederhana dengan CodeIgniter 4.

BAB 2

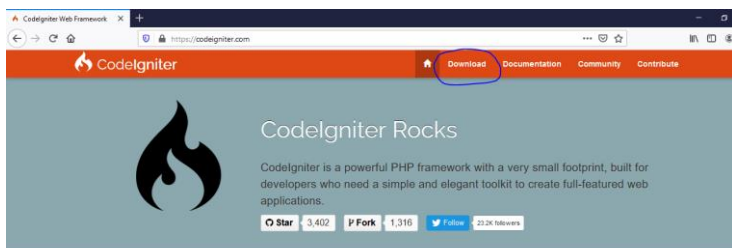
STUDI PUSTAKA

Pada bab sebelumnya kita sudah berkenalan dengan framework Codeigniter 4, mulai dari definisi, kelebihan framework ini, konsep MVC, dan lain sebagainya. Selanjutnya pada bab ini, kita akan mempelajari bagaimana untuk menginstall framework Codeigniter 4 pada komputer local kita.

Framework Codeigniter 4 memiliki beberapa cara untuk diinstall, kita dapat menginstall dengan mengunduh package framework ini yang disediakan dalam bentuk zip. Kemudian kita juga dapat menginstall dengan cara yang lebih modern, yaitu dengan menggunakan composer, seperti halnya dalam menginstall framework modern lainnya.

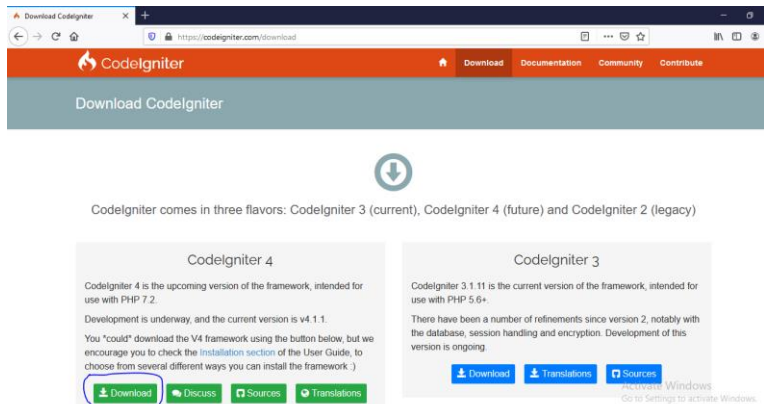
2.1. INSTALASI CODEIGNITER SECARA MANUAL

Framework Codeigniter 4 juga dapat dilakukan instalasi secara manual. Kita dapat mendownload archive file pada website resminya, yaitu pada link <https://codeigniter.com/>. Kemudian klik link download pada bagian navigation bar di paling atas, maka akan diarahkan pada halaman download.



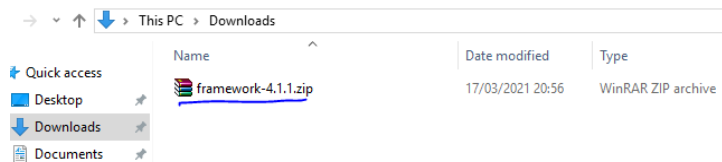
Gambar 2.1 Halaman utama Website Codeigniter

Kemudian pada bagian card Codeigniter 4, dapat ditekan tombol download.



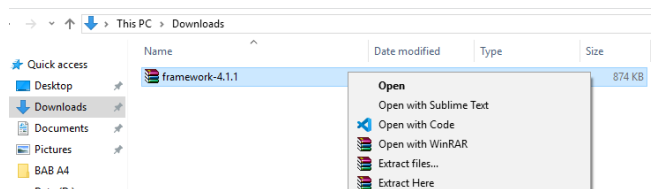
Gambar 2.2 Halaman download codeigniter

Maka archive file dalam bentuk .zip, akan tersimpan pada directory download.



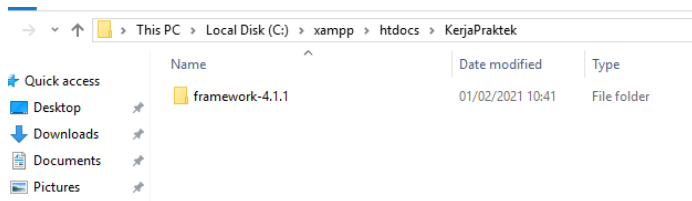
Gambar 2.3 File Archive Codeigniter 4

Kemudian extract files pada archive file tersebut dan arahkan pada directory “C:\xampp\htdocs\KerjaPraktek”. Kita juga dapat mengextractnya pada lokasi directory dimanapun pada komputer, namun yang disarankan untuk diextract pada folder /xampp/htdocs (jika menggunakan xampp), agar dapat dijalankan melalui php development server maupun menggunakan apache webserver.



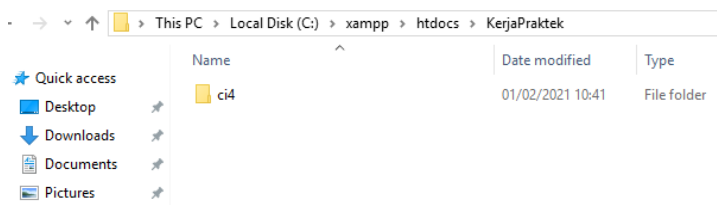
Gambar 2.4 Archive File Codeigniter

Kemudian akan mendapatkan folder framework-4.1.1 hasil extract, pada directory “C:\xampp\htdocs\KerjaPraktek”



Gambar 2.5 File extract Codeigniter 4

Setelah mengextract, rename folder framework-4.1.1 dengan nama folder “ci4”, agar mempermudah untuk mengidentifikasi ketika file project kita sudah banyak.



Gambar 2.6 Rename folder Codeigniter 4

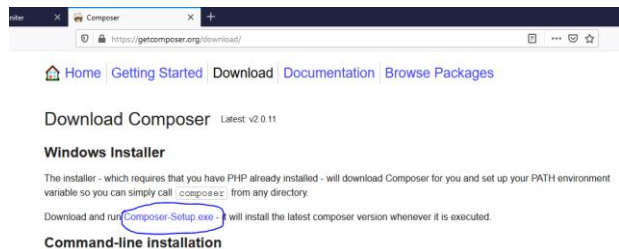
2.2. INSTALASI CODEIGNITER MENGGUNAKAN COMPOSER

2.2.1. PENGENALAN DAN INSTALASI COMPOSER

Composer merupakan aplikasi yang dapat digunakan untuk mengelola library untuk bahasa pemrograman PHP. Aplikasi ini menjadi standarisasi dalam mengelola library/package *software dependencies* saat mengerjakan project dengan PHP. Seperti halnya kita membutuhkan library PHPMailer, PHPSpreadsheet, dst. dengan composer kita dapat mendefinisikan library tersebut, dan composer

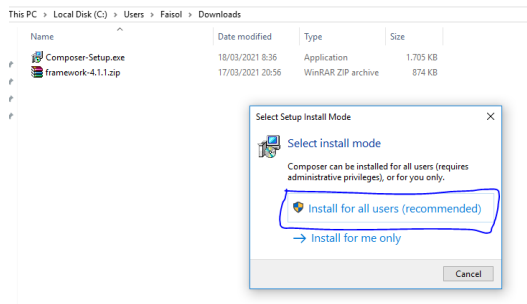
akan menangani dengan menginstalasi dan menyiapkan library untuk dapat digunakan pada project kita.

Instalasi composer dapat dilakukan setelah mendapatkan file installernya. Untuk mendapatkan file installer tersebut, kita dapat mengunjungi website resminya di <https://getcomposer.org/download/>. Kemudian klik Composer-Setup.exe, maka composer akan terdownload pada komputer kita.



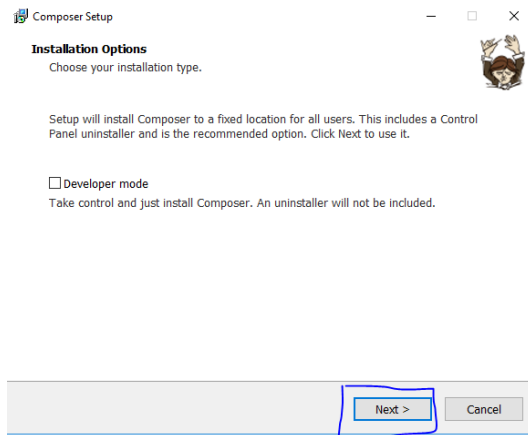
Gambar 2.7 Halaman download Composer

Setelah berhasil di download, double click pada Composer-Setup.exe, maka akan tampil Select install mode, dan pilih *install for all user (recommended)*



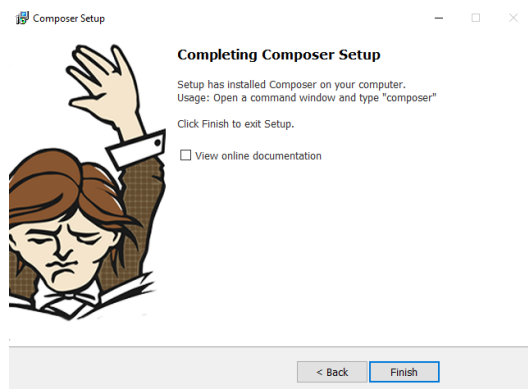
Gambar 2.8 Select install Composer

Kemudian klik tombol next dan next, seperti halnya dalam menginstall aplikasi pada umumnya



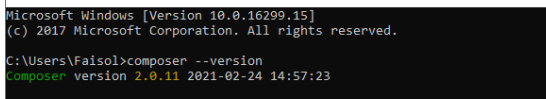
Gambar 2.9 Installation Options Composer

Kemudian akan muncul Completing Composer Setup, dan klik tombol finish untuk menyelesaikan Composer Setup.



Gambar 2.9 Completing Composer Setup

Setelah installasi composer selesai, kita dapat memastikan bahwa Composer telah terinstall dan dapat



```
Microsoft Windows [Version 10.0.16299.15]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\Faisal>composer --version
composer version 2.0.11 2021-02-24 14:57:23

C:\Users\Faisal>
```

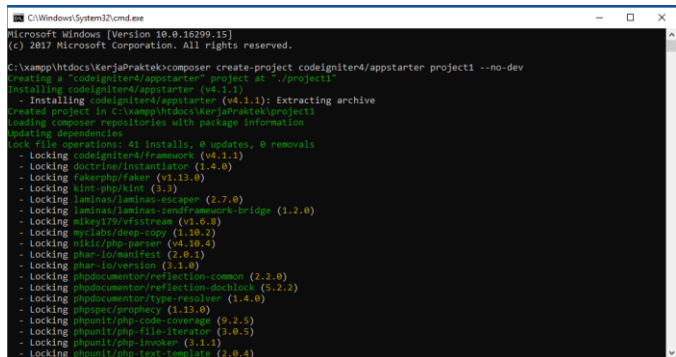
Gambar 2.10 Cek Versi Composer

2.2.2. INSTALASI CODEIGNITER 4

Setelah mengetahui cara untuk menginstall Codeigniter secara manual, Framework ini juga bisa diinstall melalui aplikasi package manager, yaitu Composer. Secara umum, kebanyakan developer lebih memilih menginstall Codeigniter 4 menggunakan Composer, karena dalam proses instalasi maupun upgradenya lebih mudah.

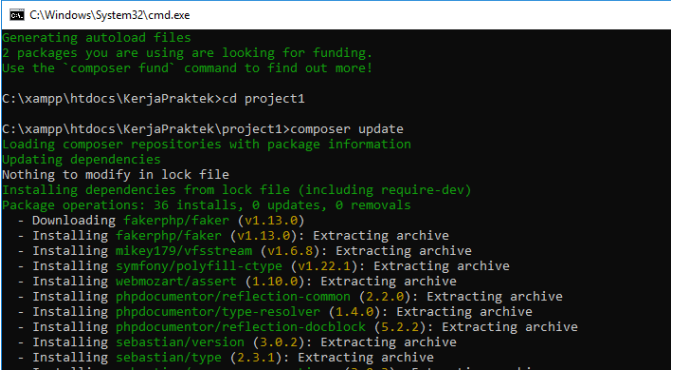
Untuk menginstallasi kita hanya perlu membuka command line komputer kita, kemudian ketikkan perintah

```
"composer create-project  
codeigniter4/appstarter nama-project --no-  
dev"
```



Gambar 2.11 Install Codeigniter Composer

Kemudian tunggu hingga proses instalasi Codeigniter 4 selesai. Jika kita ingin mengupgrade versi Codeigniter 4 maupun library yang ada didalamnya, maka ketikkan perintah “composer update”



```
GA\Windows\System32\cmd.exe
Generating autoload files
2 packages you are using are looking for funding.
Use the "composer fund" command to find out more!

C:\xampp\htdocs\KerjaPraktek>cd project1

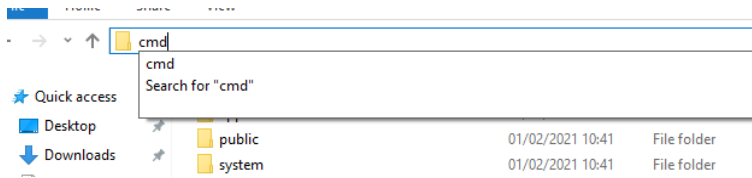
C:\xampp\htdocs\KerjaPraktek\project1>composer update
Loading composer repositories with package information
Updating dependencies
Nothing to modify in lock file
Installing dependencies from lock file (including require-dev)
Package operations: 36 installs, 0 updates, 0 removals
  - Downloading fakerphp/faker (v1.13.0)
  - Installing fakerphp/faker (v1.13.0): Extracting archive
  - Installing nikic/php-parser (v1.6.8): Extracting archive
  - Installing symfony/polyfill-ctype (v1.22.1): Extracting archive
  - Installing webmozart/assert (1.10.0): Extracting archive
  - Installing phpdocumentor/reflection-common (2.2.0): Extracting archive
  - Installing phpdocumentor/type-resolver (1.4.0): Extracting archive
  - Installing phpdocumentor/reflection-docblock (5.2.2): Extracting archive
  - Installing sebastian/version (3.0.2): Extracting archive
  - Installing sebastian/type (2.3.1): Extracting archive
```

Gambar 2.12 Upgrade Codeigniter 4

2.3. MENJALANKAN CODEIGNITER

Setelah proses instalasi codeigniter berhasil, framework ini dapat dijalankan pada web browser. Menjalankan project Codeigniter 4 dapat dilakukan dengan beberapa cara yang berbeda, kita dapat menjalankan melalui php development server maupun menjalankannya pada webserver.

Codeigniter 4 hadir dengan local development server, yang memanfaatkan development server bawaan PHP dengan routing dari Codeigniter. Untuk menjalankan pada local development server, kita dapat membuka cmd dengan lokasi yang mengarah pada folder project Codeigniter. Ketikkan “cmd” pada move menu.



Gambar 2.13 Membuka Command Line

Setelah command line tampil, ketikkan perintah “`php spark serve`” dan tekan tombol enter. Kemudian akan muncul tulisan “*CodeIgniter development server started on http://localhost:8080*”, yang menandakan project Codeigniter telah berjalan dan dapat diakses di link <http://localhost> pada port 8080.

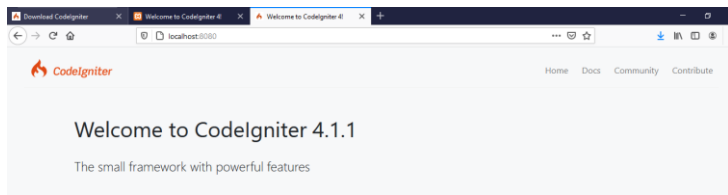
```

C:\Windows\System32\cmd.exe - php spark serve
Microsoft Windows [Version 10.0.16299.15]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\KerjaPraktek\ci4>php spark serve
CodeIgniter v4.1.1 Command Line Tool - Server Time: 2021-03-17 09:35:04 UTC-05:00
CodeIgniter development server started on http://localhost:8080
Press Control-C to stop.
[Wed Mar 17 21:35:04 2021] PHP 8.0.2 Development Server (http://localhost:8080) started
  
```

Gambar 2.14 Development Server

Kemudian buka web browser yang biasa kita gunakan, kemudian akses link <http://localhost:8080>. Apabila muncul tampilan “*Welcome to CodeIgniter*”, yang mendakan framework CodeIgniter berhasil diinstall.



Gambar 2.15 CodeIgnier melalui Development Server

Selain dengan local development server, project CodeIgniter 4 juga dapat dijalankan melalui web server, seperti apache, nginx, dsb. Dengan menginstall CodeIgniter pada folder htdocs (*jika menggunakan xampp*). Untuk menjalankan ketikkan "localhost/kerjapraktek/project1/", dan tekan enter.

BAB 3

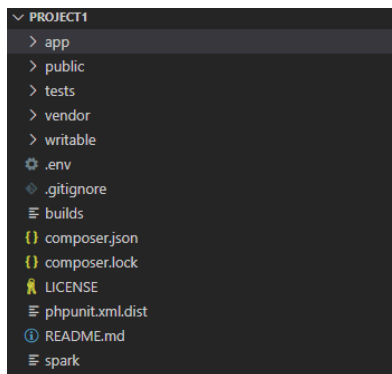
STRUKTUR FOLDER DAN ENVIRONMENT

Pada bab sebelumnya, kita telah belajar menginstall framework CodeIgniter 4 dan mendapatkan file dan folder pendukung yang dimiliki oleh CodeIgniter. Dari hasil instalasi tersebut, terdapat beberapa folder yang boleh dirubah dan tidak boleh dirubah. Folder tersebut merupakan folder default yang tersusun menjadi kerangka aplikasi yang akan memudahkan kita dalam mengembangkan project. Selain itu, terdapat folder yang akan sering digunakan dan menjadi sebagai tempat menyimpan file script kita menjadi module.

Pada bab ini akan dijelaskan tentang struktur folder yang ada dalam framework CodeIgniter 4 beserta fungsi dari folder tersebut. Kemudian folder mana yang akan digunakan untuk menyimpan file script kita. Dan juga akan dijelaskan tentang file environment yang akan menjadi file konfigurasi untuk project CodeIgniter.

3.1. STRUKTUR FOLDER CODEIGNITER 4

Setelah Codeigniter 4 berhasil diinstall, kita akan mendapatkan folder-folder dan file pendukung yang dibutuhkan oleh Codeigniter..



Gambar 3.1 Struktur Folder Root

Dari gambar 3.1 diatas, terdapat beberapa sub folder yang tersusun dari framework CodeIgniter 4. Berikut ini merupakan penjelasan sub folder root dari framework tersebut :

- **Folder Writable**

Pada folder writable, nantinya akan menyimpan file-file yang dihasilkan oleh aplikasi, seperti cache dari aplikasi, maupun file yang diupload oleh user.

- **Folder Vendor**

Folder vendor menjadi folder utama dalam framework CodeIgniter, didalamnya berisi sejumlah sub folder penting yang membangun framework ini. Selain itu, dalam folder tersebut juga menjadi tempat tersimpannya sebuah library/package yang kita install menggunakan Composer, seperti PHPMailer, PHPFaker, PHPUnit, dll.

- **Folder Test**

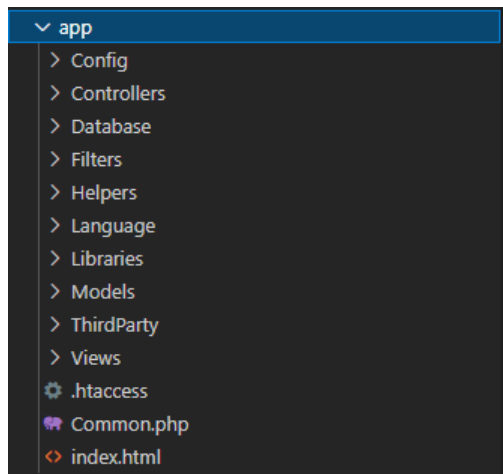
Folder test dapat digunakan sebagai tempat untuk menyimpan file untuk keperluan testing project. Ketika mengembangkan project yang membutuhkan metode yang mengharuskan membuat file debungging, folder ini disediakan oleh CodeIgniter sebagai tempat yang disarankan. Didalamnya terdapat sub folder __support yang berisi utilitas yang dapat mempermudah dalam membuat unit testing.

- **Folder Public**

Folder public sebagai tempat untuk menyimpan file-file yang dapat diakses dan tampilkan pada pengguna. Ketika kita menjalankan project melalui local development server maupun melalui webserver, folder inilah yang diakses, sehingga menampilkan isi dari project yang dibuat. Semua assets, seperti css, js, gambar, dll. Perlu disimpan pada folder tersebut.

- **Folder App**

Folder app akan menjadi folder yang akan sering dikunjungi. Dalam folder tersebut terdapat sub-folder yang menjadi tempat untuk menyimpan file-file kita dalam membangun project. File tersebut dapat berupa file Model, Views, Controller, maupun Libraries.



Gambar 3.2 Struktur Folder App

Dari gambar diatas terdapat beberapa sub folder yang tersusun dalam folder app. Berikut merupakan penjelasan sub folder dari folder app :

- **Folder Config**, dalam folder tersebut terdapat file-file konfigurasi untuk mengkonfigurasi project CodeIgniter, seperti baseurl, koneksi database, routing, dll.
- **Folder Controllers**, folder tersebut digunakan sebagai tempat untuk menyimpan file controller untuk menentukan flow suatu halaman.

- **Folder Database**, folder tersebut digunakan untuk menyimpan file migration dan seed.
- **Folder Filter**, folder tersebut untuk menyimpan file script yang akan berjalan sebelum atau sesudah controller dijalankan.
- **Folder Helper**, folder tersebut dapat digunakan untuk menyimpan file script helper yang kita buat.
- **Folder Language**, folder tersebut digunakan untuk menyimpan text content dalam berbagai bahasa, seperti text untuk validasi form yang dapat diberikan dalam bahasa indonesia, maupun bahasa lain.
- **Folder Libraries**, folder tersebut digunakan untuk menyimpan file script library yang kita buat, seperti membuat library untuk konversi tanggal.
- **Folder Model**, folder tersebut digunakan untuk menyimpan script Model yang dapat mengelola data pada database.
- **Folder ThirdParty**, folder tersebut berisi library pihak ketiga yang akan digunakan pada aplikasi kita.
- **Folder Views**, folder tersebut digunakan untuk menyimpan file view/tampilan dari project yang dibuat. Dalam setiap file view akan berisi script HTML, CSS, dan JS yang menjadi script untuk membentuk tampilan aplikasi.

3.2. MEMAHAMI ENVIRONMENT

Setelah memahami tentang folder-folder yang tersusun dalam project framework CodeIgniter 4, terdapat sebuah file yang biasanya digunakan untuk konfigurasi, yaitu file *env*. File ini dapat berfungsi setelah direname menjadi *.env*. Berikut ini merupakan isi dari file *.env* :

```

#-----
#-----
# Example Environment Configuration file
#
# This file can be used as a starting point for
your own
# custom .env files, and contains most of the
possible settings
# available in a default install.
#
# By default, all of the settings are commented
out. If you want
# to override the setting, you must un-comment
it by removing the '#'
# at the beginning of the line.
#-----
#-----

#-----
#-----
# ENVIRONMENT
#-----
#-----

# CI_ENVIRONMENT = production

#-----
#-----

# APP
#-----
#-----

# app.baseURL = ''
# app.forceGlobalSecureRequests = false

# app.sessionDriver =
'CodeIgniter\Session\Handlers\FileHandler'
# app.sessionCookieName = 'ci_session'
# app.sessionSavePath = NULL
# app.sessionMatchIP = false

```

```

# app.sessionTimeToUpdate = 300
# app.sessionRegenerateDestroy = false

# app.cookiePrefix = ''
# app.cookieDomain = ''
# app.cookiePath = '/'
# app.cookieSecure = false
# app.cookieHTTPOnly = false
# app.cookieSameSite = 'Lax'

# app.CSRFProtection = false
# app.CSRFTokenName = 'csrf_test_name'
# app.CSRFCookieName = 'csrf_cookie_name'
# app.CSRFExpire = 7200
# app.CSRFRegenerate = true
# app.CSRFExcludeURIs = []
# app.CSRFSameSite = 'Lax'

# app.CSPEnabled = false

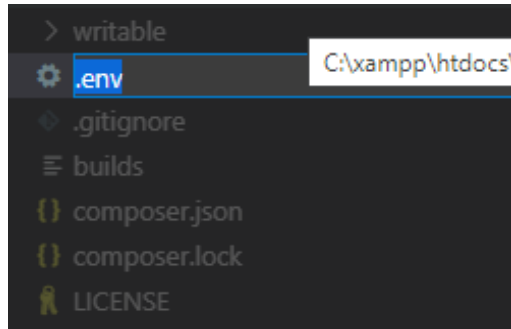
#-----
#-----
# DATABASE
#-----
#-----

# database.default.hostname = localhost
# database.default.database = ci4
# database.default.username = root
# database.default.password = root
# database.default.DBDriver = MySQLi
.
.
.
.
.

```

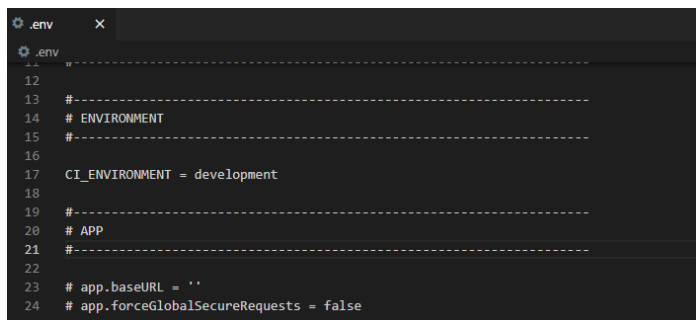
Kemudian untuk menggunakannya file env, rename file env menjadi .env dengan memberikan tanda titik (.) sebelum kata env.

Pada bagian konfigurasi DATABASE, konfigurasi seperti hostname, username, dan password menyesuaikan dengan akun yang ada pada database server yang digunakan. Dan dibagian DBDriver kita dapat menggunakan MySQLi maupun PDO sebagai drivernya, driver MySQL sudah tidak didukung lagi mulai PHP versi 7 keatas, dan digantikan dengan driver MySQLi.



Gambar 3.3 Rename File env

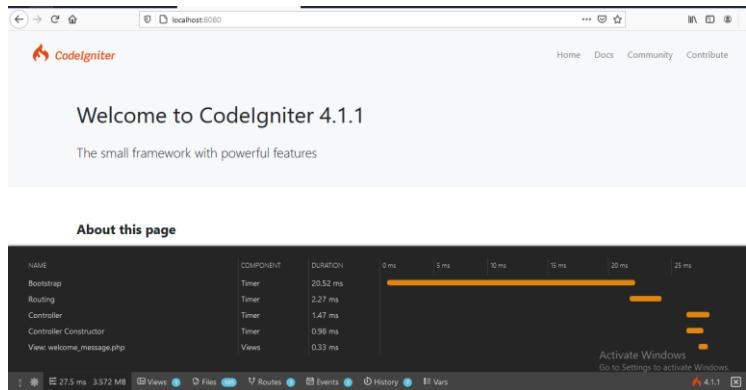
Agar konfigurasi berjalan, kita harus menghilangkan tanda pagar (#) pada bagian yang ingin diaktifkan dalam file .env tersebut. Misalkan pada bagian `#CI_ENVIRONMENT = production`, dapat diubah menjadi `CI_ENVIRONMENT = production/development`.



Gambar 3.4 Setting Environment file .env

Dengan mengubah environment menjadi *development*, project Codelgniter akan menjadi mode development yang memberikan

fitur untuk mengetahui waktu eksekusi, informasi files, routes, event, dll.



Gambar 3.5 Tampilan Mode Development

Dalam file `.env` perlu biasanya berisi informasi sensitif seperti username, password, dan token-token API penting dari project yang dibangun. Maka dari itu, kita perlu berhati-hati ketika mendeploy, atau mengirimkan project CodeIgniter 4 pada orang lain. Hal yang biasanya terjadi pada saat mendeploy CodeIgniter, kita lupa melakukan exclude file `.env` pada saat melakukan commit ke git.

BAB 4

MENGENAL CONTROLLER, ROUTING, VIEWS, DAN SISTEM TEMPLATING

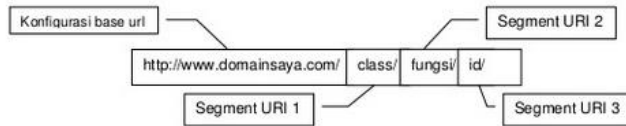
Pada bab sebelumnya, kita telah mengenal struktur folder dan melakukan konfigurasi sederhana pada file environment, sehingga project menjadi mode development. CodeIgniter sendiri merupakan framework yang tidak lepas dari sistem MVC didalamnya. Kita diharuskan untuk memahami dan menerapkan sistem MVC dalam mengembangkan aplikasi. Seperti halnya membuat halaman produk, kita harus membuat Model untuk mengambil data produk, View untuk menampilkan data dalam bentuk tabular, dan Controller untuk menjembatani keduanya.

Pada bab ini kita akan mempelajari mengenai pondasi utama dalam membuat aplikasi menggunakan CodeIgniter. Kita akan mengenal dan menerapkan controller, membuat view, melakukan routing pada url, dan membuat sistem templating sederhana.

4.1. MENGENAL DAN MEMBUAT CONTROLLER

Secara sederhana, controller hanyalah sebuah file class yang diberikan nama sedemikian rupa sehingga dapat dikaitkan dengan URI. Controller akan menjadi jantung aplikasi, karena harus menangani sebuah request yang diberikan. Dalam CodeIgniter 4, controller terletak pada directory `/root-project/app/Controllers/`.

Segment-segment yang terdapat pada URL merepresentasikan sebuah controller tertentu. Misalnya kita mengakses URL `fsol.tech/produk/hapus/3`, domain `fsol.tech` merupakan base url, segment `produk` merepresentasikan class controller, `delete` merupakan method, dan `3` adalah parameter dari method.



Gambar 4.1 Segment URL CodeIgniter

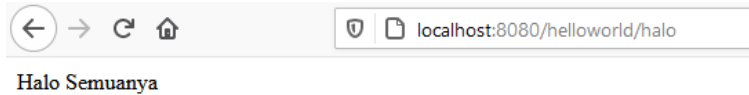
Untuk lebih memahami tentang controller, mari kita buat sebuah controller menggunakan text editor. Buat file baru dengan nama `Helloworld.php`, dan perlu diperhatikan nama controller harus menggunakan huruf besar diawal. Kemudian simpan file tersebut pada directory `/app/Controllers/`. Kemudian buat class yang merupakan turunan dari class `BaseController` yang didalamnya didefinisikan sebuah method. Nama class yang diberikan harus sama dengan nama file php tersebut. Berikut merupakan code sederhana untuk membuat controller `Helloworld`.

```
<?php

namespace App\Controllers;

class Helloworld extends BaseController
{
    public function index()
    {
        echo "Hello World";
    }
    public function halo($nama)
    {
        echo "Halo ".$nama;
    }
}
```

Kemudian kita dapat membuka URL dengan menuliskan domain, diikuti dengan nama class sebagai segment pertama, dan nama method sebagai segment kedua.



Gambar 4.1 Hasil Controller Helloworld

Secara default, method index pada controller akan dipanggil sebagai segment kedua jika segment kedua tidak dimasukkan.

4.2. MENGENAL DAN MELAKUKAN ROUTING URL

Routing adalah proses menentukan rute URL yang dapat lewat. Dalam CodeIgniter 4, routing digunakan untuk menentukan controller mana yang bertanggung jawab merespon sebuah request. Tanpa menggunakan routing, sebenarnya sudah ada hubungan satu-ke-satu antara segment URL dengan class/method controller yang berkaitan. Biasanya segment dalam URL dituliskan dengan pola berikut.

```
example.com/class/method/id/
```

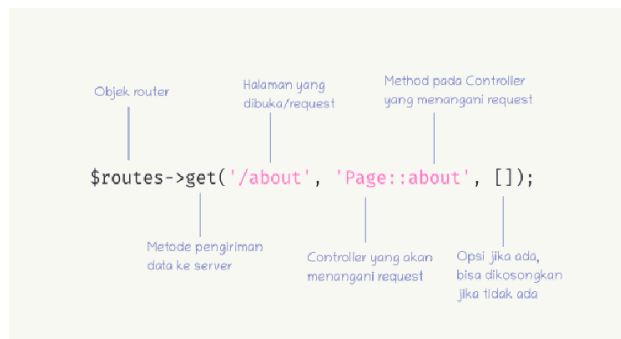
Namun dalam beberapa kasus, kita mungkin ingin memetakan kembali rute ini, sehingga class/method dapat dipanggil tidak harus sama dengan controller. Misalnya, pola url dapat dituliskan menjadi seperti berikut.

```
example.com/product/1/  
example.com/product/2/  
example.com/product/3/  
example.com/product/4/
```

Biasanya segment kedua dari URL dituliskan dengan nama method, namun pada contoh diatas langsung menuliskan ID

product. Hal tersebut terjadi ketika kita sudah melakukan pemetaan pada routes.

Dalam CodeIgniter 4, aturan pemetaan rute dapat didefinisikan dalam file `Routes.php` yang terletak pada directory `/app/Config/Routes.php`. Didalam file tersebut kita dapat melihat bahwa terdapat instansiasi dari `RouteCollection` yang memungkinkan kita dapat melakukan perutean sesuai dengan kebutuhan. Penentuan routing dapat ditentukan menggunakan placeholder maupun Regular Expression.



Gambar 4.2 Struktur Route

Secara umum, sebuah route didefinisikan seperti berikut :

```
$routes->get('product/(:num)',  
            'App\Catalog::productLookup');
```

Routing dimulai dengan menuliskan object route, kemudian dilanjutkan dengan metode pengiriman data ke server (`get`, `post`, `put`, `delete`), dan pada parameter pertama berisi URI sebagai route dan bisa diikuti dengan Placeholder maupun Regular Expression, dan parameter kedua berisi Controller tujuan diikuti dengan mothod yang akan beraksi. Berikut placeholder yang tersedia yang dapat digunakan pada route untuk mewakili pola Regular Expression.

Placeholders	Keterangan
(:any)	Menerima semua karakter hingga akhir URI
(:segment)	Menerima semua karakter, kecuali garis miring (/) yang membatasi hasil ke satu segmen
(:num)	Hanya menerima segment berupa bilangan bulat
(:alpha)	Hanya menerima segment berupa karakter alfabet
(:alphanum)	Menerima segment berupa bilangan bulat dan karakter alfabet, maupun kombinasi keduanya
(:hash)	Mirip dengan (:segment), namun bisa digunakan untuk melihat rute mana yang menggunakan id dengan mudah

Untuk lebih memahami tentang Routing, mari kita buat sebuah controller baru dengan nama `Product.php` kemudian simpan dalam directory `/app/Controllers/`. Dan tambahkan code berikut pada controller tersebut.

```
<?php

namespace App\Controllers;

class Product extends BaseController
{
    public function index()
    {
        echo "Halaman ini berisi
            daftar product";
    }
    public function detail($id)
    {
        echo "Tampilakan product
            yang memiliki ID : ". $id;
```

```

    }
    public function cari($kategori, $id)
    {
        echo "Ubah product kategori ".
            $kategori ." yang memiliki
            ID : ". $id;
    }
}

```

Kemudian buka file `Routes.php` pada directory `/app/Config/Routes.php`, dan tambahkan code berikut untuk mendefinisikan rute.

```

//latihan routing
$routees->get('/barang', 'Product::index');
$routees->get('/barang/(:num)', 'Product:
    :detail/$1');
$routees
    ->get('/barang/(:alpha)/(:any)',
        'Product::cari/$1/$2');

```

Untuk memastikan route sudah dibuat dengan benar kita dapat mengetikkan perintah “`php spark routes`” pada command prompt. Jika muncul seperti gambar berikut, maka route berhasil dibuat.

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.16299.15]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\KerjaPraktek\project1>php spark routes

CodeIgniter v4.1.1 Command Line Tool - Server Time: 2021-03-22 21:22:58 UTC-05:00

```

Method	Route	Handler
GET	/	\App\Controllers\Home::index
GET	barang	\App\Controllers\Product::index
GET	barang/([0-9]+)	\App\Controllers\Product::detail/\$1
GET	barang/([a-zA-Z]+)/(.*)	\App\Controllers\Product::cari/\$1/\$2
CLI	migrations/([^\s]+)/([^\s]+)	\CodeIgniter\Commands\MigrationsCommand::\$1/\$2
CLI	migrations/([^\s]+)	\CodeIgniter\Commands\MigrationsCommand::\$1
CLI	ci(.*)	\CodeIgniter\CLI\CommandRunner::index/\$1

Gambar 4.3 Menampilkan Routes

Kemudian kita dapat mengakses URL sesuai dengan rute yang telah didefinisikan. Berikut contoh rute yang dapat diakses dari route diatas.

```
http://localhost:8080/barang/  
http://localhost:8080/barang/3  
http://localhost:8080/barang/pakaian/10
```

4.3. MENGENAL DAN MEMBUAT VIEWS

View merupakan sebuah file yang digunakan untuk menuliskan code dalam membuat tampilan *user interface*. Didalamnya biasanya terdiri dari syntax HTML, CSS, JS, dan juga PHP sederhana. View hanyalah halaman web, fragment halaman, seperti header, footer, sidebar, dll. Biasanya view tidak ditampilkan secara langsung, melainkan harus dimuat melalui sebuah controller.

Untuk lebih memahami tentang view, mari kita buat sebuah file baru dengan nama `view_satu.php`, kemudian simpan pada directory `/app/Views/`. Kemudian tambahkan code berikut.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <title>Halaman View</title>  
</head>  
<body>  
    <h1>Selamat Datang</h1>  
    <h3>Ini merupakan percobaan view yang pertama</h3>  
</body>  
</html>
```

Kemudian buat controller untuk memuat view tersebut dengan nama controller `MuatView.php`

```
<?php

namespace App\Controllers;

class MuatView extends BaseController
{
    public function index()
    {
        return view('view_satu');
    }
}
```

Setelah itu jalankan pada web browser dengan mengakses URI <http://localhost:8080/muatview>.



Selamat Datang

Ini merupakan percobaan view yang pertama

Gambar 4.4 Mengampilkan Satu View

Pada contoh diatas, kita hanya memuat dari satu file view saja. Berikut merupakan contoh untuk membuat dari beberapa view. Buat file view baru dengan nama view1.php, kemudian simpan dalam folder Views.

```
<h1>Ini merupakan tampilan dari view 1</h1>
```

Kemudian buat view2.php

```
<h1>Ini merupakan tampilan dari view 2</h1>
```

Setelah itu buat `view3.php`

```
<h1>Ini merupakan tampilan dari view 3</h1>
<h3>Ini merupakan <?= $info ?></h3>
```

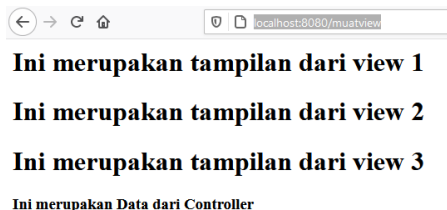
Terakhir buat file controller `MuatView.php` untuk memuat view-view tersebut. Tambahkan code berikut pada controller tersebut.

```
<?php

namespace App\Controllers;

class MuatView extends BaseController
{
    public function index()
    {
        $data['info'] = "Data dari Controller";
        echo view('view1');
        echo view('view2');
        echo view('view3', $data);
    }
}
```

Pada controller diatas juga mempassing data pada *view3*, kemudian ditangkap pada *view3* dan menampilkan isi variabel `$info`. Setelah itu jalankan pada browser dengan mengakses URL `http://localhost:8080/muatview`.



Gambar 4.5 Menampilkan Beberapa View

4.4. MEMBUAT SISTEM TEMPLETING/LAYOUTS

CodeIgniter 4 mendukung sistem layouts yang sederhana, namun sangat fleksibel yang memungkinkan kita membuat satu theme yang bisa digunakan pada banyak view. Layouts menjadi sistem view dengan membuat satu view yang dapat diterapkan pada view-view yang lainnya. Layout dibuat dengan satu view sebagai template yang menggunakan method `$this->renderSection()`. Method tersebut akan berperilaku sebagai placeholder untuk content. Biasanya didalam template tersebut berisi header dan footer.

Berikut ini merupakan contoh sederhana penerapan sistem templating. Dengan menggunakan text editor, buat file baru dengan nama `template.php`, kemudian simpan dalam folder `/app/Views/`. Kemudian tambahkan code berikut.

```
<!doctype html>
<html>
<head>
    <title>Sistem Templating</title>
</head>
<body>
    <h1>Ini merupakan teks dari template</h1>
    <?= $this->renderSection('content') ?>
</body>
</html>
```

Kita dapat membuat view baru untuk menerapkan template diatas. Setiap kali akan menyisipkan sebuah template pada view, maka gunakan method `$this->extend('namatemplate')`. Kemudian buat view `about.php`, simpan pada folder Views, dan tambahkan code berikut.

```
<?= $this->extend('template') ?>

<?= $this->section('content') ?>
```

```
<h1>Halo, ini adalah teks dari view about</h1>
<?= $this->endSection() ?>
```

Setelah itu, buat controller baru dengan nama `Template.php` untuk memuat view `about.php` tersebut.

```
<?php

namespace App\Controllers;

class Template extends BaseController
{
    public function index()
    {
        return view('about');
    }
}
```

Kemudian kita dapat mengakses halaman `http://localhost:8080/template/` untuk menampilkannya.



Ini merupakan teks dari template

Halo, ini adalah teks dari view about

Gambar 4.6 Menampilkan View Sistem Templating

Dengan menggunakan sistem template seperti diatas, kita dapat membuat struktur code pada controller menjadi lebih rapi, dari pada memanggil beberapa view dalam suatu controller.

BAB 5

KONEKSI DATABASE MYSQL DAN MODEL

Pada bab sebelumnya, kita sudah mengenal controller, views, routing, yang menjadi komponen dasar dari CodeIgniter. Sebuah aplikasi juga tidak akan pernah lepas dari database. Database menjadi aspek penting dalam pengembangan aplikasi, karena menjadi tempat menyimpan data dan informasi. Ada banyak jenis database yang bisa digunakan, seperti MySQL, SQL Server, SQLite, PostgreSQL, dll. Dari beberapa DBMS yang ada, CodeIgniter juga mendukung penggunaan beberapa database untuk digunakan, namun dari beberapa database tersebut, MySQL menjadi database yang mayoritas sering dipakai, dan CodeIgniter secara default telah menyediakan konfigurasi untuk terkoneksi ke MySQL, sedangkan untuk driver pada DBMS lain masih perlu menggunakan library pihak ketiga.

Pada bab ini, kita akan belajar mengenai database MySQL, koneksi database MySQL, dan mencoba membuat sebuah model sederhana untuk menampilkan data dari database. Sehingga nantinya kita dapat memahami peran Model pada aplikasi berbasis CodeIgniter.

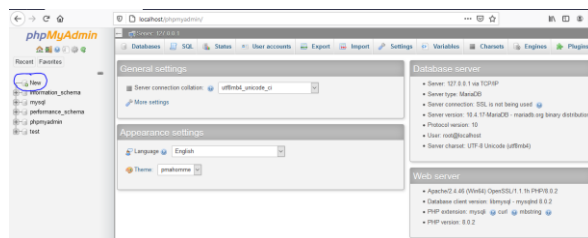
5.1. APA ITU MYSQL ?

MySQL merupakan perangkat lunak sistem manajemen basis data yang menggunakan bahasa SQL dalam pengelolaannya. DBMS ini open source yang memiliki dua buah lisensi, yaitu Free Software (*perangkat lunak gratis*) dan Shareware (*perangkat lunak yang pemakaiannya terbatas*). MySQL menjadi DBMS gratis yang juga memiliki lisensi GNU General Public License (GPL), sehingga dapat kita gunakan untuk kepentingan pribadi maupun komersial.

5.2. MEMBUAT DATABASE MYSQL

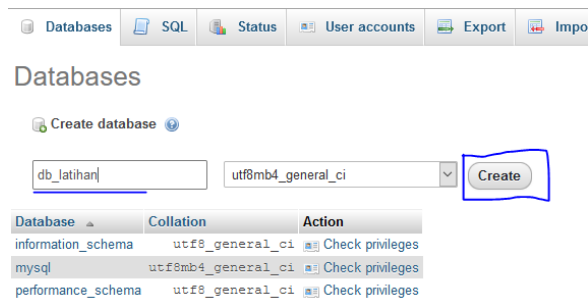
Database MySQL dapat kita buat menggunakan tools MySQL sendiri, maupun menggunakan database manager. Beberapa database manager yang biasa digunakan secara gratis, seperti PHPMyAdmin, MySQL Workbench, SQL Yog, Heidi SQL, dll. Namun dalam pembelajaran ini, kita akan menggunakan phpMyAdmin karena sudah tersedia saat menginstall pake xampp.

Pertama kita dapat menjalankan xampp, dengan mengaktifkan paket apache dan mysql. Kemudian buka phpMyAdmin pada web browser. Klik menu new untuk membuat database baru.



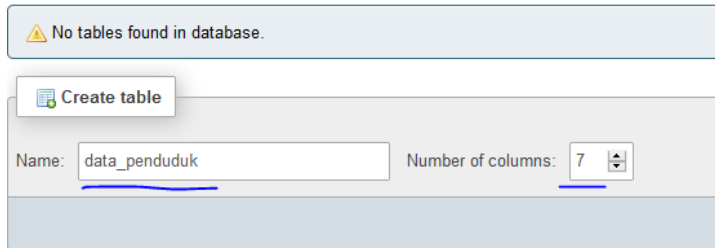
Gambar 5.1 Halaman Depan phpMyAdmin

Kemudian masukkan nama databasenya. Untuk pembelajaran ini kita dapat memberikan nama database *“db_latihan”*, dan klik tombol Create.



Gambar 5.2 Membuat Database Baru

Kemudian akan muncul halaman untuk membuat tabel baru, dan tambahkan nama tabel. Kita dapat memberikan nama tabel *“data_penduduk”* dengan 7 jumlah kolom. Nantinya dengan tabel ini kita akan membuat pengelolaan data kependudukan sederhana sebagai kasusnya.



⚠ No tables found in database.

Create table

Name: data_penduduk Number of columns: 7

Gambar 5.3 Membuat Tabel Database

Setelah itu, kita akan diarahkan untuk mengisi nama kolom. Kita dapat menambahkan nama kolom seperti gambar berikut, dan klik tombol save untuk menyimpan.

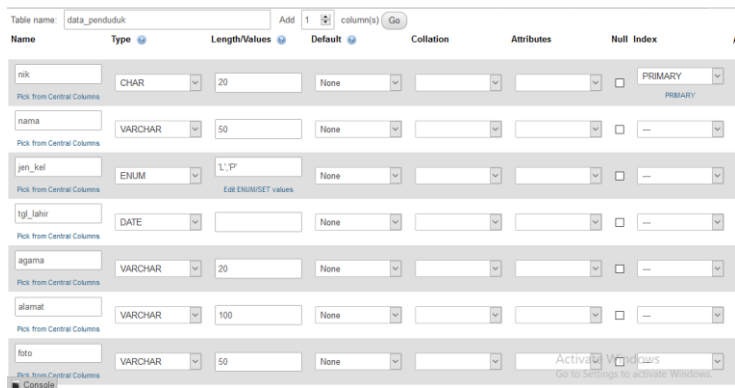


Table name: data_penduduk Add 1 column(s) Go

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index
<u>nik</u> Pick from Central Columns	CHAR	20	None			☐	PRIMARY
<u>nama</u> Pick from Central Columns	VARCHAR	50	None			☐	---
<u>jen_kel</u> Pick from Central Columns	ENUM	'L', 'P' Edit ENUMSET values	None			☐	---
<u>tgl_lahir</u> Pick from Central Columns	DATE		None			☐	---
<u>agama</u> Pick from Central Columns	VARCHAR	20	None			☐	---
<u>alamat</u> Pick from Central Columns	VARCHAR	100	None			☐	---
<u>foto</u> Pick from Central Columns	VARCHAR	50	None			☐	---

Active Windows
Go to Settings to activate Windows

Console

Gambar 5.4 Membuat Field Database

Kemudian kita buat tabel baru dengan nama *“penduduk_detail”* sebagai tabel detail dari data penduduk tersebut. Sesuaikan nama kolomnya seperti berikut.

Table name: Add 1 column(s) Go

Name	Type	Length/Values	Default	Collation	Attributes	Null Index
nik	CHAR	20	None			☐ INDEX [pk]
kecamatan	VARCHAR	100	None			☐ ---
kabupaten	VARCHAR	100	None			☐ ---

Gambar 5.5 Membuat Tabel Penduduk Detail

Setelah itu kita dapat menambahkan data dummy pada tabel *data_penduduk* tersebut. Data tersebut dapat ditambahkan melalui menu insert di phpMyAdmin atau menggunakan perintah query SQL.

Showing rows 0 - 2 (3 total, Query took 0.0008 seconds)

`SELECT * FROM `data_penduduk``

☐ Profiling [Edit inline] [Edit] [Explain SQL]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	Options	nik	nama	jen_kel	tgl_lahir	agama	alamat	foto
☐	Edit Copy Delete	3529071003200002	Ach. Faisol S. Arifin	L	1999-03-12	Islam	Jl. Asam Wulung, Sumenep	faisol.jpg
☐	Edit Copy Delete	3529071003200003	Moh. Abdullah	L	1999-03-10	Islam	Jl. Belimbing, Sumenep	abdul.jpg
☐	Edit Copy Delete	3529071003200004	Rudi Tabudi	L	1999-03-09	Islam	Jl. Salak, Sumenep	rudi.jpg

☐ Check all With selected: Edit Copy Delete Export

Gambar 5.6 Data pada Tabel Penduduk

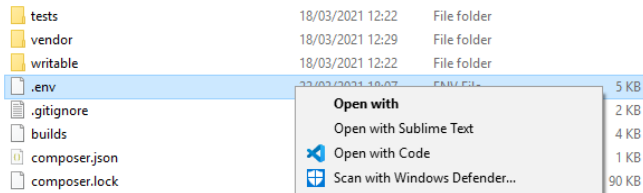
Kemudian tambahkan juga data pada tabel *penduduk_detail* yang relevan dengan data yang ada di tabel *data_penduduk*

nik	kecamatan	kabupaten
3529071003200002	Lenteng	Sumenep
3529071003200003	Butuan	Sumenep
3529071003200004	Kalianget	Sumenep
3529071003200005	Bluto	Sumenep

Gambar 5.7 Data pada Table Detail Penduduk

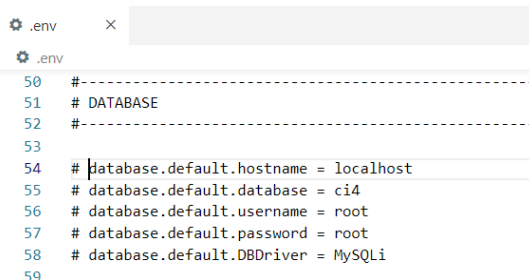
5.3. KONFIGURASI KONEKSI CODEIGNITER DENGAN MYSQL

Untuk mengkoneksikan project CodeIgniter dengan MySQL, kita dapat menkonfigurasi file environment (*.env*) yang terdapat pada folder root project kita. Kemudian buka menggunakan teks editor yang kita miliki.



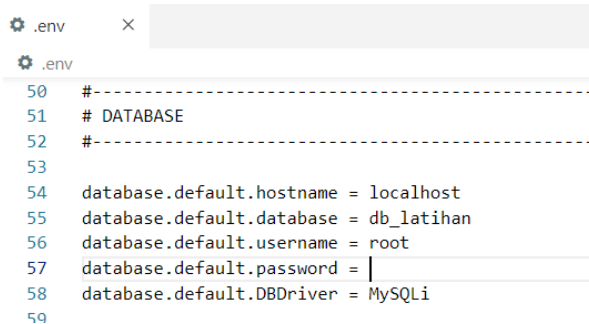
Gambar 5.8 File Environment

Kemudian cari bagian `#DATABASE` pada file environment tersebut. Terdapat dua sub konfigurasi database pada bagian tersebut, yaitu `database.default` dan `database.test`



Gambar 5.9 Mengkonfigurasi File Enviroment

Kemudian lakukan konfigurasi pada bagian `database.default` dengan mengubah hostname, database, username, dan password. Sesuaikan dengan database server yang kita miliki.



Gambar 5.10 Menkonfigurasi Database Environment

5.4. MENGENAL DAN MEMBUAT MODEL SEDERHANA

Model merupakan representasi dari struktur data. Sebuah model memiliki fungsi yang dapat melakukan aksi *create*, *read*, *update*, dan *delete* terhadap suatu tabel dalam database. Dalam CodeIgniter 4, sebuah model mewakili tabel tertentu dari database, dan didalamnya akan didefinisikan nama tabel dan primary key-nya. Model berada pada directory `/app/Models/`.

Untuk lebih memahami mengenai Model, mari kita buat sebuah Model menggunakan text editor. Buat file baru dengan nama `PendudukModel.php`, dan perlu diperhatikan nama Model harus menggunakan huruf besar diawal. Kemudian simpan file tersebut pada directory `/app/Models/`. Kemudian tambahkan code berikut untuk membuat Model yang menjadi representasi dari tabel `data_penduduk`.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class PendudukModel extends Model
```

```

{
    protected $table          = 'data_penduduk';
    protected $primaryKey     = 'nik';
    protected $useAutoIncrement = true;

    protected $returnType     = 'array';
    protected $useSoftDeletes = true;

    protected $allowedFields = ['nik', 'nama'];

    protected $useTimestamps = false;
    protected $createdField   = 'created_at';
    protected $updatedField   = 'updated_at';
    protected $deletedField   = 'deleted_at';
    protected $validationRules = [];
    protected $validationMessages = [];
    protected $skipValidation   = false;
}

```

Keterangan :

- `$table`, nama tabel database yang akan direpresentasikan oleh model tersebut
- `$primaryKey`, nama kolom unik untuk mengidentifikasi suatu baris dalam tabel tersebut
- `$useAutoIncrement`, untuk menentukan tabel menggunakan `autoincrement` pada `primary key`nya
- `$returnType`, untuk menentukan hasil yang dikembalikan saat memanggil data dalam tabel, hasil dapat berupa *array* maupun *object*.
- `$useSoftDeletes`, untuk menjadikan model tidak langsung menghapus baris data, melainkan masih menyimpannya dalam tempat sampah.
- `$allowedFields`, untuk menentukan kolom mana yang dapat di isi melalui `controller`.

- `$useTimestamps`, untuk memberikan tanggal dan waktu otomatis pada kolom `created_at`, `updated_at`, `deleted_at`
- `$createdField`, nama kolom untuk mencatat timestamp ketika data ditambahkan
- `$updatedField`, nama kolom untuk mencatat timestamp ketika data diperbaharui
- `$deletedField`, nama kolom untuk mencatat timestamp ketika terjadi softdelete
- `$validationRules`, untuk memberikan aturan validasi ketika data diinputkan.
- `validationMessages`, untuk menentukan pesan error ketika validasi berjalan
- `$skipValidation`, untuk menentukan akan digunakan validasi pada model atau tidak

Sebuah Model sudah tertanam method *query builder* yang bisa melakukan aksi CRUD. Model akan dipanggil pada sebuah controller dengan membentuk sebuah instance. Untuk memanggil Model `ModelPenduduk.php`, buat Controller dengan nama `Penduduk.php`, kemudian tuliskan code berikut untuk menggunakannya. Berikut merupakan method-method untuk mengambil data.

Didalam model terdapat method `->find()` untuk mengambil hanya satu baris data. Berikut contoh penerapannya.

```
<?php

namespace App\Controllers;
use App\Models\PendudukModel;

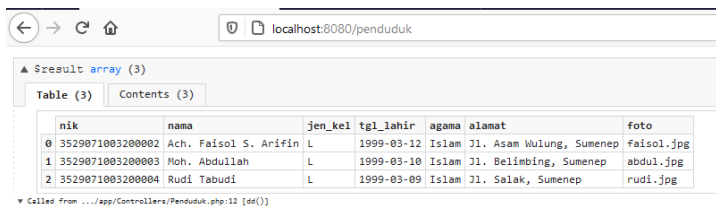
class Penduduk extends BaseController
{
    public function index()
```

```

    {
        $penduduk = new PendudukModel();
        $result = $penduduk->find();
        dd($result);
    }
}

```

Kemudian panggil controller penduduk tersebut di <http://localhost:8080/penduduk> untuk menampilkannya.



nik	nama	jen_kel	tgl_lahir	agama	alamat	foto
0 3529071003200002	Ach. Faisol S. Arifin	L	1999-03-12	Islam	Jl. Asam Wulung, Sumenep	faisol.jpg
1 3529071003200003	Moh. Abdullah	L	1999-03-10	Islam	Jl. Belimbing, Sumenep	abdul.jpg
2 3529071003200004	Rudi Tabudi	L	1999-03-09	Islam	Jl. Salak, Sumenep	rudi.jpg

Gambar 5.11 Menampilkan Model Penduduk

Didalam model juga terdapat method `->findAll()` untuk mengambil semua baris data, dan method `->where()` untuk memberikan kondisi. Berikut contoh penerapannya.

```

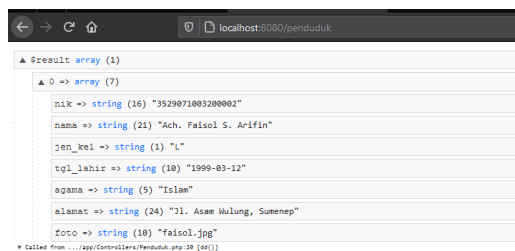
<?php

namespace App\Controllers;
use App\Models\PendudukModel;

class Penduduk extends BaseController
{
    public function index()
    {
        $penduduk = new PendudukModel();
        $result = $penduduk
            ->where('nik', 3529071003200002)
            ->findAll();
        dd($result);
    }
}

```

Dalam memanggil *model* digunakan *syntax* `use App\Models\PendudukModel`, yang mengarah ke lokasi *model* tersebut berada. Dari *model* yang sudah dipanggil dilakukan instance dengan *syntax* `$penduduk = new PendudukModel()`, dengan `$penduduk` sebagai representasi *objectnya*. Kemudian panggil kembali controller penduduk tersebut di `http://localhost:8080/penduduk` untuk menampilkannya.



Gambar 5.12 Menampilkan Menggunakan Kondisi

Didalam *model*, kita juga dapat memanggil data dari beberapa tabel sekaligus yang terelasi. Berikut penerapan untuk memanggil data dari tabel *data_penduduk* dan *detail_penduduk*. Pada *Model* `PendudukModel.php` lakukan modifikasi dengan menambahkan method `getPenduduk()` seperti berikut.

```
<?php
namespace App\Models;
use CodeIgniter\Model;

class PendudukModel extends Model
{
    protected $table          = 'data_penduduk';
    protected $primaryKey     = 'nik';
    protected $useAutoIncrement = true;

    protected $returnType     = 'array';
}
```

```

protected $useSoftDeletes = true;

public function getPenduduk()
{
    $this->join('penduduk_detail',
              'penduduk_detail.nik
              = data_penduduk.nik', 'LEFT');
    $this->select('penduduk_detail.*');
    $this->select('data_penduduk.*');
    return $this;
}
}

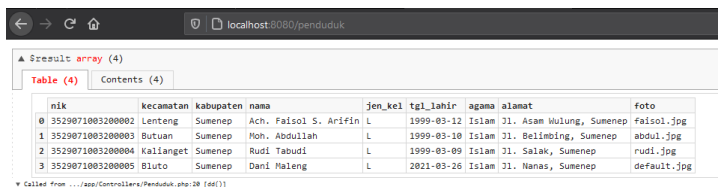
```

Kemudian pada Controller Penduduk.php sesuaikan code pada method `index()`, yaitu dengan memanggil method `getPenduduk()` yang dibuat dalam file Model.

```

<?php
namespace App\Controllers;
use App\Models\PendudukModel;
class Penduduk extends BaseController
{
    public function index()
    {
        $penduduk = new PendudukModel();
        $result = $penduduk
            ->getPenduduk()
            ->findAll();
        dd($result);
    }
}

```



▲ Result array (4)

Table (4) Contents (4)

	nik	kecamatan	kabupaten	nama	jen_kel	tgl_lahir	agama	alamat	foto
0	3529071003200002	Lenteng	Sumenep	Ach. Faisol S. Arifin	L	1999-03-12	Islam	71. Asan Wulung, Sumenep	faisol.jpg
1	3529071003200003	Butuan	Sumenep	Moh. Abdullah	L	1999-03-10	Islam	71. Belimbing, Sumenep	abdul.jpg
2	3529071003200004	Kaliangit	Sumenep	Rudi Tabudi	L	1999-03-09	Islam	71. Salak, Sumenep	rudi.jpg
3	3529071003200005	Bluto	Sumenep	Dani Maleng	L	2021-03-26	Islam	71. Nanas, Sumenep	default.jpg

▼ Called from .../app/Controllers/Penduduk.php:28 [dd()]

Gambar 5.13 Menampilkan dari Beberapa Tabel

BAB 6

CRUD DENGAN CODEIGNITER 4

Pengelolaan data pada sebuah sistem pasti akan melibatkan operasi CRUD didalamnya. Operasi CRUD menjadi empat jenis fungsionalitas dasar dalam mengelola data dalam suatu tabel. CRUD adalah singkatan dari *create*, *read*, *update*, *delete* yang akan melakukan fungsi membaca data, menambah data, menubah data, dan menghapus data. Fungsi tersebut sudah tertanam pada Model CodeIgniter yang disediakan dalam bentuk *query builder* atau sebuah method yang dapat melakukan setiap operasi tersebut.

Pada bab ini kita akan mempelajari tentang penerapan CRUD untuk mengelola data kependudukan sederhana menggunakan tabel *data_penduduk* yang telah dibuat pada bab sebelumnya.

6.1. MENAMPILKAN DATA DARI DATABASE

Menampilkan data merupakan operasi *read(R)* untuk mengambil/memanggil suatu data. Seperti menampilkan daftar produk pada website tertentu. Berikut merupakan contoh untuk menampilkan data dari tabel *data_penduduk*.

Buat file Model baru, kemudian simpan dengan nama *ModelPenduduk.php* pada folder */app/Models/*.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class PendudukModel extends Model
{
    protected $table      = 'data_penduduk';
    protected $primaryKey = 'nik';
```

```

protected $useTimestamps = false;
protected $returnType     = 'array';
protected $allowedFields = ['nik',
                            'nama', 'jen_kel', 'tgl_lahir',
                            'agama', 'alamat', 'foto'];
}

```

Pada code merupakan Model yang menjadi representasi dari tabel data_penduduk. Tabel didefinisikan pada atribut \$table, dan *primary key* pada \$primaryKey.

\$useTimestamps merupakan kondisi untuk memberikan value otomatis pada kolom *create_at* dan *update_at*, jika memiliki kolom tersebut pada tabel.

\$returnType untuk menjadikan model memberikan output *array* pada saat melakukan select data, kita juga bisa bisa mengembalikan dalam bentuk *object*.

\$allowedField untuk mengizinkan model agar bisa mengisikan data pada kolom yang ditentukan tersebut.

Kemudian buat View untuk menampilkan data dalam bentuk tabular. Simpan View tersebut dengan nama data_pdd.php, dan tambahkan code berikut.

```

<!doctype html>
<html lang="en">
<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-
width, initial-scale=1">
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivrivr.net/npm/bootst
rap@5.0.0-
beta3/dist/css/bootstrap.min.css" rel="styleshe
et" integrity="sha384-
eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+
rP48ckxlpbzKgwra6" crossorigin="anonymous">

```

```

<title>Data Penduduk</title>
</head>
<body>
<div class="container">
    <div class="row">
        <div class="col-10 mt-4">
            <a href="/penduduk/tambah">
                <button class="btn btn
                    -primary">Tambah</button>
            </a>
            <table class="table table
                -bordered mt-3">
                <tr>
                    <th scope="col">NIK</th>
                    <th scope="col">Nama</th>
                    <th scope="col">Jenis Kelamin</th>
                    <th scope="col">Tanggal Lahir</th>
                    <th scope="col">Agama</th>
                    <th scope="col">Alamat</th>
                    <th scope="col">Foto</th>
                </tr>
                <?php
                    foreach($data as $row):
                        ?>
                        <tr>
                            <td><?= $row['nik'] ?></td>
                            <td><?= $row['nama'] ?></td>
                            <td><?= $row['jen_kel'] ?></td>
                            <td><?= $row['tgl_lahir'] ?></td>
                            <td><?= $row['agama'] ?></td>
                            <td><?= $row['alamat'] ?></td>
                            <td><?= $row['foto'] ?></td>
                        </tr>
                        <?php
                            endforeach;
                        ?>
                    </table>
                </div>
            </div>
        </div>
    </body>
</html>

```

Kemudian kita buat Controller baru untuk memanggil Model dan View diatas untuk menampilkannya. Simpan dengan nama `Penduduk.php`, dan tambahkan code berikut pada Controller.

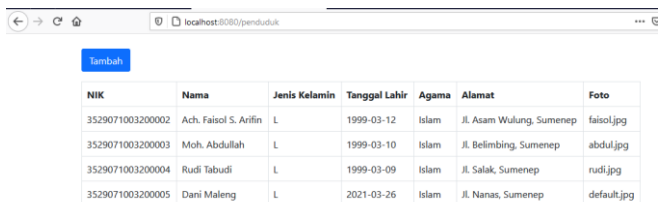
```
<?php

namespace App\Controllers;
use App\Models\PendudukModel;

class Penduduk extends BaseController
{
    private $penduduk;

    public function __construct()
    {
        $this
            ->penduduk = new PendudukModel();
    }
    public function index()
    {
        $result = $this->penduduk->findAll();
        return view('data_pdd', [
            'data' => $result
        ]);
    }
}
```

Pada controller diatas, kita memanggil model dengan `use App\Models\PendudukModel`, dan dilakukan *instance* untuk atribut `$penduduk` pada bagian konstruktor. Untuk memanggil semua data, kita menggunakan method `->findAll()` dari Model penduduk, dan data tersebut kita passing pada View `data_pdd`. Kemudian panggil controller tersebut pada web browser, maka akan tampil seperti berikut.



NIK	Nama	Jenis Kelamin	Tanggal Lahir	Agama	Alamat	Foto
3529071003200002	Ach. Faisol S. Arifin	L	1999-03-12	Islam	Jl. Asam Wulung, Sumenep	faisol.jpg
3529071003200003	Moh. Abdullah	L	1999-03-10	Islam	Jl. Belimbing, Sumenep	abdul.jpg
3529071003200004	Rudi Tabudi	L	1999-03-09	Islam	Jl. Salak, Sumenep	rudi.jpg
3529071003200005	Dani Maleng	L	2021-03-26	Islam	Jl. Nanas, Sumenep	default.jpg

Gambar 6.1 Menampilkan Data

6.2. MEMBUAT INSERT DATA

Insert data merupakan operasi *Create(C)* untuk membuat data baru. Seperti ketika registrasi pada website tertentu yang berarti telah melakukan operasi *create* untuk membuat dan menyimpan data registrasi ke database. Berikut ini merupakan contoh untuk melakukan insert data ke tabel *data_penduduk*.

Buat file View baru untuk membuat tampilan form inputan. Kemudian simpan dengan nama `tambah_pdd.php`, dan tambahkan code berikut.

```
<!doctype html>
<html lang="en">
<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-
width, initial-scale=1">
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootst
rap@5.0.0-
beta3/dist/css/bootstrap.min.css" rel="styleshe
et" integrity="sha384-
eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+
rP48ckxlpbzKgwra6" crossorigin="anonymous">
<title>Data Penduduk</title>
</head>
<body>
<div class="container">
  <div class="row">
    <div class="col-5 mt-4">
      <a href="/penduduk">
```

```

<button class="btn btn
-primary"> << Data Penduduk</button>
</a>
<form action="/penduduk " method="post">
  <?= csrf_field(); ?>
  <div class="my-3">
    <label class="form-label">NIK</label>
    <input type="text" class="form-
      control" name="nik">
  </div>
  <div class="mb-3">
    <label class="form-label">Nama</label>
    <input type="text" class="form-
      control" name="nama">
  </div>
  <div class="mb-3">
    <label class="form
      -label">Jenis Kelamin</label>
    <div class="form-check">
      <input class="form-check-input"
        type="radio" name="jenkel" value="L">
      <label class="form-check-label">
        Laki-Laki
      </label>
    </div>
    <div class="form-check">
      <input class="form-check-input"
        type="radio" name="jenkel" value="P">
      <label class="form-check-label">
        Perempuan
      </label>
    </div>
  </div>
  <div class="mb-3">
    <label class="form-label">TanggalLahir
    </label>
    <input type="date" class="form-
      control" name="tgl_lahir">
  </div>
  <div class="mb-3">
    <label class="form-label">Agama
    </label>
    <input type="text" class="form-

```

```

        control" name="agama">
    </div>
    <div class="mb-3">
        <label class="form-label">Alamat
    </label>
        <input type="text" class="form-
            control" name="alamat">
    </div>
    <button type="submit" class="btn btn-
        primary">Simpan</button>
    </form>
</div>
</div>
</div>
</body>
</html>

```

Setelah itu pada Controller Penduduk.php tambahkan method `tambah()`, dan `save()` untuk memanggil view form inputan, dan menyimpan data pada database. Tambahkan code berikut setelah method `index()`.

```

public function tambah()
{
    return view('tambah_pdd');
}
public function save()
{
    $foto = 'default.jpg';
    $this->penduduk->insert([
        'nik' => $this->request->getPost('nik'),
        'nama' => $this->request->getPost('nama'),
        'jen_kel' => $this->request
            ->getPost('jenkel'),
        'tgl_lahir' => $this->request
            ->getPost('tgl_lahir'),
        'agama' => $this->request
            ->getPost('agama'),
        'alamat' => $this->request
            ->getPost('alamat'),
        'foto' => $foto
    ]);
}

```

```
]);  
return redirect()->to('/penduduk');  
}
```

Pada method `save()`, kita menggunakan menggunakan method `->insert` untuk menambahkan data pada database, method tersebut berasal dari model penduduk. Kemudian kita mengirimkan data yang ditangkap dari form dalam bentuk array asosiatif sebagai argumentnya. Untuk menangkap data dari form kita gunakan `$this->request->getPost('name')`, jika metode pengirimannya berupa POST, sedangkan jika get bisa dilakukan dengan `$this->request->getVar('name')`.

Kemudian pada file routing yang terletak di `/app/Config/Routes.php`, tambahkan route berikut untuk menangani rute sesuai dengan request method yang diberikan.

```
//crud routing  
$routes->get('/penduduk', 'Penduduk::index');  
$routes->post('/penduduk', 'Penduduk::save');
```

Pada bagian `$routes->post`, kita gunakan request method POST, karena untuk menambah data. Selanjutnya route akan mengarahkan pada method `save` di controller `Penduduk`.

<< Data Penduduk

NIK

Nama

Jenis Kelamin

☐ Laki-Laki

☐ Perempuan

Tanggal Lahir

mm / dd / yyyy

Agama

Alamat

Simpan

Gambar 6.2 Form Inputan

6.3. MEMBUAT DELETE DATA

Delete data merupakan operasi *Delete(D)* untuk menghapus data yang ada didatabase. Berikut ini contoh untuk menghapus data dari tabel *data_penduduk*.

Pada view *data_pdd.php*, lakukan perubahan tag tabel dengan menambahkan kolom untuk tombol hapus. Kita dapat menyesuaikan seperti code berikut.

```
<table class="table table-bordered mt-3">
  <tr>
    <th scope="col">NIK</th>
    <th scope="col">Nama</th>
    <th scope="col">Jenis Kelamin</th>
    <th scope="col">Tanggal Lahir</th>
    <th scope="col">Agama</th>
    <th scope="col">Alamat</th>
    <th scope="col">Foto</th>
    <th scope="col">Aksi</th>
  </tr>
</table>
<?php
```

```

        foreach($data as $row):
            ?>
            <tr>
                <td><?= $row['nik'] ?></td>
                <td><?= $row['nama'] ?></td>
                <td><?= $row['jen_kel'] ?></td>
                <td><?= $row['tgl_lahir'] ?></td>
                <td><?= $row['agama'] ?></td>
                <td><?= $row['alamat'] ?></td>
                <td><?= $row['foto'] ?></td>
                <td>
                    <form action="<?= base_url("penduduk/
                    {$row[nik]}") ?>" method="post">
                        <input type="hidden" name="_method"
                        value="DELETE">
                        <button class="btn
                        btn-danger">Hapus</button>
                    </form>
                </td>
            </tr>
        <?php
        endforeach;
    ?>
</table>

```

Pada bagian tombol hapus, kita lakukan HTTP method spoofing, untuk menghasilkan request method DELETE pada bagian `<input type="hidden" name="_method" value="DELETE">`. Kemudian tambahkan method `delete()` untuk menghapus data dari tabel `data_penduduk`. Kita menggunakan method `->delete($id)` yang dimiliki oleh model `penduduk`.

```

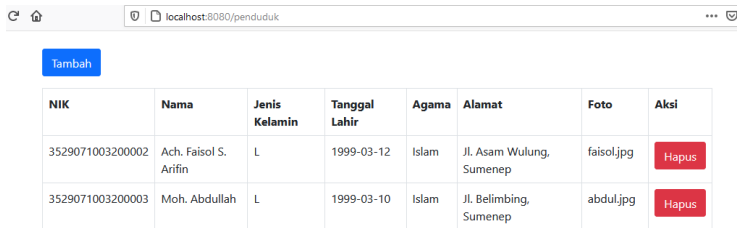
public function delete($id)
{
    $this->penduduk->delete($id);
    return redirect()->to('/penduduk');
}

```

Kemudian file `Routes.php` kita tambahkan route untuk mengarahkan pada method `delete()` di controller `Penduduk`.

```
//crud routing
$routes->get('/penduduk', 'Penduduk::index');
$routes->post('/penduduk', 'Penduduk::save');
$routes->delete('/penduduk/(:num)',
'Penduduk::delete/$1');
```

Pada bagian `$route->delete`, kita gunakan request method DELETE untuk menangani ketika terjadi request untuk menghapus.



The screenshot shows a web browser at localhost:8080/penduduk. There is a blue 'Tambah' button at the top left. Below it is a table with 8 columns: NIK, Nama, Jenis Kelamin, Tanggal Lahir, Agama, Alamat, Foto, and Aksi. There are two rows of data, each with a red 'Hapus' button in the 'Aksi' column.

NIK	Nama	Jenis Kelamin	Tanggal Lahir	Agama	Alamat	Foto	Aksi
3529071003200002	Ach. Faisol S. Arifin	L	1999-03-12	Islam	Jl. Asam Wulung, Sumenep	faisol.jpg	Hapus
3529071003200003	Moh. Abdullah	L	1999-03-10	Islam	Jl. Belimbing, Sumenep	abdul.jpg	Hapus

Gambar 6.3 Hapus Data

6.4. MEMBUAT UPDATE DATA

Update data merupakan operasi *Update(U)* untuk memperbaharui suatu data dalam database. Seperti mengubah profil social media yang akan mengganti data lama dengan data baru. Berikut ini merupakan contoh untuk melakukan update data pada tabel *data_penduduk*.

Pada view `data_pdd.php`, kita tambahkan tombol yang mengarah pada form edit. Sesuaikan view tersebut dengan code berikut.

```
<tr>
<td><?= $row['nik'] ?></td>
<td><?= $row['nama'] ?></td>
<td><?= $row['jen_kel'] ?></td>
<td><?= $row['tgl_lahir'] ?></td>
<td><?= $row['agama'] ?></td>
<td><?= $row['alamat'] ?></td>
<td><?= $row['foto'] ?></td>
<td>
```

```

<form action="<?= base_url("penduduk/
{$row[nik']}") ?>" method="post">
  <input type="hidden" name="_method"
    value="DELETE">
    <button class="btn
      btn-danger">Hapus</button>
</form>
<a href="<?= base_url("penduduk/
{$row['nik']}/edit") ?>">
  <button class="btn btn-warning
    my-1">Edit</button></a>
</td>
</tr>

```

Kemudian kita buat view untuk menampilkan form edit, dengan membuat file php dan simpan dengan nama `edit_pdd.php`, tambahkan code berikut pada view tersebut.

```

<!doctype html>
<html lang="en">
<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-
width, initial-scale=1">
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootst
rap@5.0.0-
beta3/dist/css/bootstrap.min.css" rel="styleshe
et" integrity="sha384-
eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+
rP48ckxlpbzKgwra6" crossorigin="anonymous">
<title>Data Penduduk</title>
</head>
<body>
<div class="container">
  <div class="row">
    <div class="col-5 mt-4">
      <a href="/penduduk">
        <button class="btn btn-primary"> << Data

```

```

        Penduduk</button>
    </a>
    <form action="/penduduk" method="post">
        <?= csrf_field(); ?>
        <input type="hidden" name="_method"
            value="PUT">
        <input type="hidden" name="id"
            value="<?= $data['nik'] ?>">
        <div class="my-3">
            <label class="form-label">NIK</label>
            <input type="text" class="form
                -control" name="nik" value="
                    <?= $data['nik'] ?>">
        </div>
        <div class="mb-3">
            <label class="form-label">Nama</label>
            <input type="text" class="form
                -control" name="nama" value="
                    <?= $data['nama'] ?>">
        </div>
        <div class="mb-3">
            <label class="form-label">Jenis Kelamin
            </label>
            <div class="form-check">
                <input class="form-check-input"
                    type="radio" name="jenkel" value="L"
                    <?= $data['jen_kel']=='L'
                        ? 'checked' : '' ?>>
                <label class="form-check-label">
                    Laki-Laki
                </label>
            </div>
            <div class="form-check">
                <input class="form-check-input"
                    type="radio" name="jenkel" value="P"
                    <?= $data['jen_kel']=='P'
                        ? 'checked' : '' ?>>
                <label class="form-check-label">
                    Perempuan
                </label>
            </div>
        </div>
    </div>

```

```

<div class="mb-3">
  <label class="form-label">Tanggal Lahir
</label>
  <input type="date" class="form-control"
    name="tgl_lahir" value="
    <?= $data['tgl_lahir'] ?>">
</div>
  <div class="mb-3">
    <label class="form-label">Agama</label>
    <input type="text" class="form-control"
      name="agama" value="
      <?= $data['agama'] ?>">
  </div>
  <div class="mb-3">
    <label class="form-label">Alamat</label>
    <input type="text" class="form-control"
      name="alamat" value="
      <?= $data['alamat'] ?>">
  </div>
  <button type="submit" class="btn btn
    -primary">Ubah</button>
</form>
</div>
</div>
</body>
</html>

```

Pada view diatas, kita memberikan value pada setiap inputan, value tersebut didapat dari array \$data yang dipassing dari controller penduduk. Dengan memberikan `<input type="hidden" name="_method" value="DELETE">`, akan melakukan HTTP method spoofing yang menghasilkan request method PUT. Kemudian buat route baru pada `Routes.php`, tambahkan dua route baru untuk mengarahkan pada form edit, dan untuk memproses update data.

```
//crud routing
```

```

$routes->get('/penduduk', 'Penduduk::index');
$routes->post('/penduduk', 'Penduduk::save');
$routes-
>delete('/penduduk/(:num)', 'Penduduk::delete/$
1');
$routes
  ->get('/penduduk/(:num)/edit',
    'Penduduk::edit/$1');
$routes->put('/penduduk',
  'Penduduk::update/$1');

```

Pada bagian `$route->put`, kita menangani request method PUT untuk mengarahkan pada method `update()` di controller Penduduk.

Setelah itu tambahkan method `edit()` dan `update()` pada controller Penduduk, setelah method `delete()`. Tambahkan code berikut pada method-method tersebut.

```

public function edit($id)
{
    return view('edit_pdd', [
        'data' => $this->penduduk
        ->where(['nik' => $id])->first()
    ]);
}
public function update()
{
    $foto = 'default.jpg';
    $this->penduduk->update(
        ['nik' => $this->request->getPost('id')],
        [
            'nik' => $this->request->getPost('nik'),
            'nama' => $this->request
                ->getPost('nama'),
            'jen_kel' => $this->request
                ->getPost('jenkel'),
            'tgl_lahir' => $this->request
                ->getPost('tgl_lahir'),

```

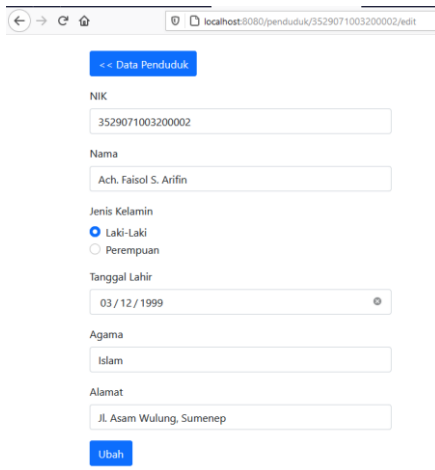
```

        'agama' => $this->request
            ->getPost('agama'),
        'alamat' => $this->request
            ->getPost('alamat'),
        'foto' => $foto
    ]);
    return redirect()->to('/penduduk');
}

```

Pada method `edit()` memanggil view `edit_pdd.php` dan mempassing data yang didapat dari model berdasarkan *primary key*, yaitu `nik`, dan digunakan method `->first()` untuk mengambil hasil pertama.

Kemudian kita dapat menekan tombol edit pada data penduduk, maka akan tampil seperti berikut.



The screenshot shows a web browser window with the address bar displaying `localhost:8080/penduduk/3529071003200002/edit`. The page has a blue header bar with a button labeled "<< Data Penduduk". Below the header, the form contains the following fields:

- NIK**: A text input field containing the value "3529071003200002".
- Nama**: A text input field containing the value "Ach. Faisol S. Arifin".
- Jenis Kelamin**: Two radio button options, "Laki-Laki" (which is selected) and "Perempuan".
- Tanggal Lahir**: A date picker field showing "03 / 12 / 1999".
- Agama**: A text input field containing the value "Islam".
- Alamat**: A text input field containing the value "Jl. Asam Wulung, Sumenep".

At the bottom of the form is a blue button labeled "Ubah".

Gambar 6.4 Form Edit Data

BAB 7

PAGINATION DAN PENCARIAN DATA

Pada bab sebelumnya kita sudah membuat pengelolaan data penduduk yang dapat menampilkan, menambah, menghapus, dan mengubah data tersebut. Dalam kasus nyata, sebuah tabel dalam database biasanya memiliki banyak data, bisa ratusan, ribuan, bahkan sampai jutaan *record* data. Hal tersebut tidak bagus jika ditampilkan hanya dalam satu halaman saja, kita membutuhkan fitur pagination untuk menampilkan data menjadi beberapa list data saja dalam satu halaman dan lanjut pada halaman berikutnya. Begitu pula ketika mencari data, akan menjadi sulit jika mencarinya satu persatu, maka kita perlu membuatkan fitur pencarian dalam menampilkan sebuah data.

7.1. MEMBUAT PAGINATION

Pagination merupakan sebuah fitur untuk membatasi jumlah data yang ditampilkan pada suatu halaman agar tidak terlalu panjang dan lebih *user friendly*. Penggunaan pagination umumnya digunakan saat menampilkan list data yang jumlahnya banyak, sehnigga bisa memisah data menjadi beberapa data saja yang ditampilkan. CodeIgniter sudah menyediakan fitur tersebut didalamnya, kita dapat menggunakan method `->paginate()` pada saat memanggil model untuk menggantikan method `->findAll()`. Berikut ini merupakan penggunaan pagination.

Pada controller `Penduduk.php`, lakukan modifikasi pada bagian method `index()`, sesuaikan method `index()` seperti code berikut.

```
public function index()
{
    $result = $this->penduduk
```

```

        ->paginate(2, 'archive');
    return view('data_pdd', [
        'data' => $result,
        'pager' => $this->penduduk->pager,
    ]);
}

```

Pada controller diatas kita menggunakan method `->paginate` dari model penduduk. Terdapat 2 argument yang diberikan, argument pertama digunakan untuk menentukan banyaknya data yang ditampilkan pada setiap halaman. Dan argument kedua menjadi identitas paginasinya. Pada saat mereturn view, kita juga mempassing `'pager' => $this->penduduk->pager` untuk memberikan tombol paginasi.

Kemudian buat view baru untuk tampilan tombol paginationnya. Simpan dengan nama `pager.php` pada folder `\app\Views\`. Tambahkan code berikut pada view tersebut.

```

<?php
/**
 * @var \CodeIgniter\Pager\PagerRenderer $pager
 */

$pager->setSurroundCount(2);
?>
<nav aria-label="<?=>
lang('Pager.pageNavigation') ?>">
    <ul class="pagination">
        <?php if ($pager->hasPrevious()) : ?>
            <li class="page-item">
                <a href="<?=> $pager->getPrevious() ?>"
class="page-link" aria-label="<?=>
lang('Pager.previous') ?>">
                    <span><</span>
                </a>
            </li>
        <?php endif ?>

```

```

        <?php foreach ($pager->links() as $link) :
        ?>
            <li <?= $link['active'] ? 'class="active
page-item"' : 'class="page-item"' ?>>
                <a href="<?= $link['uri'] ?>"
class="page-link">
                    <?= $link['title'] ?>
                </a>
            </li>
        <?php endforeach ?>

        <?php if ($pager->hasNext()) : ?>
            <li class="page-item">
                <a href="<?= $pager->getNext() ?>"
aria-label="<?= lang('Pager.next') ?>"
class="page-link">
                    <span aria-hidden="true">></span>
                </a>
            </li>
        <?php endif ?>
    </ul>
</nav>

```

Setelah membuat template tombol paginasi, kita harus mendaftarkan view tersebut agar dikenal oleh project CodeIgniter kita, dan dapat digunakan secara global. Untuk mendaftarkannya kita perlu menambahkan nama template dan lokasi template paginasi tersebut berada. Lakukan registrasi pada file `Pager.php` dalam directory `\app\Config\`. Temukan atribut `public $templates`, dan tambahkan code seperti berikut.

```

public $templates = [
    'default_full' =>
        'CodeIgniter\Pager\Views\default_full',
    'default_simple' =>
        'CodeIgniter\Pager\Views\default_simple',
    'default_head' =>
        'CodeIgniter\Pager\Views\default_head',
    'bs_paginate' => 'App\Views\pager',
];

```

Kemudian untuk menampilkan pagination pada data penduduk tersebut, tambahkan `<?php echo $pager->links('archive', 'bs_paginate'); ?>` pada view `data_pdd.php`. tambahkan code tersebut setelah tag penutup `</table>`.

```

.
.
.
</tr>
<?php
    endforeach;
?>
</table>
<?php echo $pager->links('archive',
    'bs_paginate'); ?>
</div>
</div>

```

Pada code tersebut digunakan method `->links()` dengan dua argument, yaitu identitas dari pagination, dan template pagination yang telah diregistrasikan pada `public $templates`. Kemudian kita dapat mengakses controller penduduk, maka akan tampil sebuah tabel dengan pagination dibawahnya.



Tambah

NIK	Nama	Jenis Kelamin	Tanggal Lahir	Agama	Alamat	Foto	Aksi
3529071003200004	Rudi Tabudi	L	1999-03-09	Islam	Jl. Salak, Sumenep	rudijpg	Hapus Edit
3529071003200005	Dani Maleng	L	2021-03-26	Islam	Jl. Nanas, Sumenep	default.jpg	Hapus Edit

1 2

Gambar 7.1 Tampilan Pagination

7.2. MEMBUAT PENCARIAN DATA

Pencarian data merupakan fitur yang dapat mempermudah pengguna untuk menemukan sebuah data. Dalam fitur pencarian memiliki prinsip dengan memfilter data saat dipanggil dengan memberikan *keyword* tertentu sebagai acuan. CodeIgniter sudah menyediakan method `->like()` dan `->orLike()` pada sebuah model yang berprinsip seperti perintah SQL *"where kolom like '%keyword%' or like %keyword%"*. Berikut contoh penerapan pencarian data untuk pengelolaan data penduduk sebelumnya.

Pada bagian view `data_pdd.php`, lakukan modifikasi dengan menambahkan form inputan untuk mencari data. Sesuaikan code view tersebut dengan code berikut.

```
<div class="container">
  <div class="row">
    <div class="col-10 mt-4">
      <div class="row">
        <div class="col-3">
          <a href="/penduduk/tambah">
            <button class="btn btn
              -primary">Tambah</button>
          </a>
        </div>
        <div class="col-9">
          <form method="get">
            <div class="input-group float
              -right">
              <input type="text"
                class="form-control" placeholder=
                "Cari Penduduk" name=q>
              <button class="btn btn-success">
                Cari</button>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
```

```

        <table class="table table-bordered mt-3">
            .
            .
            .
        </table>
        <?php echo $pager->links('archive',
'bs_paginate'); ?>
    </div>
</div>
</div>

```

Kemudian pada model `PendudukModel.php`, tambahkan method `search()` agar dapat memfilter data yang dipanggil berdasarkan field *nik*, *nama*, *alamat*.

```

public function search($q) {
    return $this->like('nik', $q)
        ->orLike('nama', $q)
        ->orLike('alamat', $q);
}

```

Pada method tersebut digunakan method `->like()`, dan `->orLike`, yang merupakan *query builder* dari sebuah model yang memiliki fungsi sama dengan perintah SQL "Like %teks%". Kemudian pada controller `Penduduk.php`, lakukan modifikasi ulang pada *statement* method `index()`, sesuaikan seperti code berikut.

```

public function index()
{
    $result = $this->penduduk;

    if ($this->request->getGet('q')){
        $result = $this->penduduk
            ->search($this->request
            ->getGet('q') ?? '');
    }
    return view('data_pdd', [

```

```

        'data' => $result->paginate(2, 'archive'),
        'pager' => $this->penduduk->pager,
    ] );
}

```

Pada variabel `$result` diinisiasi dengan model penduduk, namun jika terdapat request method get untuk variabel 'q', `$result` akan *diassign* dengan hasil filter menggunakan method `->search()` yang telah kita buat sebelumnya pada `PendudukModel.php`. Kemudian kita dapat mencari data pada tabel penduduk, dengan menginputkan kata kunci dari *record* yang ingin dicari.

NIK	Nama	Jenis Kelamin	Tanggal Lahir	Agama	Alamat	Foto	Aksi
3529071003200002	Ach. Faisol S. Arifin	L	1999-03-12	Islam	Jl. Asam Wulung, Sumenep	faisol.jpg	Hapus Edit

Gambar 7.2 Pencarian Data

BAB 8

UPLOAD FILE, PENANGANAN DAN VALIDASI FORM

Form merupakan fitur sensitif yang ada dalam aplikasi, karena menjadi tempat pengguna memasukkan data untuk dikirimkan ke sistem, seperti form login, form register, komentar dan lain sebagainya. Sebuah form harus meliwati validasi untuk mencegah kesalahan dalam mengisi inputan sehingga data yang dimasukkan sesuai dengan spesifikasi yang dibutuhkan oleh sistem. Pada bab ini kita akan mempelajari bagaimana untuk memvalidasi form dan upload file menggunakan method yang disediakan CodeIgniter.

8.1. MENGENAL VALIDASI FORM

Validasi form merupakan langkah untuk memverifikasi data yang dikirimkan dari form sehingga mendapatkan *feedback* informasi yang berupa pesan error atau pesan sukses. Validasi dapat dilakukan pada sisi client maupun server. CodeIgniter telah menyediakan class validasi data yang komprehensif yang dapat membantu meminimalkan code saat membuat validasi pada sebuah form. Validasi dapat kita terapkan dengan memanggil service `\Config\Services::validation()` yang didalamnya terdiri dari beberapa method. Berikut method untuk menuliskan rule yang terdapat dalam service tersebut.

Method berikut digunakan untuk memvalidasi satu inputan

```
$validation->setRule('name', 'Nama', 'rule');
```

Untuk memvalidasi beberapa form sekaligus menggunakan method berikut

```
$validation->setRules([  
    //Rules
```

```
]);
```

untuk menjalankan validasi tersebut dapat digunakan method berikut

```
$validation->run();
```

Validasi juga bisa telah ditanamkan pada controller dengan method berikut

```
if($this->validate([  
    //Rules  
]){#statement}
```

Dalam penerapan validasi kita mendefinisikan rule didalamnya untuk inputan yang akan divalidasi. Berikut rule-rule validasi yang disediakan oleh CodeIgniter.

Rule	Keterangan
alpha	Membolehkan inputan hanya alfabet
alpha_space	Membolehkan inputan berisi alfabet dan spasi
alpha_dash	Membolehkan inputan berisi alfabet dan dashes
alpha_numeric	Membolehkan inputan berisi alfabet dan angka
alpha_numeric_space	Membolehkan inputan berisi alfabet, angka, dan spasi
alpha_numeric_punct	Membolehkan inputan berisi alfabet, angka, dan tanda baca
decimal	Membolehkan inputan hanya angka desimal
hex	Membolehkan inputan hanya hexadesimal
if_exist[]	Nilai inputan harus berada pada daftar yang pada parameter

in_list[]	Membolehkan inputan hanya nilai yang berada pada list
integer	Membolehkan inputan hanya bilangan bulat
is_natural	Membolehkan inputan hanya bilangan asli
is_natural_no_zero	Membolehkan inputan hanya bilangan asli tanpa nol
is_unique[]	Memeriksa nilai apakah sudah ada didatabase
less_than[]	Membolehkan nilai inputan kurang dari parameter
less_than_equal_to[]	Membolehkan nilai inputan kurang dari atau sama dengan di parameter
Matches[]	Membolehkan nilai inputan harus sama dengan di parameter
max_length[]	Membolehkan panjang inputan kurang dari parameter
min_length[]	Membolehkan panjang inputan lebih dari parameter
not_in_list	Membolehkan inputan hanya nilai yang tidak ada pada list
numeric	Membolehkan inputan hanya angka numeric
regex_match[]	Nilai inputan harus sama dengan reguler expression di parameter
permit_empty	Mengizinkan inputan boleh kosong
required	Inputan tidak boleh dikosongkan
valid_base64	Mengizinkan inputan hanya karakter base64
valid_json	Mengizinkan inputan harus karakter json
valid_email	Mengharuskan inputan berisi alamat email
valid_ip[]	Mengharuskan inputan berisi IP Address
valid_url	Mengharuskan inputan berisi URL
valid_date	Mengharuskan inputan adalah tanggal

valid_cc _number[]	Memverifikasi bahwa nomor kartu kredit sesuai dengan format yang digunakan oleh penyedia yang ditentukan
Ket. [] Penulisan rule menggunakan parameter	

8.2. MEMBUAT VALIDASI FORM

Berikut contoh penerapan validasi untuk tiga inputan. Buat view baru untuk tampilan form input dan pesan error validasi. Simpan view tersebut dengan nama `daftar.php`, dan tambahkan code berikut.

```
<!doctype html>
<html lang="en">
<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-
width, initial-scale=1">
<!-- Bootstrap CSS -->
<link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.
0.0-beta3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+
rP48ckxlpbzKgwra6" crossorigin="anonymous">
<title>Pendaftaran</title>
</head>
<body>
<div class="container">
<div class="row">
<div class="col-5 mt-4">
<?= $validation->listErrors() ?>
<form action="/validasi/save"
method="post">
<?= csrf_field(); ?>
<div class="my-3">
<label class="form-label">Nama</label>
<input type="text" class="form
```

```

        -control" name="nama"
        value="<?= old('nama') ?>"
        <?= $validation->getError('nama') ?>
    </div>
    <div class="mb-3">
        <label class="form-label">Email</label>
        <input type="text" class="form
        -control" name="email"
        value="<?= old('email') ?>"
        <?= $validation->getError('email') ?>
    </div>
    <div class="mb-3">
        <label class="form-label">
            No. Telp</label>
        <input type="text" class="form
        -control" name="telp"
        value="<?= old('telp') ?>"
        <?= $validation->getError('telp') ?>
    </div>
        <button type="submit" class="btn btn
        -primary">Daftar</button>
    </form>
</div>
</div>
</body>
</html>

```

Pada view diatas digunakan `<?= $validation->listErrors() ?>` untuk menampilkan semua list error yang dihasilkan oleh validasi. Untuk menampilkan pesan error sesuai inputannya, dapat menggunakan *syntax* `<?= $validation->getError('name') ?>`.

Agar value yang diinputkan sebelumnya tidak hilang saat salah satu form tidak lolos validasi, fungsi `old('name')` dapat digunakan untuk mengembalikan value tersebut.

Kemudian pada `BaseController.php` yang berada di directory `/app/Controller/`, aktifkan service session agar *flash message*

validasi dapat berjalan. Sesuaikan code pada method `initController()` seperti berikut.

```
public function initController(RequestInterface
$request, ResponseInterface $response,
LoggerInterface $logger)
{
    // Do Not Edit This Line
    parent::initController($request,
$response, $logger);

    //-----
-----
    // Preload any models, libraries,
etc, here.
    //-----
-----

    $this->session =
\Config\Services::session();
    session();
}
```

Pada method `initController()` digunakan `\Config\Services::session()`, yang memang dibutuhkan saat menerapkan validasi pada form. Session tersebut dimanfaatkan untuk membuat *flash message* untuk mengirimkan pesan error yang dihasilkan saat validasi dilakukan.

Setelah membuat halaman form input, kita akan menerapkan validasi menggunakan beberapa teknik penulisan rule validasi dengan controller yang berbeda.

Rule validasi dapat didefinisikan menggunakan `$validation->setRule()` yang untuk setiap inputan. Berikut ini merupakan penerapannya.

Pada atribut `action=""` form diatas, ubah menjadi `action="validasi/save"`. Kemudian buat controller dengan nama `Validasi.php`, dan tambahkan code berikut pada controller tersebut.

```
<?php

namespace App\Controllers;

class Validasi extends BaseController
{
    public function index()
    {
        return view('daftar', [
            'validation' => \Config\Services::
                validation()
        ]);
    }

    public function save()
    {
        $validation = \Config\Services::
            validation();
        $validation->setRule('nama', 'Nama',
            'required');
        $validation->setRule('email', 'Email',
            'required|valid_email');
        $validation->setRule('telp', 'No. Telp',
            'required|numeric|min_length[11]|
                max_length[11]');

        if($validation->run($this->request
            ->getPost())) {
            return "<h1>Success</h1>";
        }
        return redirect()->to('/validasi')
            ->withInput();
    }
}
```

```
}}
```

Rule validasi juga dapat didefinisikan untuk beberapa form sekaligus menggunakan `$validation->setRules([])` dengan argumennya berupa *array asosiatif*. Berikut merupakan contoh penerapannya.

Pada atribut `action=""` form diatas, ubah menjadi `action="validasi2/save"`. Kemudian buat controller dengan nama `Validasi2.php`, dan tambahkan code berikut pada controller tersebut.

```
<?php

namespace App\Controllers;

class Validasi2 extends BaseController
{
    public function index()
    {
        return view('daftar', [
            'validation' => \Config\Services::
                validation()
        ]);
    }

    public function save()
    {
        $validation = \Config\Services::
            validation();
        $validation->setRules([
            'nama' => 'required',
            'email' => 'required|valid_email',
            'email' => 'required|numeric|
                min_length[11]|max_length[11]'
        ]);
        if($validation->run($this->request
            ->getPost())) {
            return "<h1>Success</h1>";
        }
        return redirect()->to('/validasi2')
```

```

        ->withInput();
    }
}

```

Validasi juga dapat dilakukan menggunakan `$this->validate([])` yang sudah tertanam pada setiap controller. Method tersebut mirip dengan `$validation->setRules([])` yang dapat mendefinisikan rule untuk banyak inputan.

Pada atribut `action=""` form diatas, ubah menjadi `action="validasi3/save"`. Kemudian buat controller dengan nama `Validasi3.php`, dan tambahkan code berikut pada controller tersebut.

```

<?php

namespace App\Controllers;

class Daftar extends BaseController
{
    public function index()
    {
        return view('daftar', [
            'validation' => \Config\Services::
                validation()
        ]);
    }

    public function save()
    {
        if(!$this->validate([
            'nama' => 'required|alpha',
            'email' => 'required|valid_email',
            'telp' => 'required|numeric|
                min_length[11]|max_length[11]',
        ])) {
            return redirect()->to('/validasi3')
                ->withInput();
        }

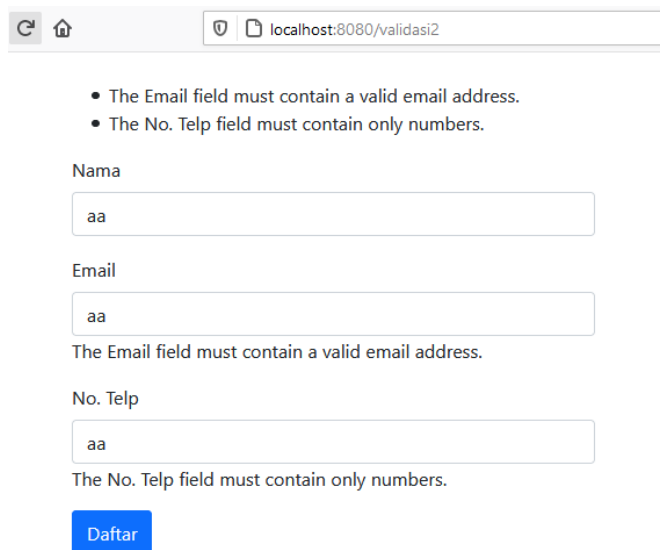
        return "<h1>Success</h1>";
    }
}

```

```
}  
}
```

Pada controller diatas, pada saat view daftar dipanggil, kita juga mempassing `\Config\Services::validation()` yang digunakan untuk memanggil pesan error yang dihasilkan dari validasi. Pada method `save()` terjadi skenario validasi, validasi dilakukan menggunakan method `$this->validate([])` dengan mengirimkan rule array assosiatif sebagai argumen validasi.

Kemudian akses setiap Controller tersebut, isikan nilai yang tidak sesuai pada inputan dan tekan tombol Daftar, maka akan tampil seperti berikut.



The screenshot shows a web browser window with the address bar displaying `localhost:8080/validasi2`. The page content includes two bullet points indicating validation errors: "The Email field must contain a valid email address." and "The No. Telp field must contain only numbers." Below these, there are three input fields: "Nama" with the value "aa", "Email" with the value "aa", and "No. Telp" with the value "aa". Each input field is followed by its respective error message. At the bottom, there is a blue button labeled "Daftar".

- The Email field must contain a valid email address.
- The No. Telp field must contain only numbers.

Nama

Email

The Email field must contain a valid email address.

No. Telp

The No. Telp field must contain only numbers.

Daftar

Gambar 8.1 Validasi Form

8.3. MENGENAL VALIDASI UNTUK UPLOAD FILE

Validasi file memungkinkan aplikasi untuk memverifikasi bahwa file yang diunggah sesuai dengan kebutuhan. Penggunaan validasi file hampir sama dengan validasi form sebelumnya, hanya saja berbeda dalam rule validasi. Rule validasi form tidak berlaku jika digunakan untuk validasi file upload. Berikut ini merupakan rule-rule validasi file upload yang disediakan CodeIgniter.

Rule	Keterangan
uploaded[]	Mengharuskan inputan harus berisi file valid
max_size[]	Mengharuskan ukuran file yang diupload kurang dari yang didefinisikan diparameter.
max_dims[]	Mengharuskan dimensi gambar kurang dari yang didefinisikan diparameter
mime_in[]	Mengharuskan tipe mime file seperti yang diparameter
ext_in[]	Membolehkan hanya file dengan ekstensi yang ditentukan diparamter
is_image[]	Mengizinkan hanya file gambar yang diunggah

8.4. MEMBUAT DAN MEMVALIDASI FORM UPLOAD FILE

Berikut merupakan contoh penggunaan upload file untuk upload foto. Buat view baru untuk tampilkan form uploadnya, simpan file tersebut dengan nama `upload_file.php`, dan tambahkan code berikut.

```
<!doctype html>
<html lang="en">
<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-
width, initial-scale=1">
<!-- Bootstrap CSS -->
```

```

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.
0.0-beta3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+
rP48ckxlpbzKgwra6" crossorigin="anonymous">
<title>Pendaftaran</title>
</head>
<body>
<div class="container">
  <div class="row">
    <div class="col-5 mt-4">
      <form action="/upload/save" method="post"
        enctype="multipart/form-data">
        <?= csrf_field(); ?>
        <div class="mb-3">
          <label class="form-label">
            Masukkan Foto :</label>
          <input class="form-control"
            type="file" name="foto">
          <?= $validation->getError('foto') ?>
        </div>
          <button type="submit" class="btn btn
            -primary">Unggah</button>
        </form>
      </div>
    </div>
  </div>
</body>
</html>

```

Kemudian buat controller Upload.php untuk memanggil view upload_file.php dan menyimpan file yang diunggah ke server.

```

<?php

namespace App\Controllers;

class Upload extends BaseController
{
    public function index()

```

```

{
    return view('upload_file', [
        'validation' =>\Config\Services::validation()
    ]);
}
public function save()
{
    if(!$this->validate([
        'foto' => [
            'rules' =>
                'uploaded[foto]|
                ext_in[foto,png,jpg,jpeg,gif]|
                max_size[foto,2048]',
            'errors' => [
                'ext_in' => 'The Image files
                    only allowed (.jpg .png .gif)'
            ]
        ],
    ])) {
        return redirect()->to('/upload')
            ->withInput();
    }
    $foto = $this->request->getFile('foto');
    if ($foto->getError() === 0) {
        $foto->move('uploads');
        echo "Nama file : ". $foto->getName();
        echo " Ukuran : ". $foto
            ->getSizeByUnit('kb');
        return "<h1>Berhasil Upload</h1>";
    }
}
}

```

Dalam controller diatas, kita menggunakan method `->getFile('foto')` untuk menangkap file yang dikirim. Untuk menyimpan file tersebut kemudian digunakan method `->move('folder')`, dengan argumennya yaitu folder tujuan tempat file tersebut akan disimpan. Dengan method tersebut

secara otomatis CodeIgniter akan membuat folder jika folder yang dituju tidak ditemukan.

Dalam controller diatas diberikan folder tujuan *"uploads"*, dihalaman public secara otomatis akan dibuatkan folder tersebut, dan file upload tersimpan didalamnya



Gambar 8.2 Upload Foto

BAB 9

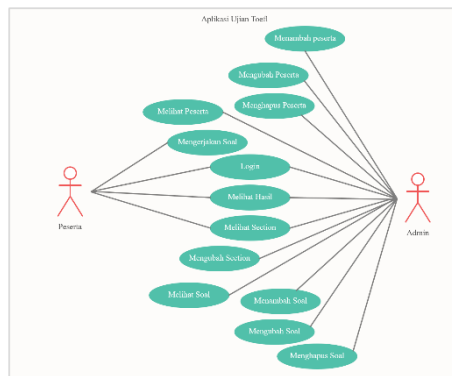
IMPLEMENTASI CODEIGNITER 4

(MEMBUAT APLIKASI UJIAN TOEFL SEDERHANA)

Pada bab sebelumnya, kita sudah mempelajari tentang dasar-dasar dari framework CodeIgniter 4, dan pada bab ini kita akan menerapkan framework ini untuk membuat project aplikasi ujian toefl sederhana, dimana dalam aplikasi tersebut merupakan *Computer Based Test* yang dapat mengerjakan soal-soal toefl, seperti *reading*, *listening*, dan *structure/written*.

9.1. RANCANGAN APLIKASI

Dalam aplikasi yang akan kita buat, nantinya akan memiliki 2 user, yaitu admin, dan peserta tes. Admin memiliki peran untuk melakukan pengelolaan data, seperti mengelola data peserta tes, dan mengelola soal. Sedangkan peserta dapat login ke aplikasi untuk mengerjakan soal-soal yang terdiri dari beberapa section dengan batas waktu yang telah ditentukan. Setelah menyelesaikan semua section, peserta juga dapat melihat hasil yang diperoleh, yaitu jawaban yang benar. Berikut use case diagram yang menggambarkan peran setiap user tersebut.



Gambar 10.1 Use Case Aplikasi Ujian Toefl

9.2. MEMBUAT DATABASE APLIKASI

Untuk membuat database kita dapat membuka tools phpMyAdmin, kemudian pilih *new* untuk membuat database baru, dan berikan nama database “ujiantoefl_db”, dan tekan tombol create. Kemudian pada menu SQL, masukkan perintah SQL berikut untuk membuat tabel yang akan digunakan nantinya.

```
CREATE TABLE `t_bagian` (  
  `id_bagian` int(11) NOT NULL AUTO_INCREMENT,  
  `nm_section` varchar(50) NOT NULL,  
  `limit_time` int(11) NOT NULL,  
  `keterangan` varchar(100) NOT NULL,  
  `direction` text NOT NULL,  
  PRIMARY KEY (`id_bagian`)  
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT  
CHARSET=utf8mb4;
```

```
CREATE TABLE `t_jawab` (  
  `id_tes` int(11) NOT NULL,  
  `id_soal` int(11) NOT NULL,  
  `jawaban` varchar(4) NOT NULL,  
  `eval` enum('benar','salah') NOT NULL,  
  KEY `id_tes` (`id_tes`),  
  KEY `id_soal` (`id_soal`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE `t_soal` (  
  `id_soal` int(11) NOT NULL AUTO_INCREMENT,  
  `id_bagian` int(11) NOT NULL,  
  `id_topik` int(11) DEFAULT NULL,  
  `no_soal` int(11) NOT NULL,  
  `soal` text NOT NULL,  
  `opsi_a` text NOT NULL,  
  `opsi_b` text NOT NULL,  
  `opsi_c` text NOT NULL,  
  `opsi_d` text NOT NULL,  
  `jawaban` varchar(1) NOT NULL,  
  PRIMARY KEY (`id_soal`),  
  KEY `id_bagian` (`id_bagian`),  
  KEY `id_topik` (`id_topik`)
```

```

) ENGINE=InnoDB AUTO_INCREMENT=76 DEFAULT
CHARSET=utf8mb4;

CREATE TABLE `t_tes` (
  `id_tes` int(11) NOT NULL AUTO_INCREMENT,
  `id_user` int(11) NOT NULL,
  `id_bagian` int(11) NOT NULL,
  `jml_benar` varchar(10) NOT NULL,
  `status` enum('ready','start','finish') NOT
NULL,
  `tgl_start` date DEFAULT NULL,
  `tgl_finish` date DEFAULT NULL,
  PRIMARY KEY (`id_tes`),
  KEY `id_user` (`id_user`),
  KEY `id_bagian` (`id_bagian`)
) ENGINE=InnoDB AUTO_INCREMENT=16 DEFAULT
CHARSET=utf8mb4;

CREATE TABLE `t_topik` (
  `id_topik` int(11) NOT NULL AUTO_INCREMENT,
  `nm_topik` varchar(100) NOT NULL,
  `id_bagian` int(11) NOT NULL,
  `content` text NOT NULL,
  PRIMARY KEY (`id_topik`),
  KEY `id_bagian` (`id_bagian`)
) ENGINE=InnoDB AUTO_INCREMENT=30 DEFAULT
CHARSET=utf8mb4;

CREATE TABLE `t_user` (
  `id_user` int(11) NOT NULL AUTO_INCREMENT,
  `nama` varchar(50) NOT NULL,
  `username` varchar(50) NOT NULL,
  `password` varchar(50) NOT NULL,
  `role` enum('admin','peserta') NOT NULL,
  PRIMARY KEY (`id_user`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT
CHARSET=utf8mb4;

CREATE VIEW `summary` AS select `a`.`id_user`
AS `id_user`,`a`.`tgl_start` AS
`tgl_start`,(select `t_tes`.`jml_benar` from
`t_tes` where `t_tes`.`id_bagian` = 1 and

```

```

`t_tes`.`id_user` = `a`.`id_user`) AS
`r`,(select `t_tes`.`jml_benar` from `t_tes`
where `t_tes`.`id_bagian` = 2 and
`t_tes`.`id_user` = `a`.`id_user`) AS
`l`,(select `t_tes`.`jml_benar` from `t_tes`
where `t_tes`.`id_bagian` = 3 and
`t_tes`.`id_user` = `a`.`id_user`) AS `s` from
`t_tes` `a` group by `a`.`id_user`;

```

Setelah membuat database, kemudian kita perlu menginstalasi framework codeigniter 4 baik secara manual maupun menggunakan composer, dan melakukan konfigurasi pada file environment untuk menghubungkan pada database “ujiantoefl_db”.

9.3. MEMBUAT HALAMAN UJIAN

9.3.1. Sistem Layouting

Sistem layouting ini merupakan sebuah view yang nantinya akan diterapkan pada berbagai view turunannya. View ini menjadi view dasar yang akan memanggil library seperti, bootstrap, fonts google, dan font awesome.

Pada bagian folder /app/Views/ buat folder baru, berikan nama template, dan buat file view baru dengan nama layout.php, dan tambahkan code berikut.

```

<!doctype html>
<html lang="en" class="h-100">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1">

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivrivr.net/npm
/bootstrap@5.0.0-
beta3/dist/css/bootstrap.min.css"
rel="stylesheet">

```

```

<link href="https://fonts.googleapis.com/css2?
  family=Poppins:wght@100;200;300&display=swap"
  rel="stylesheet">
<link rel="stylesheet"
  href="http://cdnjs.cloudflare.com/ajax/libs
  /font-awesome/4.4.0/css/font-awesome.min.css">
<style>
  * {
    font-family: Poppins;
  }
  .menu i {
    font-size: 5em;
  }
  .navbar {
    position: static;
    min-height: 10vw;
    z-index: 1;
  }
  .main-container {
    position: relative;
    margin-top: -4.7vw;
    z-index: 2;
  }
  .main-container .col, .main-container .col-10
{
  background: #fff;
}
</style>
</head>
<body class="d-flex flex-column h-100">

<!-- Begin page content -->
<main class="flex-shrink-0 mb-5">
  <?= $this->renderSection('content') ?>
</main>

<footer class=" mt-auto py-3"
  style="background:#33485d">
  <div class="container">
    <span class="text-light">
      Copyright &copy; <?= date('Y') ?></span>
    </div>
  </footer>

```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap
@5.0.0-beta3/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>

```

Kemudian buat view baru dengan nama `nav.php`, untuk membuat tampilan navbar yang akan di include pada semua file view.

```

<nav class="navbar navbar-expand-lg
navbar-light py-3" style="background:#20c997">
<div class="container mb-auto">
  <a class="navbar-brand" href="#">
    CBT TOEFL EXAM</a>
  <div class="dropdown">
    <a class="btn dropdown-toggle"
href="#" role="button" id="dropdownMenuLink"
data-bs-toggle="dropdown">
      
      <?php
        $session = \Config\Services::session();
        echo @$session->nama;
      ?>
    </a>
    <ul class="dropdown-menu"
aria-labelledby="dropdownMenuLink">
      <li><a class="dropdown-item"
href="/logout">Logout</a></li>
    </ul>
  </div>
</div>
</nav>

```

9.3.2. Halaman Login

Halaman login merupakan halaman yang nantinya digunakan untuk mengauthentikasi peserta maupun admin yang akan mengakses

sistem ujian. Pada folder Views buat file baru, berikan nama login.php, dan tambahkan code berikut pada view tesebut.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<div class="container">
  <div class="row justify-content-center mt-5">
    <div class="col-5 shadow p-5">

      <?php if ($session->has('error')): ?>
        <div class="alert alert-danger">
          <?= $session->getFlashdata('error') ?>
        </div>
      <?php endif; ?>

      <h4>Welcome CBT Toefl Exam</h4>
      <small>please log in with the username
        and password that you get</small>
      <form action="/auth" method="post">

        <div class="form-floating my-3">
          <input type="text" name="username"
            class="form-control <?= ($val
              ->hasError('username')) ?
              'is-invalid' : '' ?>" id="usr"
            placeholder="Username">
          <label for="usr">Username</label>
          <div class="invalid-feedback">
            <?= $val->getError('username') ?>
          </div>
        </div>

        <div class="form-floating">
          <input type="password" name="password"
            class="form-control <?= ($val
              ->hasError('password'))
              ? 'is-invalid' : '' ?>" id="pwd"
            placeholder="Password">
          <label for="pwd">Password</label>
          <div class="invalid-feedback">
```

```

        <?= $val->getError('password') ?>
    </div>
</div>

    <div class="d-grid gap-2 mt-3">
        <button type="submit"
            class="btn btn-info d-block">Login
        </button>
    </div>
</form>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian kita buat Model `UserModel.php` untuk membaca dan mengelola data dari tabel user, dan tambahkan code berikut pada model tersebut.

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class UserModel extends Model
{
    protected $table          = 't_user';
    protected $primaryKey     = 'id_user';

    protected $useAutoIncrement = true;

    protected $allowedFields = ['id_user', 'nama',
                                'username', 'password', 'role'];
    protected $returnType     = 'array';
    protected $useSoftDeletes = false;
}

```

Kemudian buat controller dengan nama `Login.php`, untuk memanggil model `UserModel.php` dan view login, serta controller tersebut untuk melakukan autentikasi login dan logout. Tambahkan code berikut didalam controller tersebut.

```
<?php

namespace App\Controllers;
use \App\Models\UserModel;

class Login extends BaseController
{
    public function __construct()
    {
        $this->user = new UserModel();
    }

    public function index()
    {
        return view('login', [
            'session' => $this->session,
            'val' => \Config\Services::validation()
        ]);
    }

    public function auth()
    {
        if (!$this->validate([
            'username' => 'required',
            'password' => 'required'
        ])) {
            return redirect()->to('/auth')->withInput();
        }

        $user = $this->request->getPost('username');
        $pass = $this->request->getPost('password');
        $auth = $this->user
            ->where('username', $user)->first();
        if ($auth):
            if ($auth['password'] == $pass) {
                $this->session->set([
```

```

        'id' => $auth['id_user'],
        'nama' => $auth['nama'],
        'role' => $auth['role']
    ]);
    if ($auth['role'] == 'admin') {
        return redirect()->to('/peserta');
    }
    else if ($auth['role'] == 'peserta') {
        return redirect()->to('/home');
    }
}
else {
    $this->session->setFlashdata('error',
        'your username was not found !');
    return redirect()
        ->to('/auth')->withInput();
}
else:
    $this->session->setFlashdata('error',
        'your username was not found !');
    return redirect()->to('/auth')->withInput();
endif;
}
public function logout()
{
    $this->session->destroy();
    return redirect()->to('/auth');
}
}

```

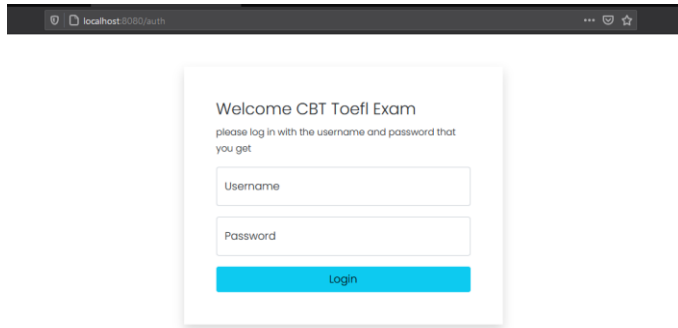
Kemudian pada file routing /app/Config/Routes.php tambahkan code berikut untuk membuat rute baru untuk halaman login.

```

$routes->get('/', 'Login::index');
//routing login
$routes->get('/auth', 'Login::index');
$routes->post('/auth', 'Login::auth');
$routes->get('/logout', 'Login::logout');

```

Kemudian kita dapat mengakses link <http://localhost:8080/auth> untuk menampilkan halaman login.



Gambar 9.1 Halaman Login

9.3.3. Halaman Home

Halaman home merupakan halaman utama yang ditampilkan saat proses autentikasi login berhasil. Halaman tersebut akan berisi beberapa menu untuk menavigasi pada halaman tertentu.

Pada folder `/app/Veiws/` buat file view baru dengan nama `home.php`, dan tambahkan code berikut untuk tampilan viewnya.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
  <div class="row justify-content-center">
    <div class="col-10 px-5 py-4 text-center
      rounded shadow menu">
      <a class="btn m-4 p-3 shadow-sm border-top"
        href="/section">
        <i class="fa fa-puzzle-piece"></i>
        <span class="badge bg-primary mt-2
          d-block">Section</span>
```

```

</a>
<a class="btn m-4 p-3 shadow-sm border-top"
    href="/score">
    <i class="fa fa fa-star"></i>
    <span class="badge bg-primary mt-2
        d-block">Score</span>
</a>
<br>
<a class="btn m-4 p-3 shadow-sm border-top"
    href="#">
    <i class="fa fa-question"></i>
    <span class="badge bg-primary mt-2
        d-block">Hint</span>
</a>
<a class="btn m-4 p-3 shadow-sm border-top"
    href="/logout">
    <i class="fa fa-sign-out"></i>
    <span class="badge bg-primary mt-2
        d-block">Logout</span>
</a>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian untuk menampilkan view home, buat controller baru dengan nama `Section.php`. Controller tersebut nantinya juga akan ditambahkan method-method lain untuk menampilkan view lainnya. Tambahkan code berikut pada controller `Section.php` untuk menampilkan view home dari method `index()` .

```

<?php

namespace App\Controllers;

class Section extends BaseController
{

```

```

public function __construct()
{
    $session = \Config\Services::session();
    if (!$session->has('id')) {
        throw
        \CodeIgniter\Exceptions\PageNotFoundException::
        forPageNotFound();
        die;
    }
}

public function index()
{
    return view('home');
}

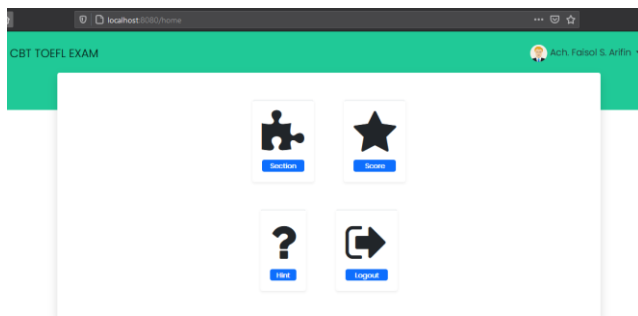
}

```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman home.

```
$routes->get('/home', 'Section::index');
```

Kemudian kita dapat mengakses link <http://localhost:8080/home> untuk menampilkan halaman home.



Gambar 9.2 Halaman Home

9.3.4. Halaman Section

Halaman section merupakan halaman yang akan menampilkan list section yang akan diujikan dalam ujian toefl tersebut, yaitu terdapat *reading*, *listening*, dan *structure/written* dan dari setiap section tersebut terdapat tombol start untuk memulai.

Pada folder `/app/Veiws/` buat file view baru dengan nama `list_section.php`, dan tambahkan code berikut sebagai tampilan viewnya.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
  <div class="row justify-content-center">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col-1">
          <a href="/home"><span class="badge
            bg-primary"> < back </span></a>
        </div>
        <div class="col-11 text-center">
          <h4>TOEFL SECTION</h4>
        </div>
      </div>
    </div>
    <div class="row">
      <div class="col">
        <?php
        foreach($data as $row):
          ?>

          <div class="shadow-sm p-3 mb-3
            bg-body rounded border">
            <div class="row">
              <div class="col-6">
                <p><?= $row['nm_section'] ?></p>
                <small><?= $row['keterangan'] ?>
```

```

</small>
</div>
<div class="col-4">
  <p><?= $row['limit_time'] ?> Minute
  </p>
  <small><?= $row['jml_soal'] ?>
    Question</small>
</div>
<div class="col">
  <?php
  if ($row['status'] == 'finish') :
    echo "<button class='btn btn-success
      float-end mt-2'>
        Complete</button>";
    elseif ($row['status'] == 'start') :
      ?>

    <form action="/continue"
      method="post">
      <input type="hidden" name="idtes"
        value="<?= $row['id_tes'] ?>">
      <input type="hidden"
        name="idbagian"
        value="<?= $row['id_bagian'] ?>">
      <button type="submit"
        class="btn btn-warning
        float-end mt-2">Continue</button>
    </form>
    <?php
  else:
    ?>

    <form action="/direction"
      method="post">
      <input type="hidden"
        name="idtes"
        value="<?= $row['id_tes'] ?>">
      <input type="hidden"
        name="idbagian"
        value="<?= $row['id_bagian'] ?>">
      <button type="submit"
        class="btn btn-danger
        float-end mt-2">Start</button>

```

```

        </form>

        <?php
        endif;
        ?>
    </div>
</div>
</div>
<?php
endforeach;
?>
</div>
</div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian kita buat Model `TesModel.php` untuk merepresentasikan data dari tabel user, dan tambahkan code berikut untuk model tersebut.

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class TesModel extends Model
{
    protected $table      = 't_tes';
    protected $primaryKey = 'id_tes';

    protected $useAutoIncrement = true;

    protected $returnType     = 'array';
    protected $useSoftDeletes = false;
    protected $allowedFields = ['id_tes',
    'id_user', 'id_bagian', 'jml_benar', 'status',
    'tgl_start', 'tgl_finish'];
}

```

```

public function getSection($user)
{
    $this->join('t_bagian', 't_bagian.id_bagian
                = t_tes.id_bagian', 'LEFT');
    $this->join('t_soal', 't_bagian.id_bagian
                = t_soal.id_bagian', 'LEFT');
    $this->select('t_tes.*');
    $this->select('t_bagian.*');
    $this->select('count(t_soal.id_bagian
                    jml_soal');
    return $this->where(['t_tes.id_user'
                        => $user])->groupBy('t_bagian.id_bagian');
}

public function getSectionById($tes)
{
    $this->join('t_bagian', 't_bagian.id_bagian
                = t_tes.id_bagian', 'LEFT');
    $this->select('t_bagian.*');
    return $this->where(['t_tes.id_tes'
                        => $tes])->first();
}
}

```

Kemudian pada controller `Section.php` lakukan modifikasi dengan menambahkan pemanggilan namespace `TesModel`, membuat instansiasi class `TesModel`, dan menambahkan beberapa method. Berikut code yang harus ditambahkan pada controller `section`.

```

use \App\Models\TesModel; //panggil TesModel

public function __construct()
{
    ..
    $this->tes = new TesModel(); //instans class
}

protected function setSessionSection()
{

```

```

$idtes = $this->request->getPost('idtes');
if ($idtes === null) return redirect()
    ->to('/section');

$this->session->set('idtes', $idtes);
$this->session->set('idsection', $this
    ->request->getPost('idbagian'));
}

public function bagian()
{
    return view('list_section', [
        'data' => $this->tes->getSection($this
            ->session->id)->findAll()
    ]);
}

public function continue()
{
    $this->setSessionSection();
    return redirect()->to('/exam');
}

```

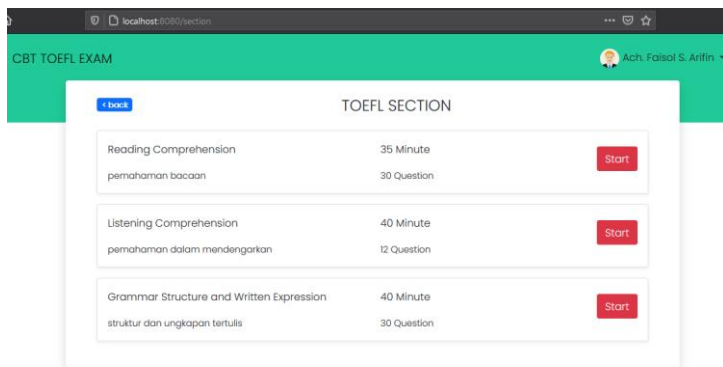
Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman section.

```

$routes->get('/section', 'Section::bagian');
$routes->post('/continue',
    'Section::continue');

```

Kemudian kita dapat mengakses link <http://localhost:8080/home> untuk menampilkan halaman home.



Gambar 9.3 Halaman Section

9.3.5. Halaman Direction

Halaman direction merupakan halaman yang ditampilkan setelah menekan tombol start dari salah satu section di halaman section. Halaman tersebut akan berisi petunjuk dalam pengerjaan soal toefl dari masing-masing section dan tombol start exam untuk memulai ujian.

Pada folder `/app/Veiws/` buat file view baru dengan nama `direction.php`, dan tambahkan code berikut sebagai tampilan viewnya.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
  <div class="row">
    <div class="col px-5 py-4 shadow rounded">
      <div class="row mb-5 border-bottom pb-2">
        <div class="col-1">
          <a href="/section"><span class="badge
            bg-primary"> < back </span></a>
        </div>
      <div class="col-11">
```

```

        <h5 class="text-center text-uppercase">
            <?= $data['nm_section'] ?></h5>
        </div>
    </div>
    <div class="row mt-5
        justify-content-center">
        <div class="col-8">
            <?= $data['direction'] ?>
            <a href="/start"
                class="btn btn-success">Start Exam</a>
        </div>
    </div>
</div>
</div>
</div>

<script>localStorage.removeItem("countdown")
</script>

<?= $this->endSection() ?>

```

Kemudian pada controller `Section.php`, lakukan modifikasi dengan menambahkan method `direction()` dan `start()` untuk memanggil view `direction` dan menstart ujian. Berikut code dari masing-masing method yang harus ditambahkan.

```

public function direction()
{
    $this->setSessionSection();
    return view('direction', [
        'data' => $this->tes->getSectionById($this
            ->session->idtes)
    ]);
}

public function start()
{
    if ($this->tes->update($this->session->idtes, [

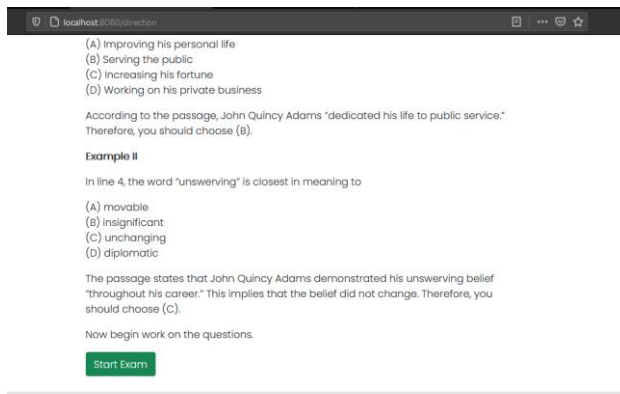
```

```
'tgl_start' => date('Y-m-d H:i:s'),
'status' => 'start',
])) {
    return redirect()->to('/exam');
}
}
```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman direction.

```
$routes->post('/direction',
              'Section::direction');
$routes->get('/start', 'Section::start');
```

Kemudian kita dapat mengakses link <http://localhost:8080/direction> untuk menampilkan halaman home.



Gambar 9.4 Halaman Direction

9.3.6. Halaman Exam

Halaman exam merupakan halaman yang berisi soal *multiple choice* dari masing-masing section dan memiliki timer yang secara otomatis akan mengakhiri ujian setelah waktu habis.

Berikut ini merupakan code untuk membuat timer/countdown yang nantinya akan dipanggil pada setiap sheet section. Simpan code berikut dengan nama `countdown.js` pada folder `/public/assets/js`.

```
function padLeadingZeros(num) {
  var s = num+"";
  while (s.length < 2) s = "0" + s;
  return s;
}

var
countDownDate=localStorage.getItem("countdown")
;
var x = setInterval(function() {

  var now = new Date().getTime();
  var distance = countDownDate - now;

  var hours = Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
  var minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
  var seconds = Math.floor((distance % (1000 * 60)) / 1000);

  document.getElementById("countdown").innerHTML
=   padLeadingZeros(hours) + ":" +
    padLeadingZeros(minutes) + ":" +
    padLeadingZeros(seconds);

  if (distance < 0) {
    clearInterval(x);
    localStorage.removeItem('countdown')
    alert('Time Out');
    window.location.reload(true);
    window.location.assign("http://localhost:8080
                              /finish");
  }
}, 1000);
```

Kemudian buat view untuk menampilkan soal ujian dari section *reading comprehension*. Buat view baru dengan nama `reading_sheet.php` dan tambahkan code berikut.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
    <div class="row">
        <div class="col px-5 py-3 shadow rounded">
            <div class="row justify-content-end
                mb-4 border-bottom">
                <div class="col-3 border-start
                    border-end">
                    <h4 class="text-center"
                        id="countdown">00:00:00</h4>
                </div>
            </div>
            <div class="row">
                <div class="col-6 border-end">
                    <p><?= $no. ". ". $data['soal'] ?></p>

                <form action="/save/<?= $no ?>" method="post">
                    <div class="row">
                        <div class="col-sm-10 ms-4 border">
                            <input type="hidden" name="id_soal"
                                value="<?= $data['id_soal'] ?>">

                            <div class="form-check my-3">
                                <input class="form-check-input"
                                    type="radio" name="jawaban"
                                    id="gridRadios1" value="A"
                                    <?= $jawab=='A' ? 'checked' : '' ?>>
                                <label class="form-check-label"
                                    for="gridRadios1">
                                    <?= $data['opsi_a'] ?>
                                </label>
                            </div>
                        </div>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>
```

```

<div class="form-check mb-3">
  <input class="form-check-input"
    type="radio" name="jawaban"
    id="gridRadios2" value="B"
    <?= $jawab=='B' ? 'checked' : '' ?>>
  <label class="form-check-label"
    for="gridRadios2">
    <?= $data['opsi_b'] ?>
  </label>
</div>

<div class="form-check mb-3">
  <input class="form-check-input"
    type="radio" name="jawaban"
    id="gridRadios3" value="C"
    <?= $jawab=='C' ? 'checked' : '' ?>>
  <label class="form-check-label"
    for="gridRadios3">
    <?= $data['opsi_c'] ?>
  </label>
</div>

<div class="form-check mb-3">
  <input class="form-check-input"
    type="radio" name="jawaban"
    id="gridRadios4" value="D"
    <?= $jawab=='D' ? 'checked' : '' ?>>
  <label class="form-check-label"
    for="gridRadios4">
    <?= $data['opsi_d'] ?>
  </label>
</div>
</div>
</div>

<div class="row mt-5">
  <div class="col text-center">
    <?= $no > 1 ? '<button type="submit"
      class="btn btn-primary btn-sm"
      name="back" value="back">
      < Back</button>' : '' ; ?>
    <button type="submit"
      class="btn btn-primary btn-sm" name="next"

```

```

        value="next">Next ></button>
    </div>
</div>
</form>

    </div>
    <div class="col-6">
        <?= $data['content'] ?>
    </div>
</div>
</div>
</div>
</div>
</div>

<script>
    var minutes = <?= $limit ?>;

    if (localStorage.getItem("countdown") ===
        null)
    {
        var countDown = new Date(Date.now()
            + minutes*60000).getTime();
        localStorage.setItem("countdown",
            countDown);
    }
</script>
<script src="<?= base_url('assets/js
    /countdown.js') ?>"></script>

<?= $this->endSection() ?>

```

Kemudian buat view untuk menampilkan soal ujian dari section *listening comprehension*. Buat view baru dengan nama `listening_sheet.php` dan tambahkan code berikut.

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">

```

```

<div class="row">
  <div class="col px-5 py-3 shadow rounded">
    <div class="row justify-content-end mb-3">
      <div class="col-3">
        <h4 class="text-center"
          id="countdown">00:00:00</h4>
      </div>
    </div>
  </div>
  <div class="row">
    <div class="col-12">
      <p><strong>Question <?= $no ?></strong>
      </p>
      <audio class="w-100" controls>
        <source src="<?= base_url("listening
          /{$data['content']}") ?>"
          type="audio/mpeg">
        Your browser does not support
        the audio element.
      </audio>
      <p class="ms-2 mt-4"><?= $data['soal'] ?>
      </p>
    </div>
  </div>
</div>
<form action="/save/<?= $no ?>" method="post">
  <div class="row">
    <div class="col-sm-10 ms-4 border">
      <input type="hidden" name="id_soal"
        value="<?= $data['id_soal'] ?>">

      <div class="form-check my-3">
        <input class="form-check-input"
          type="radio" name="jawaban"
          id="gridRadios1" value="A"
          <?= $jawab=='A' ? 'checked' : '' ?>>
        <label class="form-check-label"
          for="gridRadios1">
          <?= $data['opsi_a'] ?>
        </label>
      </div>

      <div class="form-check mb-3">
        <input class="form-check-input"
          type="radio" name="jawaban"
          id="gridRadios2" value="B"

```

```

        <?= $jawab=='B' ? 'checked' : '' ?>>
        <label class="form-check-label"
        for="gridRadios2">
            <?= $data['opsi_b'] ?>
        </label>
    </div>

    <div class="form-check mb-3">
        <input class="form-check-input"
        type="radio" name="jawaban"
        id="gridRadios3" value="C"
        <?= $jawab=='C' ? 'checked' : '' ?>>
        <label class="form-check-label"
        for="gridRadios3">
            <?= $data['opsi_c'] ?>
        </label>
    </div>

    <div class="form-check mb-3">
        <input class="form-check-input"
        type="radio" name="jawaban"
        id="gridRadios4" value="D"
        <?= $jawab=='D' ? 'checked' : '' ?>>
        <label class="form-check-label"
        for="gridRadios4">
            <?= $data['opsi_d'] ?>
        </label>
    </div>
</div>
</div>

<div class="row mt-5">
    <div class="col text-center">
        <?= $no > 1 ? '<button type="submit"
        class="btn btn-primary btn-sm"
        name="back" value="back">
        < Back</button>' : '' ; ?>
        <button type="submit"
        class="btn btn-primary btn-sm" name="next"
        value="next">Next ></button>
    </div>
</div>
</form>

```

```

        </div>
    </div>
</div>
</div>
</div>
</div>
<script>
    var minutes = <?= $limit ?>;

    if (localStorage.getItem("countdown") ===
        null)
    {
        var countDown = new Date(Date.now()
            + minutes*60000).getTime();
        localStorage.setItem("countdown",
            countDown);
    }
</script>
<script src="<?= base_url('assets
    /js/countdown.js') ?>"></script>

<?= $this->endSection() ?>

```

Kemudian buat view untuk menampilkan soal ujian dari section *Grammar Structure and Written Expression*. Buat view baru dengan nama `grammar_sheet.php` dan tambahkan code berikut.

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
    <div class="row">
        <div class="col px-5 py-3 shadow rounded">
            <div class="row justify-content-end mb-4
                border-bottom">
                <div class="col-3 border-start border-end">
                    <h4 class="text-center"
                        id="countdown">00:00:00</h4>

```

```

    </div>
</div>
<div class="row">
  <div class="col-8">
    <p><?= $no. ". ". $data['soal'] ?></p>

<form action="/save/<?= $no ?>" method="post">
  <div class="row">
    <div class="col-sm-10 ms-4 border">
      <input type="hidden" name="id_soal"
        value="<?= $data['id_soal'] ?>">

      <div class="form-check my-3">
        <input class="form-check-input"
          type="radio" name="jawaban"
          id="gridRadios1" value="A"
          <?= $jawab=='A' ? 'checked' : '' ?>>
        <label class="form-check-label"
          for="gridRadios1">
          <?= $data['opsi_a'] ?>
        </label>
      </div>

      <div class="form-check mb-3">
        <input class="form-check-input"
          type="radio" name="jawaban"
          id="gridRadios2" value="B"
          <?= $jawab=='B' ? 'checked' : '' ?>>
        <label class="form-check-label"
          for="gridRadios2">
          <?= $data['opsi_b'] ?>
        </label>
      </div>

      <div class="form-check mb-3">
        <input class="form-check-input"
          type="radio" name="jawaban"
          id="gridRadios3" value="C"
          <?= $jawab=='C' ? 'checked' : '' ?>>
        <label class="form-check-label"
          for="gridRadios3">
          <?= $data['opsi_c'] ?>
        </label>

```

```

</div>

<div class="form-check mb-3">
  <input class="form-check-input"
    type="radio" name="jawaban"
    id="gridRadios4" value="D"
    <?=$jawab=='D' ? 'checked' : '' ?>
  <label class="form-check-label"
    for="gridRadios4">
    <?=$data['opsi_d'] ?>
  </label>
</div>
</div>
</div>

<div class="row mt-5">
  <div class="col text-center">
    <?=$no > 1 ? '<button type="submit"
      class="btn btn-primary btn-sm"
      name="back" value="back">
      < Back</button>' : '' ; ?>
    <button type="submit"
      class="btn btn-primary btn-sm" name="next"
      value="next">Next ></button>
  </div>
</div>
</form>

</div>
<div class="col-4">
  <?=$data['content'] ?>
</div>
</div>
</div>
</div>
<script>
  var minutes = <?=$limit ?>;

  if (localStorage.getItem("countdown") ===
    null)
  {
    var countDown = new Date(Date.now()

```

```

        + minutes*60000).getTime();
        localStorage.setItem("countdown", countdown);
    }
</script>
<script src="<?= base_url('assets
                        /js/countdown.js') ?>"></script>

<?= $this->endSection() ?>

```

Kemudian kita buat Model SoalModel.php untuk merepresentasikan data dari tabel t_soal, dan tambahkan code berikut untuk model tersebut.

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class SoalModel extends Model
{
    protected $table          = 't_soal';
    protected $primaryKey     = 'id_soal';

    protected $useAutoIncrement = true;
    protected $allowedFields = ['id_soal',
    'id_bagian', 'id_topik', 'no_soal', 'soal',
    'opsi_a', 'opsi_b', 'opsi_c', 'opsi_d',
    'jawaban'];

    protected $returnType     = 'array';
    protected $useSoftDeletes = false;

    public function getSoal()
    {
        $this->join('t_topik', 't_topik.id_topik
                    = t_soal.id_topik', 'LEFT');
        $this->select('t_soal.*');
        $this->select('t_topik.content,
                    t_topik.nm_topik');
        return $this;
    }
}

```

```
}
```

Kemudian kita buat Model `JawabModel.php` untuk merepresentasikan data dari tabel `t_jawab`, dan tambahkan code berikut untuk model tersebut.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class JawabModel extends Model
{
    protected $table      = 't_jawab';
    protected $primaryKey = 'id_tes';

    protected $useAutoIncrement = false;

    protected $returnType     = 'array';
    protected $useSoftDeletes = false;
    protected $allowedFields = ['id_tes',
    'id_soal', 'jawaban', 'eval'];
}
```

Kemudian kita buat Model `SoalModel.php` untuk merepresentasikan data dari tabel `t_bagian`, dan tambahkan code berikut untuk model tersebut.

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class BagianModel extends Model
{
    protected $table      = 't_bagian';
    protected $primaryKey = 'id_bagian';
```

```

protected $useAutoIncrement = false;

protected $returnType      = 'array';
protected $useSoftDeletes = false;
protected $allowedFields = ['id_bagian',
'nm_section', 'limit_time', 'keterangan',
'direction'];
}

```

Kemudian pada controller `Section.php` lakukan modifikasi kembali dengan syntax `use` untuk memanggil model `SoalModel`, `JawabModel`, `Tes Model`, dan `BagianModel`. dan lakukan instansiasi pada setiap model dibagian konstaktor dan tambahkan method `exam()` dan `save()` yang masing-masing untuk menampilkan soal dan menyimpan jawaban dari soal yang bersangkutan. Berikut code yang harus ditambahkan pada controller `Section` tersebut.

```

use \App\Models\SoalModel; //panggil SoalModel
use \App\Models\JawabModel; //Panggil JawabModel
use \App\Models\TesModel; //panggil TesModel
use \App\Models\BagianModel; //panggil Bagian Model

public function __construct()
{
    ...
    $this->soal = new SoalModel(); //instansiasi
    $this->jawab = new JawabModel();
    $this->tes = new TesModel();
    $this->bagian = new BagianModel();
}

public function exam($no=1)
{
    if ($this->session->idtes==null || $this->session->idsection==null) {
        return redirect()->to('/section');
    }
}

```

```

    }

    $idsection = $this->session->idsection;
    if ($this->tes->where('id_tes',
        $this->session->idtes)
->findColumn('status')[0] != 'start') return;

    if ($no <= 0) {
        throw \CodeIgniter\Exceptions
        \PageNotFoundException::forPageNotFound();
        return;
    }

    $banyak_soal = $this->soal
        ->where(['id_bagian' => $idsection])
        ->countAllResults();
    if ($no > $banyak_soal) {
        return view('finish_section', [
            'n_soal' => $banyak_soal
        ]);
    }

    $data['no'] = $no;
    $data['limit'] = $this->bagian
        ->where('id_bagian', $this->session
        ->idsection)->findColumn('limit_time')[0];
    $data['data'] = $this->soal->getSoal()
        ->where(['no_soal' => $no,
            't_soal.id_bagian' => $idsection])
        ->first();
    $data['jawab'] = $this->jawab->where([
        'id_tes' => $this->session->idtes,
        'id_soal' => $data['data']['id_soal']
    ])->findColumn('jawaban')[0] ?? '';

    if ($idsection == 1) return
    view('reading_sheet', $data);
    else if ($idsection == 2) return
    view('listening_sheet', $data);
    else if ($idsection == 3) return
    view('grammar_sheet', $data);

    }

```

```

public function save($no='')
{
    $idtes = $this->session->idtes;
    if ($idtes===null) return;

    if ($this->request->getPost('next')) ++$no;
    elseif ($this->request->getPost('back')) --
    $no;

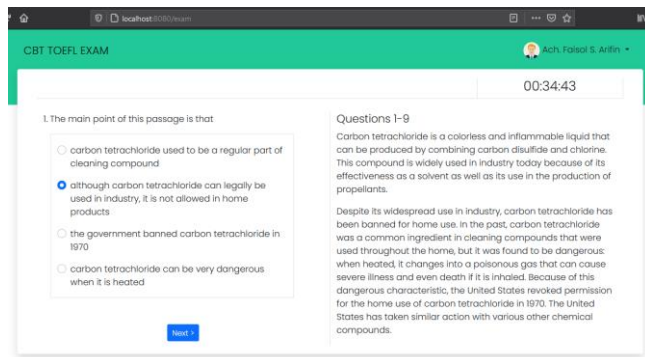
    $eval = 'salah';
    $id_soal = $this->request
        ->getPost('id_soal') ?? '';
    $jawaban = $this->request
        ->getPost('jawaban') ?? '';
    if ($this->soal->where(['id_soal' =>
    $id_soal])->findColumn('jawaban')[0] ===
    $jawaban) $eval = 'benar';

    if ($this->jawab->where(['id_tes' => $idtes,
    'id_soal' => $id_soal])->countAllResults() > 0)
    {
        $this->jawab->where([
        'id_tes' => $idtes,
        'id_soal' => $id_soal
        ])->update(
        [ 'id_tes' => $idtes ],
        [
        'id_soal' => $id_soal,
        'jawaban' => $jawaban,
        'eval' => $eval
        ]);
    }
    else {
        $this->jawab->insert([
        'id_tes' => $idtes,
        'id_soal' => $id_soal,
        'jawaban' => $jawaban,
        'eval' => $eval
        ]);
    }
    return redirect()->to('/exam/'. $no);
}

```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman exam dan rute untuk menyimpan jawaban.

```
$routes->get('/exam', 'Section::exam');
$routes->get('/exam/(:num)',
            'Section::exam/$1');
$routes->post('/save/(:num)',
            'Section::save/$1');
```



Gambar 9.5 Halaman Exam

9.3.7. Halaman Finish

Halaman finish merupakan halaman yang akan ditampilkan setelah soal terakhir dan di halaman tersebut terdapat tombol finish untuk mengakhiri ujian.

Pada folder `/app/Views/` buat file view baru dengan nama `finish_section.php`, dan tambahkan code berikut untuk tampilan viewnya.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>
```

```

<div class="container main-container">
  <div class="row justify-content-center text-center">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3 border-bottom">
        <div class="col-9">
          <h5>FINISH SECTION</h5>
        </div>
        <div class="col-3 border-start border-end">
          <h4 id="countdown">00:00:00</h4>
        </div>
      </div>
      <div class="row">
        <div class="col-9">
          <h6>End of Reading Section</h6>
          <p>You have seen all of the
            questions in the Reading Section.
          <br>You can go back and check your
            work as long as there is time left.</p>
          <p>Click on Return to check
            your work.</p>
          <p>Click on Review to see the
            review screen for this section.</p>
          <p>Click on the Proceed button
            to go on.</p>
          <p>Once you leave this Reading
            Section, you WILL NOT be able to return.</p>

          <div class="mt-5">
            <a href="/exam/<?= $n_soal ?>"
              class="btn btn-primary">< Back</a>
            <a href="/finish"
              class="btn btn-danger">Finish Exam</a>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
<script src="<?= base_url('assets
  /js/countdown.js') ?>"></script>

```

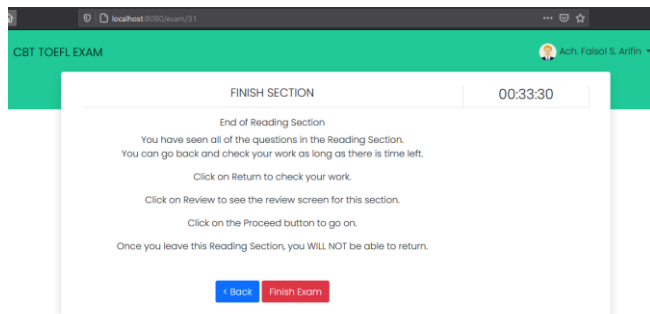
```
<?= $this->endSection() ?>
```

Kemudian pada controller `Section.php`, lakukan modifikasi dengan menambahkan method `finish()` untuk mengakhiri ujian. Berikut code untuk method yang harus ditambahkan pada controller tersebut.

```
public function finish()
{
    $this->tes->update($this->session->idtes, [
        'jml_benar' => $this->jawab
        ->where(['id_tes' => $this->session
        ->idtes, 'eval' => 'benar'])
        ->countAllResults(),
        'tgl_finish' => date('Y-m-d H:i:s'),
        'status' => 'finish',
    ]);
    //hapus session
    $this->session->remove('idtes');
    $this->session->remove('idsection');
    return redirect()->to('/section');
}
```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman finish.

```
$routes->get('/finish', 'Section::finish');
```



Gambar 9.6 Halaman Finish

9.3.8. Halaman Score

Halaman score merupakan halaman untuk menampilkan hasil ujian toefl, serta jumlah soal yang telah terjawab benar dari setiap sectionnya.

Pada folder `/app/Veiws/` buat file view baru dengan nama `score.php`, dan tambahkan code berikut untuk tampilan viewnya.

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav') ?>

<div class="container main-container">
  <div class="row justify-content-center">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col-1">
          <a href="/home"><span class="badge
            bg-primary"> < back </span></a>
        </div>
        <div class="col-11 text-center">
          <h4>TOEFL SCORE</h4>
        </div>
      </div>
    </div>
    <div class="row">
      <div class="col">
        <table class="table table-bordered">
          <tr>
            <td>Nama :
              <?php
                $session = \Config\Services::session();
                echo @$session->nama;
              ?>
            </td>
          </tr>
          <tr>
            <td>Your Toefl Score : <strong>
              <?= $score ?></strong></td>
          </tr>
        </table>
      </div>
    </div>
  </div>
</div>
```

```

</table>
<table class="table mt-4 table-bordered
            table-striped">

    <tr>
        <th>List Section</th>
        <th>Correct</th>
        <th>Number of Question</th>
    </tr>
    <?php
    $sum=0;
    foreach($data as $row):
        ?>
        <tr>
            <td><?= $row['nm_section'] ?></td>
            <td class="text-center">
                <?= $row['jml_benar'] ?>
            </td>
            <td class="text-center">
                <?= $row['jml_soal'] ?>
            </td>
        </tr>
        <?php
        endforeach;
        ?>
    </table>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

Kemudian pada controller `Section.php`, lakukan modifikasi dengan menambahkan method `score()` untuk untuk menampilkan hasil ujian tersebut. Berikut code untuk method yang harus ditambahkan dalam controller tersebut.

```
public function score()
```

```

{
    $score = 0;
    $sc = $this->tes->where(['id_user' => $this
        ->session->id])->findColumn('jml_benar');
    foreach($sc as $s)
    {
        if ($s == "") {
            $score = "not finished";
            break;
        }
        else $score += $s;
    }
    if ($score != "not finished") $score =
        ceil(($score / 3) * 10). '/300';

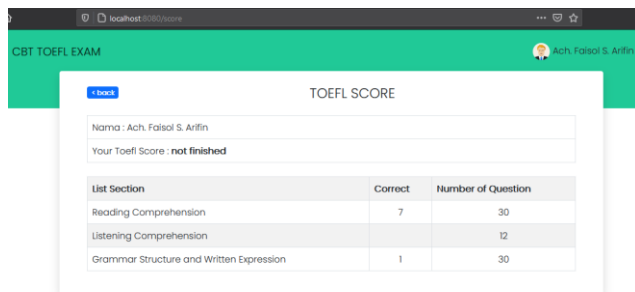
    return view('score', [
        'data' => $this->tes->getSection($this
            ->session->id)->findAll(),
        'score' => $score
    ]);
}

```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route berikut untuk membuat rute halaman score.

```
$routes->get('/score', 'Section::score');
```

Kemudian kita dapat mengakses link <http://localhost:8080/score> untuk menampilkan halaman score.



Gambar 9.7 Halaman Score

9.4. MEMBUAT HALAMAN ADMIN

9.4.1. Data Peserta

Data peserta merupakan halaman untuk menampilkan data peserta dalam bentuk tabel dan mengelola data peserta tersebut. Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada `/app/Views/admin/peserta/` dan berikan file dengan nama `data.php`

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row mt-3">
    <div class="col px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>PESERTA TOEFL</h4>
        </div>
        <div class="row mt-4">
          <div class="col-3">
            <a href="/peserta/tambah" class="btn btn-primary">Tambah Peserta</a>
          </div>
          <div class="col-9">
            <form action="/peserta">
              <div class="input-group mb-3">
                <input type="text" name="q"
                  class="form-control"
                  placeholder="Cari Peserta"
                  value="<?= @$_GET['q'] ?>">
                <button class="btn btn-success">
                  <i class="fa fa-search"></i></button>
              </div>
            </form>
          </div>
        </div>
      </div>
    <div class="row">
```

```

<div class="col">
  <table class="table table-bordered
  table-striped">
    <tr>
      <th>No. Peserta</th>
      <th>Nama</th>
      <th>Username</th>
      <th>Password</th>
      <th>Aksi</th>
    </tr>
    <?php
    foreach ($user as $row):
      ?>
      <tr>
        <td><?= $row['id_user'] ?></td>
        <td><?= $row['nama'] ?></td>
        <td><?= $row['username'] ?></td>
        <td><?= $row['password'] ?></td>
        <td class="text-center">
          <a href="/peserta
          /<?= $row['id_user'] ?>/edit"
          class="btn btn-warning btn-sm">
            <i class="fa fa-edit"></i></a>
          <form action="/peserta
          /<?= $row['id_user'] ?>"
          class="d-inline" method="post">
            <input type="hidden"
            name="_method" value="DELETE">
            <button class="btn btn-danger
            btn-sm"><i class="fa
            fa-trash"></i></button>
          </form>
        </td>
      </tr>
    <?php
    endforeach;
    ?>
  </table>
  <?php echo $pager->links('user',
                           'bs_paginate'); ?>

</div>
</div>

```

```

</div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian buat controller baru dengan nama Admin.php untuk menangani segala request yang berhubungan dengan pengelolaan data dihalaman admin. Dan tambahkan code berikut dalam controller tersebut.

```

<?php
namespace App\Controllers;
use \App\Models\UserModel;
use \App\Models\BagianModel;
use \App\Models\SoalModel;
use \App\Models\TopikModel;
use \App\Models\TesModel;

class Admin extends BaseController
{
    public function __construct()
    {
        $session = \Config\Services::session();
        if (!$session->has('id') || $session->role !=
'admin') {
            throw \CodeIgniter\Exceptions
                \PageNotFoundException::forPageNotFound();
            die;
        }
        $this->user = new UserModel();
        $this->bagian = new BagianModel();
        $this->soal = new SoalModel();
        $this->topik = new TopikModel();
        $this->tes = new TesModel();
    }

    public function index()
    {

```

```

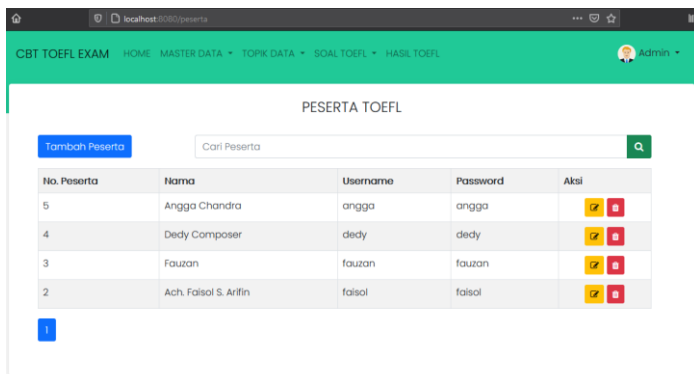
        $result = $this->user->where('role !=',
'admin');
        if ($this->request->getGet('q'))
        {
            $result = $this->user
            ->search($this->request->getGet('q') ?? '');
        }
        return view('admin/peserta/data', [
            'user' => $result->orderBy('id_user',
                'desc')->paginate(10, 'user'),
            'pager' => $this->user->pager,
        ]);
    }
}

```

Kemudian pada file routing `/app/Config/Routes.php` tambahkan route baru berikut untuk membuat rute halaman peserta.

```
$routes->get('/peserta', 'Admin::index');
```

Kemudian kita dapat mengakses link <http://localhost:8080/peserta> untuk menampilkan halaman peserta setelah login sebagai admin.



Gambar 9.8 Halaman Data Peserta

9.4.2. Hapus Peserta

Untuk membuat hapus peserta kita perlu memodifikasi controller Admin.php dengan menambahkan method berikut.

```
public function deletePeserta($id='')
{
    $this->user->delete($id);
    return redirect()->to('/peserta');
}
```

Kemudian tambahkan route baru untuk mengarahkan pada method deletePeserta()

```
$routes->delete('/peserta/(:num)',
                'Admin::deletePeserta/$1');
```

9.4.3. Tambah Peserta

Tambah peserta merupakan halaman untuk membuat peserta baru setelah menekan tombol tambah peserta di halaman data peserta.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada /app/Views/admin/peserta/ dan berikan file dengan nama tambah.php

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row justify-content-center mt-3">
        <div class="col-10 px-5 py-4 shadow rounded">
            <div class="row mb-3">
                <div class="col text-center">
                    <h4>TAMBAH PESERTA TOEFL</h4>
                </div>
            </div>
            <div class="row mt-4">
                <div class="col">
```

```

        <a href="/peserta" class="btn
            btn-primary">< </a>
    </div>
</div>
<div class="row mt-3">
    <div class="col">
        <form action="/peserta" method="post">

            <div class="mb-3">
                <label class="form-label">Nama</label>
                <input type="text" class="form-control
                    <?= ($val->hasError('nama'))
                        ? 'is-invalid' : '' ?>" name="nama">
                <div class="invalid-feedback">
                    <?= $val->getError('nama') ?>
                </div>
            </div>

            <div class="mb-3">
                <label class="
                    form-label">Username</label>
                <input type="text" class="form-control
                    <?= ($val->hasError('uname'))
                        ? 'is-invalid' : '' ?>" name="uname">
                <div class="invalid-feedback">
                    <?= $val->getError('uname') ?>
                </div>
            </div>

            <div class="mb-3">
                <label
                    class="form-label">Password</label>
                <input type="text" class="form-control
                    <?= ($val->hasError('pass'))
                        ? 'is-invalid' : '' ?>" name="pass">
                <div class="invalid-feedback">
                    <?= $val->getError('pass') ?>
                </div>
            </div>
            <button type="submit"
                class="btn btn-success">Simpan</button>
        </form>
    </div>

```

```

        </div>
    </div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian pada controller `Admin.php` lakukan modifikasi dengan menambahkan method-method berikut untuk menangani request tambah peserta.

```

public function tambahPeserta()
{
    return view('admin/peserta/tambah', [
        'val' => \Config\Services::validation()
    ]);
}

public function savePeserta()
{
    if (!$this->validate([
        'nama' => 'required',
        'uname' =>
'required|min_length[4]|is_unique[t_user.userna
me]',
        'pass' => 'required|min_length[4]'
    ])) {
        return redirect()->to('/peserta/tambah')
            ->withInput();
    }
    $this->user->insert([
        'nama' => $this->request->getPost('nama'),
        'username' => $this->request
            ->getPost('uname'),
        'password' => $this->request
            ->getPost('pass'),
        'role' => 'peserta'
    ]);
    $userID = $this->user->insertID;
    for ($i=1; $i<=3; $i++) {
        $this->tes->insert([

```

```

        'id_user' => $userID,
        'id_bagian' => $i,
        'status' => 'ready',
    ]);
}
return redirect()->to('/peserta');
}

```

Kemudian buat rute baru untuk mengarahkan pada method `tambahPeserta()` dan `savePeserta()`

```

$routes->get('/peserta/tambah',
            'Admin::tambahPeserta');
$routes->get('/peserta/{:num}/edit',
            'Admin::editPeserta/$1');

```

9.4.4. Edit Peserta

Edit peserta merupakan halaman untuk mengubah peserta setelah menekan tombol edit di halaman data peserta.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada `/app/Views/admin/peserta/` dan berikan file dengan nama `edit.php`

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row justify-content-center mt-3">
        <div class="col-10 px-5 py-4 shadow rounded">
            <div class="row mb-3">
                <div class="col text-center">
                    <h4>EDIT PESERTA TOEFL</h4>
                </div>
            <div class="row mt-4">
                <div class="col">
                    <a href="/peserta" class="btn btn-primary">< /a>

```

```

</div>
</div>
<div class="row mt-3">
  <div class="col">
    <form action="/peserta" method="post">
      <input type="hidden"
        name="_method" value="PUT">
      <input type="hidden" name="id"
        value="<?= $user['id_user'] ?>">

    <div class="mb-3">
      <label class="form-label">Nama</label>
      <input type="text" class="form-control"
        name="nama" value="<?= $user['nama'] ?>">
    </div>

    <div class="mb-3">
      <label class="form-label">Username</label>
      <input type="text" class="form-control"
        name="uname" value="<?=
        $user['username'] ?>">
    </div>

    <div class="mb-3">
      <label class="form-label">Password</label>
      <input type="text" class="form-control"
        name="pass" value="<?=
        $user['username'] ?>">
    </div>
    <button type="submit"
      class="btn btn-success">Edit</button>
    </form>
  </div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

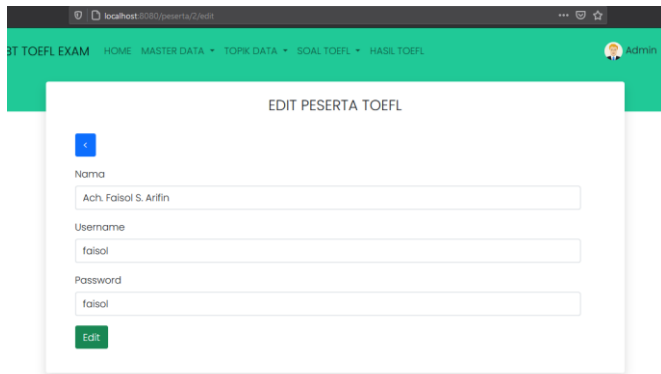
Kemudian pada controller `Admin.php` lakukan modifikasi untuk menambahkan method-method berikut dalam menangani request edit peserta.

```
public function editPeserta($id='')
{
    return view('admin/peserta/edit', [
        'user' => $this->user->find($id)
    ]);
}

public function updatePeserta()
{
    $this->user->update($this->request
        ->getPost('id'), [
        'nama' => $this->request->getPost('nama'),
        'username' => $this->request
            ->getPost('uname'),
        'password' => $this->request
            ->getPost('pass'),
        'role' => 'peserta'
    ]);
    return redirect()->to('/peserta');
}
```

Kemudian buat route baru untuk edit peserta yang mengarahkan pada method `editPeserta()` dan `updatePeserta()`

```
$routes->post('/peserta',
    'Admin::savePeserta');
$routes->put('/peserta',
    'Admin::updatePeserta');
```



Gambar 9.9 Edit Peserta Toefl

9.4.5. Data Section

Data section merupakan halaman untuk menampilkan kategori yang akan diujikan dalam bentuk tabel, seperti *reading*, *listening*, dan *structure/written* dan membuat pengelolaan data section tersebut.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada `/app/Views/admin/section/` dan berikan file dengan nama `data.php`

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row mt-3">
    <div class="col px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>LIST SECTION</h4>
        </div>
      <div class="row mt-4">
        <div class="col">
          <table class="table table-bordered
```

```

table-striped">
<tr>
  <th>#</th>
  <th>Section</th>
  <th>Batas Waktu</th>
  <th>Keterangan</th>
  <th>Direction</th>
  <th>Aksi</th>
</tr>
<?php
foreach ($section as $row):
  ?>
  <tr>
    <td><?= $row['id_bagian'] ?></td>
    <td><?= $row['nm_section'] ?></td>
    <td><?= $row['limit_time'] ?>
                                Menit</td>
    <td><?= $row['keterangan'] ?></td>
    <td><?= $row['direction'] ?></td>
    <td class="text-center">
      <a href="/bagian
        /<?= $row['id_bagian'] ?>/edit"
        class="btn btn-warning btn-sm">
        <i class="fa fa-edit"></i>
      </a>
    </td>
  </tr>
<?php
endforeach;
?>
</table>
</div>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

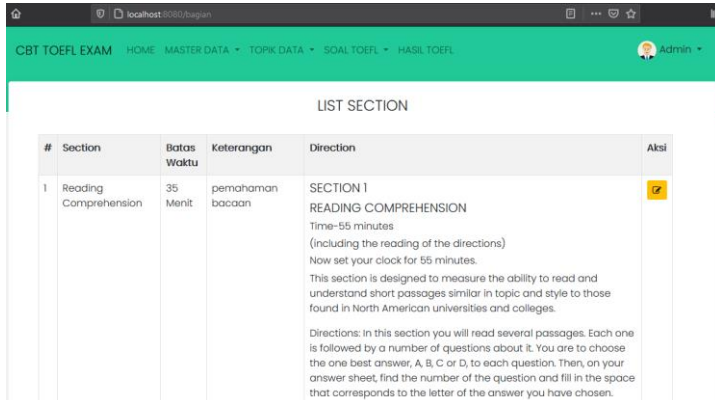
Kemudian pada controller `Admin.php` lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam menampilkan data section.

```
public function dataSection()
{
    return view('admin/section/data', [
        'section' => $this->bagian->findAll(),
    ]);
}
```

Kemudian buat route baru untuk data section yang mengarahkan pada method `dataSection()`

```
$routes->get('/bagian', 'Admin::dataSection');
```

Kemudian kita dapat mengakses link <http://localhost:8080/bagian> untuk menampilkan halaman data section setelah login sebagai admin.



#	Section	Batas Waktu	Keterangan	Direction	Aksi
1	Reading Comprehension	35 Menit	pemahaman bacaan	SECTION 1 READING COMPREHENSION Time-55 minutes (including the reading of the directions) Now set your clock for 55 minutes. This section is designed to measure the ability to read and understand short passages similar in topic and style to those found in North American universities and colleges. Directions: In this section you will read several passages. Each one is followed by a number of questions about it. You are to choose the one best answer, A, B, C or D, to each question. Then, on your answer sheet, find the number of the question and fill in the space that corresponds to the letter of the answer you have chosen.	

Gambar 9.10 Halaman Data Section

9.4.6. Edit Section

Edit peserta merupakan halaman untuk mengubah data section setelah menekan tombol edit dihalaman data section.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code berikut pada /app/Views/admin/section/ dan berikan view dengan nama edit.php

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row justify-content-center mt-3">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>EDIT SECTION</h4>
        </div>
        <div class="row mt-4">
          <div class="col">
            <a href="/peserta" class="btn btn-primary">< /a>
          </div>
        </div>
        <div class="row mt-3">
          <div class="col">
            <form action="/bagian" method="post">
              <input type="hidden"
                name="_method" value="PUT">
              <input type="hidden" name="id"
                value="<?= $b['id_bagian'] ?>">

              <div class="mb-3">
                <label class="form-label">Section</label>
                <input type="text" class="form-control"
                  name="nm_s" value="<?=
                                                              $b['nm_section'] ?>">
              </div>

              <div class="mb-3">
                <label class="form-label">
                  Batas Waktu</label>
                <input type="number">
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```

```

        class="form-control" name="time"
        value="<?= $b['limit_time'] ?>">
    </div>

    <div class="mb-3">
        <label class="form-label">
            Keterangan</label>
        <input type="text" class="form-control"
            name="ket"
            value="<?= $b['keterangan'] ?>">
    </div>

    <div class="mb-3">
        <label class="form-label">
            Direction</label>
        <textarea class="form-control" rows="3"
            name="dirc"><?= $b['direction'] ?>
        </textarea>
    </div>
    <button type="submit"
        class="btn btn-success">Edit</button>
    </form>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam mengedit data section.

```

public function editSection($id='')
{
    return view('admin/section/edit', [
        'b' => $this->bagian->find($id)
    ]);
}

```

```

public function updateSection()
{
    $this->bagian->update($this->request
        ->getPost('id'), [
        'nm_section' => $this->request
            ->getPost('nm_s'),
        'limit_time' => $this->request
            ->getPost('time'),
        'keterangan' => $this->request
            ->getPost('ket'),
        'direction' => $this->request
            ->getPost('dirc')
    ]);
    return redirect()->to('/bagian');
}

```

Kemudian buat route baru untuk edit section yang mengarahkan pada method editSection() dan updateSection()

```

$routes->get('/bagian/(:num)/edit',
    'Admin::editSection/$1');
$routes->put('/bagian',
    'Admin::updateSection');

```

9.4.7. Data Artikel Reading

Data Artikel Reading merupakan halaman untuk menampilkan artikel untuk pertanyaan-pertanyaan pada soal reading.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada folder /app/Views/admin/topik/ dan berikan file dengan nama data_article.php

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row mt-3">
        <div class="col px-5 py-4 shadow rounded">

```

```

<div class="row mb-3">
  <div class="col text-center">
    <h4>READING ARTICLE</h4>
  </div>
  <div class="row mt-4">
    <div class="col-3">
      <a href="/topik/article/tambah"
        class="btn btn-primary">Tambah Topik</a>
    </div>
    <div class="col-9">
      <form action="/topik">
        <div class="input-group mb-3">
          <input type="text" name="q"
            class="form-control"
            placeholder="Cari Topik"
            value="<?=@$_GET['q'] ?>">
          <button class="btn btn-success">
            <i class="fa fa-search"></i></button>
        </div>
      </form>
    </div>
  </div>
  <div class="row">
    <div class="col">
      <table class="table table-bordered
        table-striped">
        <tr>
          <th width="50">#</th>
          <th width="120">Nama Topik</th>
          <th>Content</th>
          <th width="120">Aksi</th>
        </tr>
        <?php
        $no=0;
        foreach ($topik as $row):
          ?>
          <tr>
            <td><?=@++$no ?></td>
            <td><?=@$row['nm_topik'] ?></td>
            <td><?=@$row['content'] ?></td>
            <td class="text-center">
              <a href="/topik/article
                /<?=@$row['id_topik'] ?>/edit"

```

```

class="btn btn-warning my-1 btn-sm">
<i class="fa fa-edit"></i></a>
    <form action="/topik/
        <?= $row['id_topik'] ?>"
            class="d-inline" method="post">
                <input type="hidden"
                    name="_method" value="DELETE">
                <button class="btn btn-danger btn-sm">
                    <i class="fa fa-trash"></i></button>
            </form>
        </td>
    </tr>
<?php
endforeach;
?>
</table>
<?php echo $pager->links('topik',
    'bs_paginate'); ?>
</div>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam menampilkan data artikel reading.

```

public function dataTopik($b=1)
{
    if ($this->request->getGet('q')) {
        $result = $this->topik->like('nm_topik',
            $this->request->getGet('q'));
    }
    else {
        $result = $this->topik;
    }
    $view = ($b==2) ? 'data_sound' :
    'data_article';
}

```

```

return view('admin/topik/'.$view , [
    'topik' => $result->where('id_bagian', $b)
                ->paginate(5, 'topik'),
    'pager' => $this->topik->pager,
]);
}

```

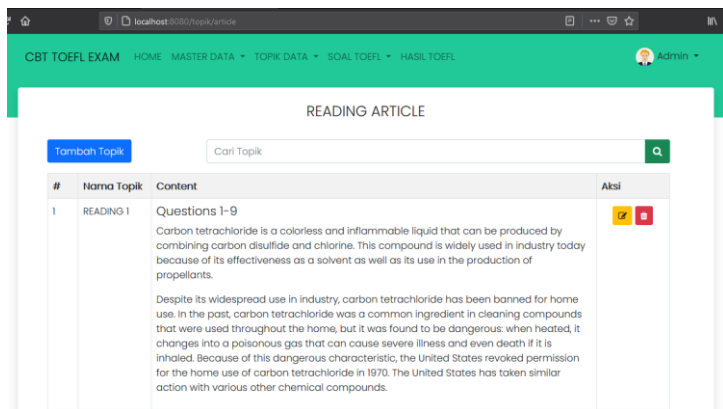
Kemudian buat route baru untuk data section yang mengarahkan pada method `dataTopik()`

```

$routes->get('/topik', 'Admin::dataTopik');
$routes->get('/topik/article',
            'Admin::dataTopik/1');

```

Kemudian kita dapat mengakses link <http://localhost:8080/topik> untuk menampilkan halaman data artikel reading setelah login sebagai admin.



Gambar 9.11 Halaman Artikel Reading

9.4.8. Hapus Artikel Reading

Untuk membuat hapus artikel reading kita perlu memodifikasi controller `Admin.php` dengan menambahkan method berikut.

```

public function deleteArticle($id='')
{

```

```
$this->topik->delete($id);
return redirect()->to('/topik');
}
```

Kemudian tambahkan *route* baru untuk mengarahkan pada *method* deleteArticle()

```
$routes->delete('/topik/(:num)',
                'Admin::deleteArticle/$1');
```

9.4.9. Tambah Artikel Reading

Tambah Artikel Reading merupakan halaman untuk membuat artikel baru pada pertanyaan-pertanyaan section reading.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada /app/Views/admin/topik/ dan berikan file dengan nama tambah_article.php

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row justify-content-center mt-3">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>TAMBAH READING ARTICLE</h4>
        </div>
        <div class="row mt-4">
          <div class="col">
            <a href="/topik/article" class="btn btn-primary"></a>
          </div>
        </div>
        <div class="row mt-3">
          <div class="col">
            <form action="/topik/article"
              method="post">
```

```

<div class="mb-3">
  <label class="form-label">
    Nama Topik</label>
    <input type="text" class="form-control
      <?=( $val->hasError('nama'))
        ? 'is-invalid' : '' ?>" name="nama">
    <div class="invalid-feedback">
      <?=( $val->getError('nama')) ?>
    </div>
  </div>
  <div class="mb-3">
    <label class="form-label">
      Article</label>
      <textarea class="form-control
        <?=( $val->hasError('article'))
          ? 'is-invalid' : '' ?>"
        rows="5" name="article"></textarea>
      <div class="invalid-feedback">
        <?=( $val->getError('article')) ?>
      </div>
    </div>
    <button type="submit" class="btn
      btn-success">Simpan</button>
  </form>
</div>
</div>
</div>
</div>
</div>
</div>
<?=( $this->endSection() ) ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan method-method berikut untuk menangani request tambah artikel reading.

```

public function tambahTopik($b='')
{
  $view = ($b==2) ? 'tambah_sound' :
'tambah_article';
  return view('admin/topik/'.$view, [

```

```

        'val' => \Config\Services::validation(),
    ]);
}
public function saveArticle()
{
    if (!$this->validate([
        'nama' => 'required',
        'article' => 'required',
    ])) {
        return redirect()->to('/topik/article/tambah')
            ->withInput();
    }
    $this->topik->insert([
        'id_bagian' => 1,
        'nm_topik' => $this->request
            ->getPost('nama'),
        'content' => $this->request
            ->getPost('article'),
    ]);
    return redirect()->to('/topik');
}

```

Kemudian buat rute baru untuk mengarahkan pada method tambahTopik() dan saveArticle()

```

$routes->get('/topik/article/tambah',
            'Admin::tambahTopik/1');
$routes->post('/topik/article',
            'Admin::saveArticle');

```

9.4.10. Edit Artikel Reading

Edit Artikel Reading merupakan halaman untuk mengubah data artikel setelah menekan tombol edit dihalaman data artikel.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code berikut pada /app/Views/admin/topik/ dan berikan view dengan nama edit_article.php

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

```

```

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row justify-content-center mt-3">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>EDIT READING ARTICLE</h4>
        </div>
        <div class="row mt-4">
          <div class="col">
            <a href="/topik/article" class="btn btn-primary">< </a>
          </div>
        </div>
        <div class="row mt-3">
          <div class="col">
            <form action="/topik/article" method="post">
              <input type="hidden" name="_method" value="PUT">
              <input type="hidden" name="id" value="<?= $t['id_topik'] ?>">
              <div class="mb-3">
                <label class="form-label">
                  Nama Topik</label>
                <input type="text" class="form-control" name="nama" value="<?= $t['nm_topik'] ?>">
              </div>
              <div class="mb-3">
                <label class="form-label">
                  Article</label>
                <textarea class="form-control" rows="3" name="article">
                  <?= $t['content'] ?></textarea>
              </div>
              <button type="submit" class="btn btn-success">Edit</button>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

</div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam mengedit data artikel reading.

```

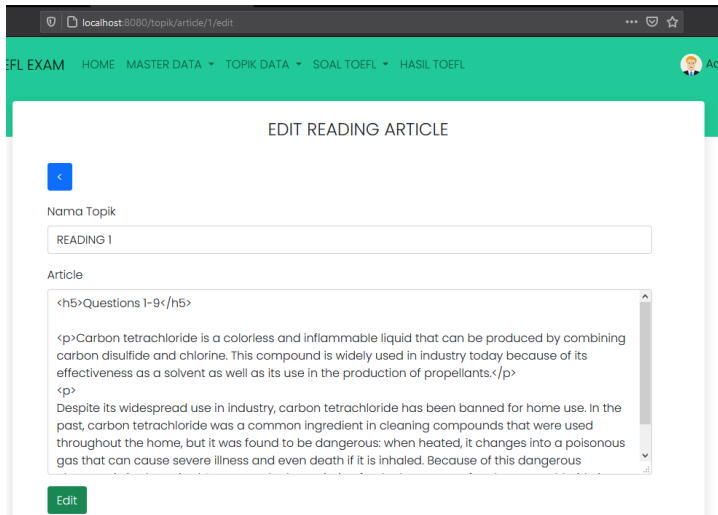
public function editTopik($b, $id='')
{
    $view = ($b==2) ? 'edit_sound' :
    'edit_article';
    return view('admin/topik/'.$view, [
        't' => $this->topik->find($id),
    ]);
}

public function saveArticle()
{
    if (!$this->validate([
        'nama' => 'required',
        'article' => 'required',
    ])) {
        return redirect()->to('/topik/article/tambah')
            ->withInput();
    }
    $this->topik->insert([
        'id_bagian' => 1,
        'nm_topik' => $this->request
            ->getPost('nama'),
        'content' => $this->request
            ->getPost('article'),
    ]);
    return redirect()->to('/topik');
}

```

Kemudian buat route baru untuk edit section yang mengarahkan pada method editTopik() dan updateArticle()

```
$routes->get('/topik/article/(:num)/edit',
'Admin::editTopik/1/$1');
$routes->put('/topik/article',
'Admin::updateArticle');
```



Gambar 9.12 Halaman Edit Artikel Reading

9.4.11. Data Audio Listening

Data Audio Listening merupakan halaman untuk menampilkan data suara untuk pertanyaan-pertanyaan pada soal listening.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada folder `/app/Views/admin/topik/` dan berikan file dengan nama `data_sound.php`

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row mt-3">
    <div class="col px-5 py-4 shadow rounded">
```

```

<div class="row mb-3">
  <div class="col text-center">
    <h4>LISTENING SOUND</h4>
  </div>
  <div class="row mt-4">
    <div class="col-3">
      <a href="/topik/sound/tambah"
        class="btn btn-primary">Tambah Topik</a>
    </div>
    <div class="col-9">
      <form action="/topik">
        <div class="input-group mb-3">
          <input type="text" name="q"
            class="form-control"
            placeholder="Cari Topik"
            value="<?=@$_GET['q'] ?>">
          <button class="btn btn-success">
            <i class="fa fa-search"></i></button>
        </div>
      </form>
    </div>
  </div>
  <div class="row">
    <div class="col">
      <table class="table table-bordered
        table-striped">
        <tr>
          <th width="50">#</th>
          <th width="120">Nama Topik</th>
          <th>Content</th>
          <th width="120">Aksi</th>
        </tr>

        <?php
        $no=0;
        foreach ($topik as $row):
          ?>
          <tr>
            <td><?=@++$no ?></td>
            <td><?=@$row['nm_topik'] ?></td>
            <td>
              <audio class="w-100" controls>
                <source src="<?=@base_url("listening/

```

```

{$row['content']}) ?>" type="audio/mpeg">
Your browser does not
support the audio element.
</audio>
</td>
<td class="text-center">
<a href="/topik/sound/
<?= $row['id_topik'] ?>/edit"
class="btn btn-warning my-1 btn-sm">
<i class="fa fa-edit"></i></a>
<form action="/topik/sound/
<?= $row['id_topik'] ?>"
class="d-inline" method="post">
<input type="hidden"
name="_method" value="DELETE">
<button class="btn btn-danger btn-sm">
<i class="fa fa-trash"></i>
</button>
</form>
</td>
</tr>
<?php
endforeach;
?>
</table>
<?php echo $pager->links('topik',
'bs_paginate'); ?>
</div>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

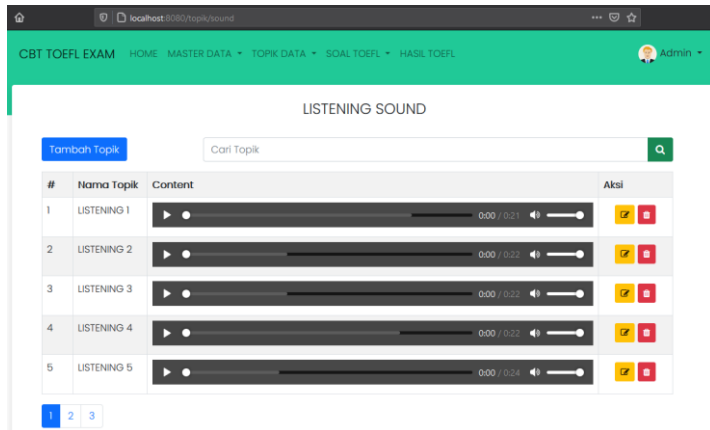
```

Untuk controller dalam menampilkan data sound listening ini, menggunakan controller yang dibuat sebelumnya, yaitu *method* `dataTopik()`

Kemudian buat *route* baru untuk data sound listening yang mengarahkan pada *method* `dataTopik()`

```
$routes->get('/topik', 'Admin::dataTopik');
$routes->get('/topik/sound',
            'Admin::dataTopik/2');
```

Kemudian kita dapat mengakses link <http://localhost:8080/topik/sound> untuk menampilkan halaman data artikel reading setelah login sebagai admin.



Gambar 9.13 Halaman Sound Listening

9.4.12. Hapus Audio Listening

Untuk membuat hapus audio listening kita perlu memodifikasi Controller Admin.php dengan menambahkan method berikut.

```
public function deleteSound($id='')
{
    unlink('listening/'. $this->topik
            ->find($id) ['content']);
    $this->topik->delete($id);
    return redirect()->to('/topik/sound');
}
```

Kemudian tambahkan *route* baru untuk mengarahkan pada *method* deleteSound()

```
$routes->delete('/topik/sound/(:num)',  
'Admin::deleteSound/$1');
```

9.4.13. Tambah Sound Listening

Tambah Audio Listening merupakan halaman untuk mengupload suara listening yang digunakan untuk pertanyaan-pertanyaan section listening.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada /app/Views/admin/topik/ dan berikan file dengan nama tambah_sound.php

```
<?= $this->extend('template/layout') ?>  
  
<?= $this->section('content') ?>  
  
<?= $this->include('template/nav_admin') ?>  
  
<div class="container main-container">  
  <div class="row justify-content-center mt-3">  
    <div class="col-10 px-5 py-4 shadow rounded">  
      <div class="row mb-3">  
        <div class="col text-center">  
          <h4>TAMBAH LISTENING SOUND</h4>  
        </div>  
        <div class="row mt-4">  
          <div class="col">  
            <a href="/topik/sound"  
              class="btn btn-primary">< </a>  
          </div>  
        </div>  
        <div class="row mt-3">  
          <div class="col">  
            <form action="/topik/sound" method="post"  
              enctype='multipart/form-data'>  
              <div class="mb-3">  
                <label class="form-label">  
                  Nama Topik</label>  
                <input type="text" class="form-control"  
                  <?= ($val->hasError('nama'))  
                    ? 'is-invalid' : '' ?>" name="nama">
```

```

        <div class="invalid-feedback">
            <?= $val->getError('nama') ?>
        </div>
    </div>
    <div class="mb-3">
        <label class="form-label">Sound</label>
        <input class="form-control
            <?= ($val->hasError('sound'))
                ? 'is-invalid' : ' ' ?>"
            type="file" name="sound">
        <div class="invalid-feedback">
            <?= $val->getError('sound') ?>
        </div>
    </div>
    <button type="submit"
        class="btn btn-success">Simpan</button>
    </form>
</div>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan method-method berikut untuk menangani request tambah sound listening.

```

public function saveSound()
{
    if (!$this->validate([
        'nama' => 'required',
        'sound' =>
        'uploaded[sound]|ext_in[sound,mp3,wav]',
    ])) {
        return redirect()->to('/topik/sound/tambah')
            ->withInput();
    }
    $sound = $this->request->getFile('sound');
    if ($sound->getError() == 0) {

```

```

    $sound->move('listening');
}
$this->topik->insert([
    'id_bagian' => 2,
    'nm_topik' => $this->request
                    ->getPost('nama'),
    'content' => $sound->getName(),
]);
return redirect()->to('/topik/sound');
}

```

Kemudian buat rute baru untuk mengarahkan pada method tambahTopik() dan saveSound()

```

$routes->get('/topik/sound/tambah',
            'Admin::tambahTopik/2');
$routes->post('/topik/sound',
            'Admin::saveSound');

```

9.4.14. Edit Audio Listening

Edit Audio Listening merupakan halaman untuk mengubah data audio listening setelah menekan tombol edit dihalaman data audio listening.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code berikut pada /app/Views/admin/topik/ dan berikan view dengan nama edit_sound.php

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row justify-content-center mt-3">
        <div class="col-10 px-5 py-4 shadow rounded">
            <div class="row mb-3">
                <div class="col text-center">
                    <h4>EDIT LISTENING SOUND</h4>
                </div>
            </div>
        </div>
    </div>

```

```

<div class="row mt-4">
  <div class="col">
    <a href="/topik/article"
      class="btn btn-primary">< </a>
    </div>
  </div>
<div class="row mt-3">
  <div class="col">
    <form action="/topik/sound" method="post"
      enctype='multipart/form-data'>
      <input type="hidden"
        name="_method" value="PUT">
      <input type="hidden" name="id"
        value="<?= $t['id_topik'] ?>">
      <div class="mb-3">
        <label class="form-label">
          Nama Topik</label>
        <input type="text" class="form-control"
          name="nama"
          value="<?= $t['nm_topik'] ?>">
      </div>
      <div class="mb-3">
        <label class="form-label">Sound</label>
        <input class="form-control"
          type="file" name="sound">
      </div>
      <button type="submit"
        class="btn btn-success">Edit</button>
    </form>
  </div>
</div>
</div>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam memperbaharui data audio.

```

public function updateSound()
{
    $id = $this->request->getPost('id');
    $fileName = $this->topik
        ->find($id)['content'];
    $sound = $this->request->getFile('sound');
    if ($sound->getError() == 0) {
        unlink('listening/'.$fileName);
        $sound->move('listening');
        $fileName = $sound->getName();
    }
    $this->topik->update($id, [
        'id_bagian' => 2,
        'nm_topik' => $this->request
            ->getPost('nama'),
        'content' => $fileName,
    ]);
    return redirect()->to('/topik/sound');
}

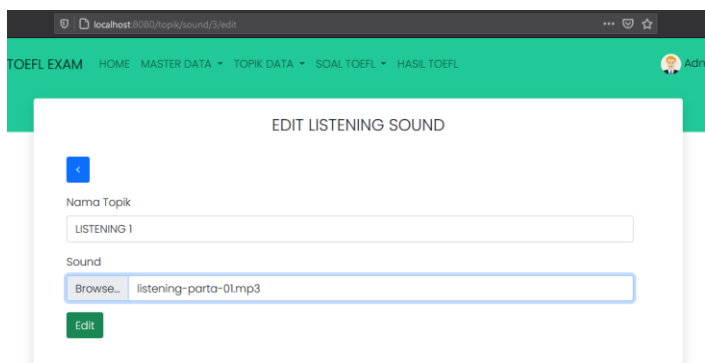
```

Kemudian buat route baru untuk edit section yang mengarahkan pada method `editTopik()` dan `updateSound()`

```

$routes->get('/topik/sound/tambah',
    'Admin::tambahTopik/2');
$routes->put('/topik/sound',
    'Admin::updateSound');

```



Gambar 9.14 Halaman Edit Sound Listening

9.4.15. Data Soal

Data soal merupakan halaman untuk menampilkan data soal dari salah satu section dalam bentuk tabel dan membuat pengelolaan data tersebut.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code tersebut pada `/app/Views/admin/soal/` dan berikan file dengan nama `data.php`

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row mt-3">
    <div class="col px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>SOAL <?= $sec ?></h4>
        </div>
        <div class="row mt-4">
          <div class="col-3">
            <a href="/soal/tambah" class="btn btn-primary">Tambah Soal</a>
          </div>
          <div class="col-9">
            <form action="/soal">
              <div class="input-group mb-3">
                <input type="text" name="q"
                  class="form-control" placeholder="Cari Soal" value="<?= @$_GET['q'] ?>">
                <button class="btn btn-success">
                  <i class="fa fa-search"></i></button>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="row">
    <div class="col">
```

```

<table class="table table-bordered
table-striped">
<tr>
<th>No. Soal</th>
<th>Topik</th>
<th>Soal</th>
<th>Opsi A</th>
<th>Opsi B</th>
<th>Opsi C</th>
<th>Opsi D</th>
<th>Jawaban</th>
<th>Aksi</th>
</tr>
<?php
foreach ($soal as $row):
    ?>
    <tr>
        <td><?= $row['no_soal'] ?></td>
        <td width="90px">
            <?= $row['nm_topik'] ?? '-' ?></td>
        <td><?= $row['soal'] ?></td>
        <td><?= $row['opsi_a'] ?></td>
        <td><?= $row['opsi_b'] ?></td>
        <td><?= $row['opsi_c'] ?></td>
        <td><?= $row['opsi_d'] ?></td>
        <td><?= $row['jawaban'] ?></td>
        <td class="text-center">
            <a href="/soal
            /<?= $row['id_soal'] ?>/edit"
            class="btn btn-warning my-1 btn-sm">
                <i class="fa fa-edit"></i></a>

            <form action="/soal
                /<?= $row['id_soal'] ?>"
                class="d-inline" method="post">
                <input type="hidden"
                    name="_method" value="DELETE">
                <button class="btn btn-danger btn-sm">
                    <i class="fa fa-trash"></i></button>
            </form>
        </td>
    </tr>
<?php

```

```

        endforeach;
        ?>
    </table>
    <?php echo $pager->links('user',
                            'bs_paginate'); ?>

    </div>
</div>
</div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian pada controller `Admin.php` lakukan modifikasi dengan menambahkan code berikut untuk method yang akan menangani request dalam menampilkan data soal.

```

public function dataSoal($t=1)
{
    if ($t == 1) $sec = 'READING';
    elseif ($t == 2) $sec = 'LISTENING';
    elseif ($t == 3) $sec = 'STRUCTURE/WRITTEN ';
    return view('admin/soal/data', [
        'soal' => $this->soal->getSoal()
            ->where(['t_soal.id_bagian' => $t])
            ->paginate(10, 'user'),
        'pager' => $this->soal->pager,
        'sec' => $sec,
    ]);
}

```

Kemudian buat route baru untuk halaman data soal yang mengarahkan pada method `dataSoal()`

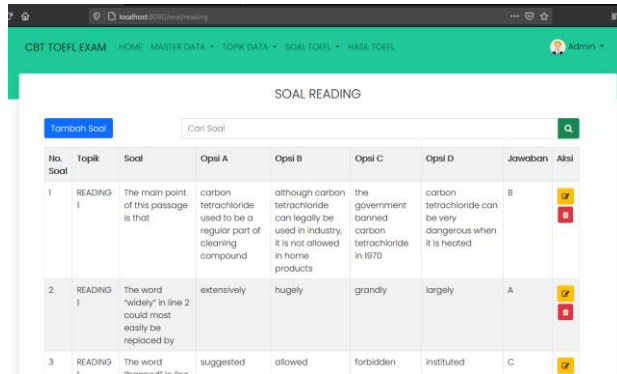
```

$routes->get('/soal', 'Admin::dataSoal');
$routes->get('/soal/reading',
            'Admin::dataSoal/1');
$routes->get('/soal/listening',
            'Admin::dataSoal/2');
$routes->get('/soal/structure',

```

```
'Admin::dataSoal/3');
```

Kemudian kita dapat mengakses link <http://localhost:8080/soal> untuk menampilkan halaman data soal pada panel admin.



No. Soal	Topik	Soal	Opsil A	Opsil B	Opsil C	Opsil D	Jawaban	Alas
1	READING I	The main point of this passage is that.	carbon tetrachloride used to be a regular part of cleaning compound	although carbon tetrachloride can legally be used in industry, it is not allowed in home products	the government banned carbon tetrachloride in 1970	carbon tetrachloride can be very dangerous when it is heated	B	
2	READING I	The word "widely" in line 2 could most easily be replaced by	extensively	hugely	grandly	largely	A	
3	READING I	The word "thwarted" in line	suggested	allowed	forbidden	instituted	C	

Gambar 9.15 Halaman Data Soal

9.4.16. Hapus Soal

Untuk membuat hapus soal kita perlu memodifikasi controller Admin.php dengan menambahkan method berikut.

```
public function deleteSoal($id='')
{
    $this->soal->delete($id);
    return redirect()->to('/soal');
}
```

Kemudian tambahkan route baru untuk request hapus soal yang mengarahkan pada method deleteSoal()

```
$routes->delete('/soal/(:num)',
                'Admin::deleteSoal/$1');
```

9.4.17. Tambah Soal

Tambah soal merupakan halaman untuk menambah soal baru setelah menekan tombol tambah soal di halaman data soal. Dalam

halaman tambah soal, akan terdapat select box yang berisi section, setelah memilih section, kemudian memilih topik yang dapat berupa artikel maupun sound listening. Untuk menampilkan data topik pada select box yang tertrigger setelah memilih section, maka digunakan javascript. Berikut merupakan code javascript untuk menangani select box, simpan code berikut pada folder /public/assets/js dengan nama file select.js

```
document.getElementById("select").onchange =
function() {
    val = this.value;
    if (val == 1 || val == 2) {

document.getElementById('formTopik').style.display = 'block';
        fetch(`http://localhost:8080/topikby/${ val
}` , {
            method: "get",
            headers: {
                "Content-Type": "application/json",
                "X-Requested-With": "XMLHttpRequest"
            }
        }).then(response => {
            console.log(response.status);
            return response.json();
        }).then(data => {
            options = "";
            for (var i = 0; i < data.length; i++) {
                options += `<option value="${{
data[i].id_topik }}"> ${ data[i].nm_topik }
</option>`;
            }
            document.getElementById('topik').innerHTML =
options;
        }).catch(err => {
            console.log('Fetch Error :-S', err);
        });
    } else {
```

```

document.getElementById('formTopik').style.display = 'none';
    }
}

```

Kemudian buat view baru untuk menampilkan form untuk menambah soal. Simpan code tersebut pada /app/Views/admin/soal/ dan berikan file dengan nama tambah.php

```

<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row justify-content-center mt-3">
        <div class="col-10 px-5 py-4 shadow rounded">
            <div class="row mb-3">
                <div class="col text-center">
                    <h4>TAMBAH SOAL</h4>
                </div>
                <div class="row mt-4">
                    <div class="col">
                        <a href="/soal" class="btn btn-primary">
                            < </a>
                    </div>
                </div>
                <div class="row mt-3">
                    <div class="col">
                        <form action="/soal" method="post">
                            <div class="mb-3">
                                <label class="form-label">
                                    Section</label>
                                <select class="form-select"
                                    id="select" name="section">
                                    <option value="">--</option>
                                <?php
                                    foreach ($topik as $r) {
                                        echo "<option value='{$r['id_bagian']}'>

```

```

        {$r['nm_section']}</option>";
    }
    ?>
</select>
</div>
<div class="mb-3" id="formTopik">
    <label class="form-label">Topik</label>
    <select class="form-select"
        id="topik" name="topik"></select>
</div>
<div class="mb-3">
    <label class="form-label">
        No Soal</label>
    <input type="text" class="form-control"
        name="no">
</div>
<div class="mb-3">
    <label class="form-label">Soal</label>
    <textarea class="form-control"
        rows="3" name="soal"></textarea>
</div>
<div class="mb-3">
    <label class="form-label">
        Opsi A</label>
    <input type="text" class="form-control"
        name="a">
</div>
<div class="mb-3">
    <label class="form-label">
        Opsi B</label>
    <input type="text" class="form-control"
        name="b">
</div>
<div class="mb-3">
    <label class="form-label">
        Opsi C</label>
    <input type="text" class="form-control"
        name="c">
</div>
<div class="mb-3">
    <label class="form-label">
        Opsi D</label>
    <input type="text" class="form-control"

```

```

        name="d">
    </div>
    <div class="mb-3">
        <label class="form-label">
            Jawaban</label>
        <select class="form-select" name="jawab">
            <option value="A">A</option>
            <option value="B">B</option>
            <option value="C">C</option>
            <option value="D">D</option>
        </select>
    </div>
    <button type="submit"
        class="btn btn-success">Simpan</button>
    </form>
</div>
</div>
</div>
</div>
</div>
</div>

<script src="<?= base_url('assets
                        /js/select.js') ?>"></script>

<?= $this->endSection() ?>

```

Kemudian pada controller `Admin.php` lakukan modifikasi dengan menambahkan method-method berikut yang akan menangani request dalam menampilkan form dan menyimpan soal.

```

public function tambahSoal()
{
    return view('admin/soal/tambah', [
        val => \Config\Services::validation(),
        'topik' => $this->bagian->find()
    ]);
}

public function saveSoal()
{
    $this->soal->insert([

```

```

'id_bagian' => $this->request
                ->getPost('section'),
'id_topik' => $this->request
                ->getPost('topik'),
'no_soal' => $this->request->getPost('no'),
'soal' => $this->request->getPost('soal'),
'opsi_a' => $this->request->getPost('a'),
'opsi_b' => $this->request->getPost('b'),
'opsi_c' => $this->request->getPost('c'),
'opsi_d' => $this->request->getPost('d'),
'jawaban' => $this->request
                ->getPost('jawab'),

]);
return redirect()->to('/soal');
}

public function getTopicBySection($id='')
{
    echo json_encode($this->topik
        ->where('id_bagian', $id)->find());
}

```

Kemudian buat route baru untuk halaman form tambah soal dan route untuk menyimpan soal yang mengarahkan pada method yang bersangkutan.

```

$routes->get('/soal/tambah',
                'Admin::tambahSoal');
$routes->get('/topikby/(:num)',
                'Admin::getTopicBySection/$1');
$routes->post('/soal', 'Admin::saveSoal');

```

9.4.18. Edit Soal

Edit soal merupakan halaman untuk mengubah data soal setelah menekan tombol edit di halaman data soal.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code berikut pada /app/Views/admin/soal/ dan berikan view dengan nama edit.php

```
<?= $this->extend('template/layout') ?>
```

```

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
  <div class="row justify-content-center mt-3">
    <div class="col-10 px-5 py-4 shadow rounded">
      <div class="row mb-3">
        <div class="col text-center">
          <h4>EDIT SOAL</h4>
        </div>
        <div class="row mt-4">
          <div class="col">
            <a href="/soal" class="btn btn-primary">
              < </a>
          </div>
        </div>
        <div class="row mt-3">
          <div class="col">
            <form action="/soal" method="post">
              <input type="hidden"
                name="_method" value="PUT">
              <input type="hidden" name="id"
                value="<?= $s['id_soal'] ?>">
              <div class="mb-3">
                <label class="form-
label">Section</label>
                <select class="form-select" id="select"
name="section">
                  <?php
                    foreach ($topik as $r):
                      ?>
                      <option value='<?= $r['id_bagian']
?>' <?= $r['id_bagian']==$s['id_bagian'] ?
'selected' : '' ?> > <?= $r['nm_section']
?></option>";
                      <?php
                        endforeach;
                      ?>
                    </select>
                  </div>
                <div class="mb-3" id="formTopik">

```

```

        <label class="form-label">Topik</label>
        <select class="form-select" id="topik"
name="topik">
            <option value="<?= $s['id_topik']
?>"><?= $s['nm_topik'] ?></option>
        </select>
    </div>
    <div class="mb-3">
        <label class="form-label">No
Soal</label>
        <input type="text" class="form-control"
name="no" value="<?= $s['no_soal'] ?>">
    </div>
    <div class="mb-3">
        <label class="form-label">Soal</label>
        <textarea class="form-control" rows="3"
name="soal"><?= $s['soal'] ?></textarea>
    </div>
    <div class="mb-3">
        <label class="form-label">Opsi
A</label>
        <input type="text" class="form-control"
name="a" value="<?= $s['opsi_a'] ?>">
    </div>
    <div class="mb-3">
        <label class="form-label">Opsi
B</label>
        <input type="text" class="form-control"
name="b" value="<?= $s['opsi_b'] ?>">
    </div>
    <div class="mb-3">
        <label class="form-label">
Opsi C</label>
        <input type="text" class="form-control"
name="c" value="<?= $s['opsi_c'] ?>">
    </div>
    <div class="mb-3">
        <label class="form-label">
Opsi D</label>
        <input type="text" class="form-control"
name="d" value="<?= $s['opsi_d'] ?>">
    </div>
    <div class="mb-3">

```

```

        <label class="form-label">
            Jawaban</label>
        <select class="form-select" name="jawab">
            <option value="A" <?= $s['jawaban']=='A'
            ? 'selected' : '' ?>>A</option>
            <option value="B" <?= $s['jawaban']=='B'
            ? 'selected' : '' ?>>B</option>
            <option value="C" <?= $s['jawaban']=='C'
            ? 'selected' : '' ?>>C</option>
            <option value="D" <?= $s['jawaban']=='D'
            ? 'selected' : '' ?>>D</option>
        </select>
    </div>
    <button type="submit"
        class="btn btn-success">Edit</button>
</form>
</div>
</div>
</div>
</div>
</div>
</div>

<script src="<?= base_url('assets
                        /js/select.js') ?>"></script>

<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan method-method berikut yang akan menangani request untuk menampilkan form edit dan memperbaharui soal.

```

public function editSoal($id='')
{
    return view('admin/soal/edit', [
        's' => $this->soal->getSoal()->find($id),
        'topik' => $this->bagian->find()
    ]);
}
public function updateSoal()
{
    $this->soal->update($this->request

```

```

        ->getPost('id'), [
'id_topik' => $this->request
        ->getPost('topik'),
'id_bagian' => $this->request
        ->getPost('section'),
'no_soal' => $this->request->getPost('no'),
'soal' => $this->request->getPost('soal'),
'opsi_a' => $this->request->getPost('a'),
'opsi_b' => $this->request->getPost('b'),
'opsi_c' => $this->request->getPost('c'),
'opsi_d' => $this->request->getPost('d'),
'jawaban' => $this->request
        ->getPost('jawab'),
]);
return redirect()->to('/soal');
}

```

Kemudian buat route baru untuk halaman form edit soal berdasarkan id dan route untuk mengubah soal untuk mengarahkan pada method yang bersangkutan.

```

$routes->get('/soal/(:num)/edit',
    'Admin::editSoal/$1');
$routes->put('/soal', 'Admin::updateSoal');

```

Kemudian kita dapat mengakses link <http://localhost:8080/1/edit> untuk mengubah data soal dengan id 1.

Gambar 9.16 Halaman Edit Soal

9.4.19. Hasil Toefl

Hasil Toefl merupakan halaman untuk menampilkan hasil ujian toefl dari semua peserta yang juga dapat difilter berdasarkan tanggal tesnya.

Berikut code yang harus ditambahkan untuk tampilan viewnya, simpan code berikut pada `/app/Views/admin/` dengan nama `hasil.php`

```
<?= $this->extend('template/layout') ?>

<?= $this->section('content') ?>

<?= $this->include('template/nav_admin') ?>

<div class="container main-container">
    <div class="row mt-3">
        <div class="col px-5 py-4 shadow rounded">
            <div class="row mb-3">
                <div class="col text-center">
                    <h4>HASIL TOEFL</h4>
                </div>
            <div class="row mt-4">
                <div class="col-3">
                    <form action="/hasil">
                        <div class="input-group mb-3">
                            <input type="date" name="q"
                                class="form-control"
                                value="<?= @$_GET['q'] ?>">
                            <button class="btn btn-success">
                                <i class="fa fa-search"></i></button>
                        </div>
                    </form>
                </div>
            </div>
            <div class="row">
                <div class="col">
                    <table class="table table-bordered
                        table-striped">
                        <tr>
                            <th width="50">#</th>
                            <th>Nama</th>
```

```

        <th width="120">Reading</th>
        <th width="120">Listening</th>
        <th width="120">Structure</th>
        <th width="120">Score</th>
    </tr>
    <?php
    $no=0;
    foreach ($hasil as $row):
        ?>
        <tr>
            <td><?= ++$no ?></td>
            <td><?= $row['nama'] ?></td>
            <td><?= $row['r'] ?></td>
            <td><?= $row['l'] ?></td>
            <td><?= $row['s'] ?></td>
            <td>
                <?= ($row['r'] && $row['l'] && $row['s'])
                ? (($row['r']+$row['l']+$row['s'])/3)*10
                : ' ' ?>
            </td>
        </tr>
        <?php
        endforeach;
        ?>
    </table>
    <?php echo $pager->links('hasil',
                                'bs_paginate'); ?>

    </div>
</div>
</div>
</div>
</div>
</div>

<?= $this->endSection() ?>

```

Kemudian pada controller Admin.php lakukan modifikasi dengan menambahkan code berikut untuk mengambil data dan menampilkannya pada view hasil.

```

public function Hasil()
{

```

```

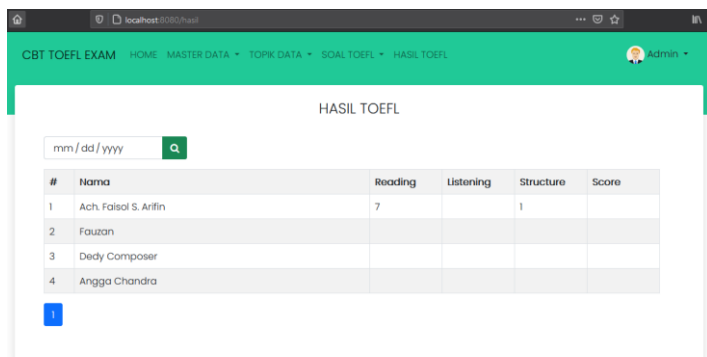
if ($this->request->getGet('q')) {
    $result = $this->user->getHasil()
        ->where('tgl_start', $this
            ->request->getGet('q'));
}
else{
    $result = $this->user->getHasil();
}
return view('admin/hasil' , [
    'hasil' => $result->paginate(30, 'hasil'),
    'pager' => $this->user->pager,
]);
}

```

Kemudian buat route baru untuk halaman hasil toefl yang diambil dari method Hasil() pada controller Admin.php

```
$routes->get('/hasil', 'Admin::Hasil');
```

Kemudian kita dapat mengakses link <http://localhost:8080/hasil> untuk menampilkan data hasil toefl pada panel admin.



Gambar 9.17 Halaman Hasil Toefl

Untuk source code dari aplikasi toefl sederhana tersebut, juga dapat di clone melalui link <https://github.com/faisolarifin/ujian-toefl-ci-4>.

BAB 10

UPLOAD PROJECT

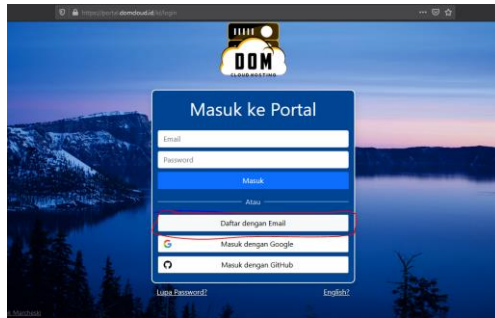
CODEIGNITER KE WEB HOSTING

Web hosting adalah jasa layanan internet yang menyediakan resource server untuk menempatkan dan menyimpan file website sehingga memungkinkan untuk dapat diakses oleh publik. File yang disimpan dalam hosting biasanya berupa, script, gambar, video, suara, dan lain sebagainya. Jika dianalogikan pada sebuah bangunan, web hosting merupakan tempat hunian yang bisa menyimpan segala peralatan dan memiliki alamat untuk dituju. Sebuah server hosting memiliki IP address public tertentu yang terhubung dengan jaringan internet berkecepatan tinggi. Ada beberapa jenis layanan web hosting, seperti *shared hosting*, *dedicated hosting*, *virtual private server*, dan *colocation server* yang dapat digunakan sesuai dengan kebutuhan.

Dalam hal ini, kita akan menggunakan layanan hosting dari domcloud.id yang menyediakan layanan *shared hosting* gratis yang cukup mempuni untuk trafik website skala rendah. Dalam layanan gratis tersebut, kita akan mendapatkan 256 MB Storage dan 12 GB Bandwith, dan juga diberikan fasilitas SSH Login, FTP, Webmin, phpMyAdmin/phpPgAdmin yang memudahkan kita dalam mengelola web. Web hosting tersebut menggunakan nginx sebagai web servernya, sehingga membuat web lebih cepat saat diakses.

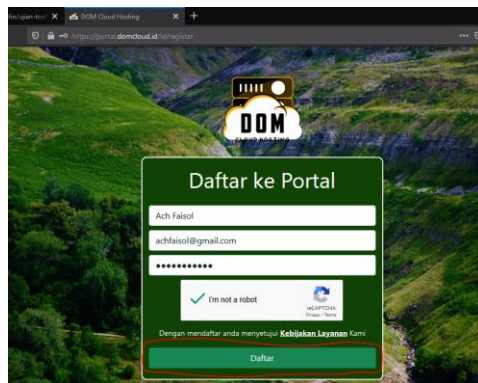
10.1. MENDAFTAR LAYANAN WEB HOSTING

Untuk mendaftar layanan web hosting tersebut, kita dapat mengunjungi link portalnya di <https://portal.domcloud.id/id/login>, kemudian kita dapat menekan menu “Daftar dengan Email” untuk membuat akun pada web hosting tersebut.



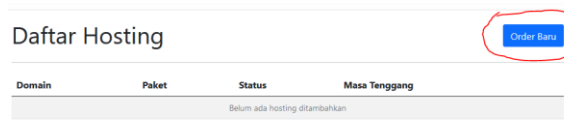
Gambar 10.1 Login Portal domcloud

Kemudian kita akan diarahkan ke halaman pendaftaran untuk mengisi nama, email, dan password, tekan tombol Daftar.



Gambar 10.2 Daftar Portal domcloud

Kemudian kita diarahkan ke halaman Daftar Hosting yang berisi list domain yang kita miliki. Untuk membuat domain baru, kita tekan tombol Order Baru.



Gambar 10.3 Daftar Hosting

Setelah itu akan diarahkan untuk mengisi username portal, memilih paket, dan domain. Kita dapat mengisi form Username Portal sesuai dengan kebutuhan dan pilih paket Freedom dan tekan tombol Daftar Sekarang.

Harga Paket		Kosong	
Paket	Free		
Pilih jenis paket	Freedom		
Jangka tahun	1		
Data Add-ons (GB)	0		
Total Pembayaran		Tanggal Kadaluarsa	
Disk Space	256 MB	Bandwidth	12 GB
Add-ons	0 GB		

Gambar 10.4 Membuat Hosting Baru

Setelah proses pembuatan hosting selesai, kita akan diarahkan ke halaman detail host, kita dapat menekan tab Login untuk menampilkan informasi akun, seperti webmin, database, SSH, dan FTP.

Webmin	Database	SSH	FTP
URL: https://sga.domcloud.id:2443	Hostname: sga.domcloud.id	Run: ssh	Hostname: sga.domcloud.id
Username: toeflexam	Username: toeflexam	Username: toeflexam@sga.domcloud.id	Username: toeflexam
Password: U9K9Ox4S+...	Password: U9K9Ox4S+...	Password: U9K9Ox4S+...	Password: U9K9Ox4S+...
Buka Portal / Auto Login	phpMyAdmin / phpMyAdmin	Web SSH	
Punya masalah?	Punya masalah?	Punya masalah?	

Gambar 10.5 Daftar Akun Login

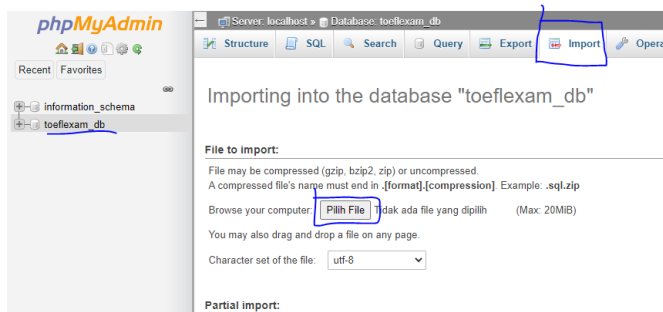
10.2. IMPORT DATABASE KE WEB HOSTING

Setelah berhasil membuat hosting baru, kita dapat menekan menu phpMyAdmin pada tab Login, untuk mengelola database. Kemudian kita akan diarahkan ke halaman login portal phpMyAdmin dan masukkan username dan password seperti yang sudah tertera dalam informasi host pada bagian akun Database.



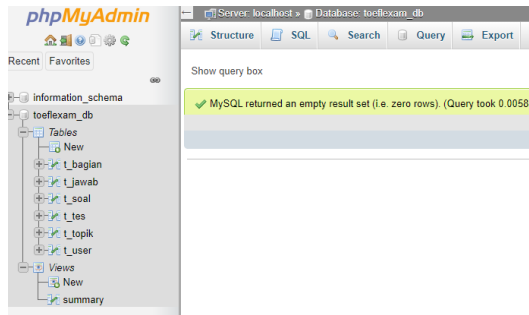
Gambar 10. 6 Login Portal phpMyAdmin

Setelah login berhasil, kita akan diarahkan ke halaman dashboard phpMyAdmin. Ketika membuat hosting baru, secara otomatis kita dibuatkan satu database seperti nama username portal, dengan sufiks `_db`. Kemudian kita tekan databasenya `"toeflexam_db"`, pilih menu Import, dan browser file database sql dari aplikasi toefl.



Gambar 10.7 Import File Database

Setelah database berhasil diimport, kita akan mendapatkan beberapa tabel seperti, pada phpMyAdmin di komputer local.



Gambar 10. 8 Tabel dan Views Database

10.3. INSTALASI DAN KONFIGURASI PROJECT

Terdapat dua cara yang dapat dilakukan untuk instalasi tersebut, kita dapat menggunakan layanan SSH maupun FTP, namun dalam hal ini kita akan menggunakan layanan SSH untuk menginstalasi dan mengkonfigurasi project.

Pertama kita buka terminal/command prompt dan masukkan perintah “ssh username@hostname” dan tekan enter. Informasi mengenai username, password, dan hostname berada didaftar akun login.

```
toeflexam@sga:~
Microsoft Windows [Version 10.0.18363.1440]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\FAISOL>ssh toeflexam@sga.domcloud.id
toeflexam@sga.domcloud.id's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sun Apr 25 07:31:10 2021
[toeflexam@sga ~]$ ls
cgi-bin  homes  Maildir  ssl.ca  ssl.combined  ssl.key  virtualmin-backup
etc      logs  public_html  ssl.cert  ssl.everything  tmp
[toeflexam@sga ~]$
```

Gambar 10.9 Login SSH

Untuk menginstalasi project melalui SSH, kita harus mengupload project kita ke github terlebih dahulu, agar nantinya project tersebut dapat diclone dari server.

Kemudian kita jalankan perintah “git clone https://github.com/faisolarifin/ujian-toefl-ci-4.git” untuk mengcloning project codeigniter yang ada digithub.

```
[toeflexam@sga ~]$ git clone https://github.com/faisolarifin/ujian-toefl-ci-4.git
Cloning into 'ujian-toefl-ci-4'...
remote: Enumerating objects: 194, done.
remote: Counting objects: 100% (194/194), done.
remote: Compressing objects: 100% (146/146), done.
remote: Total 194 (delta 49), reused 183 (delta 38), pack-reused 0
Receiving objects: 100% (194/194), 4.12 MiB | 4.10 MiB/s, done.
Resolving deltas: 100% (49/49), done.
[toeflexam@sga ~]$ ls
cgi-bin  homes  Maildir  ssl.ca  ssl.combined  ssl.key  ujian-toefl-ci-4
etc      logs   public_html  ssl.cert  ssl.everything  tmp      virtualmin-backup
[toeflexam@sga ~]$
```

Gambar 10.10 Clone Project

Kemudian kita jalankan perintah untuk merename folder project dan memindahkan isi dari folder public ke folder public_html yang di baca server sebagai directory utama

```
"mv ujian-toefl-ci-4/ toeflexam"
"mv toeflexam/public/* public_html/"
"cd public_html"
"ls"
```

```
[toeflexam@sga ~]$ mv ujian-toefl-ci-4/ toeflexam
[toeflexam@sga ~]$ mv toeflexam/public/* public_html/
[toeflexam@sga ~]$ cd public_html/
[toeflexam@sga public_html]$ ls
assets  favicon.ico  index.html  index.php  listening  robots.txt
[toeflexam@sga public_html]$
```

Gambar 10.11 Rename Folder Project

Kemudian kita ubah syntax php pada file index.php dengan menjalankan perintah "nano index.php", dan temukan statement berikut.

```
// Ensure the current directory is pointing to the front controller's directory
chdir(_DIR_);

// Load our paths config file
// This is the line that might need to be changed, depending on your folder structure.
require realpath(FCPATH . '../app/Config/Paths.php') ?: FCPATH . '../app/Config/Paths.php';
// ^^^ Change this if you move your application folder

$paths = new Config\Paths();
```

Gambar 10.12 File Index

Kemudian ubah statement berikut

```
require realpath(FCPATH .
'../app/Config/Paths.php') ?: FCPATH .
'../app/Config/Paths.php';
```

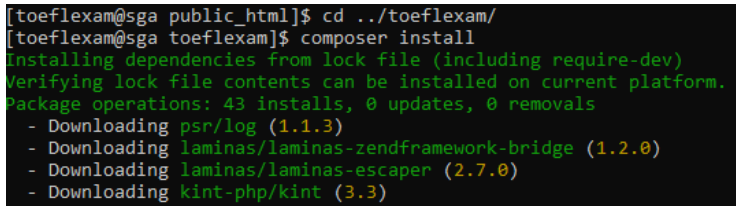
menjadi

```
require realpath(FCPATH .  
'../toeflexam/app/Config/Paths.php') ?: FCPATH .  
'../toeflexam/app/Config/Paths.php';
```

Setelah itu simpan kembali file tersebut dengan menekan “ctrl + x + y + enter”.

Kemudian masuk pada folder project dan lakukan instalasi package yang dibutuhkan

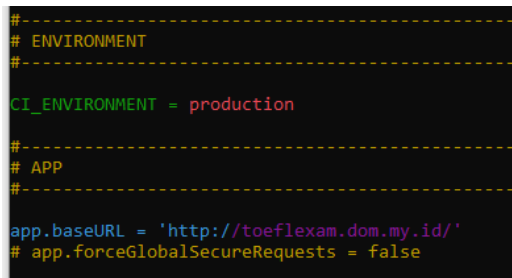
```
"cd ../toeflexam/"  
"composer install"
```



```
[toeflexam@sga public_html]$ cd ../toeflexam/  
[toeflexam@sga toeflexam]$ composer install  
Installing dependencies from lock file (including require-dev)  
Verifying lock file contents can be installed on current platform.  
Package operations: 43 installs, 0 updates, 0 removals  
- Downloading psr/log (1.1.3)  
- Downloading laminas/laminas-zendframework-bridge (1.2.0)  
- Downloading laminas/laminas-escaper (2.7.0)  
- Downloading kint-php/kint (3.3)
```

Gambar 10.13 Instalasi Packages

Ketika mengupload sebuah project codeigniter ke github, biasanya file enviroment tidak ikut terupload, maka kita perlu membuat file environment dengan perintah “nano .env”, dan copykan perintah dari file envirotnment komputer local dan lakukan konfigurasi pada bagian CI_ENVIRONMENT, baseURL.



```
#-----  
# ENVIRONMENT  
#-----  
  
CI_ENVIRONMENT = production  
  
#-----  
# APP  
#-----  
  
app.baseURL = 'http://toeflexam.dom.my.id/'  
# app.forceGlobalSecureRequests = false
```

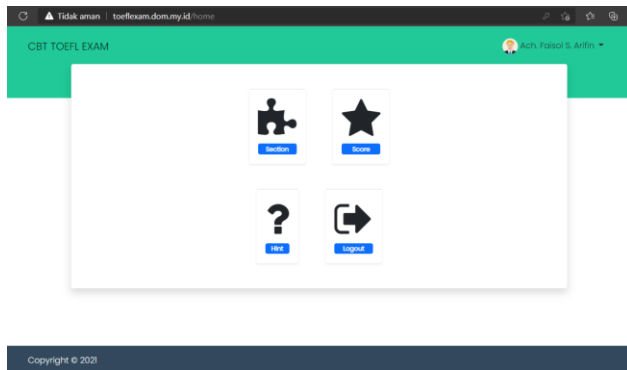
Gambar 10.14 Setting Base Url

Setelah itu lakukan konfigurasi juga pada bagian database dengan memasukkan hostname, username, password, dan database seperti yang terdapat dalam informasi akun login.

```
#-----  
# DATABASE  
#-----  
  
database.default.hostname = sga.domcloud.id  
database.default.database = toeflexam_db  
database.default.username = toeflexam  
database.default.password = U19k90s4S+__  
database.default.DBDriver = MySQLi
```

Gambar 10.15 Setting Database Enviroment

Setelah konfigurasi telah dilakukan, maka kita dapat mengakses link domain dari hosting yang kita buat, dalam hal ini domain yang digunakan yaitu *http://toeflexam.dom.my.id*



Gambar 10.16 Aplikasi di Web Hosting

Demikian tutorial mengenai CodeIgniter 4 hingga melakukan hosting project pada sebuah layanan web hosting gratis. Tentunya dengan menggunakan CodeIgniter, proses pengembangan aplikasi web akan terasa lebih cepat dan lebih mengefisien waktu. Kita juga dapat mengeksplorasi lebih jauh tentang CodeIgniter dengan mengunjungi portal dokumentasinya, yaitu di <https://codeigniter4.github.io/> atau bergabung dengan komunitas CodeIgniter. Semoga Bermanfaat 😊.

DAFTAR PUSTAKA

- [1] https://codeigniter.com/user_guide/intro/index.html
- [2] <https://www.sitepoint.com/rasmus-lerdorf-php-frameworks-think-again/>
- [3] <https://dev.to/jonathanlamim/10-differences-between-codeigniter-3-and-4-5526>
- [4] <https://codeigniter.com/userguide3/overview/mvc.html>
- [5] <https://codeigniter4.github.io/>